# Modul 8: HMVC, *Authorization*, dan AJAX pada Laravel

## 8.1 Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep HMVC (*Hierarchical Model View Controller*) dan menerapkannya pada Laravel menggunakan *package* nwidart/laravel-modules.
2. Memahami perbedaan *authentication* (siapa pengguna) dan *authorization* (apa yang boleh dilakukan pengguna), serta menerapkan *middleware* berbasis peran (*role*) *admin* dan *user*.
3. Mampu membuat fitur *register*, *login*, dan *logout* secara manual di luar modul HMVC.
4. Mampu membangun fitur CRUD dengan mekanisme refresh halaman (*synchronous*) di dalam modul HMVC yang hanya dapat diakses oleh *admin*.
5. Mampu membangun fitur CRUD tanpa refresh halaman (AJAX/asynchronous) di dalam modul HMVC menggunakan jQuery dan respons JSON.
6. Mampu menambahkan fitur *upload file* pada CRUD AJAX menggunakan *FormData*, menyimpan berkas di *storage*, serta menampilkan, mengganti, dan menghapus berkas tersebut melalui antarmuka aplikasi.
7. Mampu menerapkan validasi menggunakan *Form Request* serta menampilkan pesan berhasil dan pesan *error* yang jelas, baik pada *form* biasa maupun pada *form* AJAX.

## 8.2 Alat dan Bahan

Untuk menjalankan modul ini, diperlukan:

1. **Perangkat Keras:** komputer atau laptop dengan sistem operasi Windows/Linux/MacOS, prosesor minimal 1 GHz, RAM 4 GB (disarankan 8 GB), dan ruang penyimpanan kosong minimal 2 GB.
2. **Perangkat Lunak:**
   - XAMPP atau Laragon sebagai *server* lokal (Apache & MySQL/MariaDB), termasuk phpMyAdmin untuk mengelola *database*.
   - Composer sebagai *dependency manager* PHP.
   - PHP versi 8.2 atau lebih tinggi.
   - Laravel *Framework* 12 (pada modul ini digunakan versi 12.70).
   - *Package* nwidart/laravel-modules versi 12 beserta wikimedia/composer-merge-plugin (terpasang otomatis bersama *package* tersebut).
   - Bootstrap 5.3, Bootstrap Icons, dan jQuery 3.6 yang dimuat melalui CDN, sehingga tidak memerlukan Node.js dan npm run dev.
   - Pest sebagai *framework* pengujian otomatis (sudah tersedia pada *project* Laravel baru).
   - Teks Editor/IDE seperti Visual Studio Code, serta *browser web* seperti Google Chrome yang dilengkapi DevTools (untuk melihat *request* AJAX pada *tab Network*).
   - Beberapa *file* contoh untuk uji coba *upload*, misalnya gambar (.jpg/.png) dan dokumen (.pdf) berukuran di bawah 2 MB, serta satu *file* .zip untuk menguji penolakan format.

## 8.3 Tugas Pendahuluan

1. Apa yang dimaksud dengan HMVC? Jelaskan perbedaannya dengan MVC biasa dan sebutkan keuntungan memecah aplikasi menjadi beberapa modul!
2. Jelaskan perbedaan *authentication* dan *authorization*, lalu berikan masing-masing satu contoh penerapannya pada aplikasi *web*!

3. Apa fungsi *middleware* pada Laravel? Bagaimana sebuah *middleware* dapat menerima parameter, misalnya role:admin?
4. Apa yang dimaksud dengan AJAX? Jelaskan perbedaan *request synchronous* dan *asynchronous* dari sisi pengguna maupun dari sisi *server*!
5. Mengapa *request* AJAX dengan *method* POST, PUT, atau DELETE ke Laravel wajib menyertakan *token* CSRF, dan bagaimana cara mengirimkannya?
6. Jelaskan arti kode status HTTP 403, 404, 419, dan 422, serta kapan masing-masing kode tersebut muncul pada aplikasi Laravel!
7. Mengapa *form* AJAX yang mengirim *file* harus menggunakan objek *FormData* (bukan *serialize()*), dan apa fungsi opsi *processData: false* serta *contentType: false* pada jQuery?
8. Apa perbedaan menyimpan berkas di *folder* storage/app/public dengan menyimpan *path* berkas di *database*? Mengapa perintah php artisan storage:link diperlukan?

## 8.4 Materi Praktikum

### 8.4.1 Konsep HMVC pada Laravel

HMVC (*Hierarchical Model View Controller*) adalah pengembangan dari pola MVC, di mana aplikasi dipecah menjadi beberapa **modul** yang berdiri sendiri. Setiap modul memiliki *controller*, model, *view*, *route*, *migration*, dan *request* miliknya sendiri. Agar mudah dibayangkan, aplikasi ibarat sebuah gedung kampus dan modul adalah ruangan-ruangannya: setiap ruangan memiliki perlengkapan sendiri, dapat direnovasi tanpa mengganggu ruangan lain, dan dapat ditambah atau ditutup sesuai kebutuhan.

Pada Laravel, HMVC dapat diterapkan menggunakan *package* **nwidart/laravel-modules** (dokumentasi: https://laravelmodules.com dan https://github.com/nWidart/laravel-modules). Beberapa istilah yang perlu dipahami:

- ***Folder* Modules/** → tempat seluruh modul disimpan. Setiap modul memiliki struktur seperti aplikasi Laravel kecil: app/, config/, database/, resources/, dan routes/.
- **php artisan module:make NamaModul** → membuat modul baru beserta *service provider*, *route*, dan *view* bawaan.
- ***Namespace view*** → *view* di dalam modul dipanggil dengan format namamodul::namaview, misalnya view('pendaftaran::index') untuk *file* Modules/Pendaftaran/resources/views/index.blade.php.
- **modules_statuses.json** → menyimpan status aktif (true) atau nonaktif (false) setiap modul. Modul yang dinonaktifkan tidak memuat *route* dan *view*-nya.
- ***Merge plugin*** → wikimedia/composer-merge-plugin menggabungkan *file* composer.json milik setiap modul ke composer.json utama sehingga class di dalam modul dapat di-*autoload*.

### 8.4.2 Konsep *Authentication* dan *Authorization*

***Authentication*** adalah proses memastikan identitas pengguna (misalnya *login* dengan *email* dan *password*), sedangkan ***authorization*** adalah proses menentukan hak akses pengguna yang sudah *login*. Contohnya, seorang mahasiswa dapat *login* (*authentication* berhasil), tetapi tidak boleh membuka halaman kelola *user* yang hanya untuk *admin* (*authorization* ditolak).

- ***Middleware* auth** → hanya mengizinkan pengguna yang sudah *login*. Tamu otomatis diarahkan ke *route* bernama *login*.

- ***Middleware guest*** → hanya mengizinkan tamu (belum *login*). Pengguna yang sudah *login* diarahkan ke *dashboard* ketika membuka halaman *login* atau *register*.
- ***Middleware role*** → *middleware* buatan sendiri yang memeriksa kolom *role* pada tabel *users*. Parameter ditulis setelah tanda titik dua, misalnya role:admin atau role:admin,*user*. Jika *role* tidak sesuai, aplikasi menampilkan *error* **403 *Forbidden***.
- **Enum UserRole** → daftar nilai *role* yang sah (*admin* dan *user*), sehingga penulisan *role* tidak tersebar sebagai teks bebas di banyak tempat.
  Hak akses yang diterapkan pada praktikum ini adalah sebagai berikut:

| **Halaman** | **Tamu** | ***User*** | ***Admin*** |
|---|---|---|---|
| *Login* dan *Register* | Ya | Tidak (dialihkan ke *dashboard*) | Tidak (dialihkan ke *dashboard*) |
| *Dashboard* | Tidak (ke *login*) | Ya | Ya |
| Modul Pendaftaran (CRUD AJAX) | Tidak (ke *login*) | Ya | Ya |
| Modul UserManagement (kelola *user*) | Tidak (ke *login*) | Tidak (403) | Ya |

### 8.4.3 Konsep AJAX pada Laravel

**AJAX** (*Asynchronous* JavaScript and XML) adalah teknik mengirim dan menerima data dari *server* di latar belakang tanpa memuat ulang seluruh halaman. Pada CRUD biasa (*synchronous*), setiap kali tombol Simpan ditekan *browser* mengirim *form* lalu memuat halaman baru. Pada CRUD AJAX, JavaScript mengirim data, *server* membalas dalam format **JSON**, lalu JavaScript memperbarui tabel secara langsung.

- **$.ajax({...}) dan $.get(...)** → fungsi jQuery untuk mengirim *request* HTTP dari JavaScript.
- **response()->json([...])** → *controller* Laravel mengembalikan data JSON, bukan *view*.
- ***Header* X-CSRF-TOKEN** → *token* CSRF diambil dari *tag* <meta name="csrf-token"> lalu dikirim pada setiap *request* AJAX melalui $.ajaxSetup(). Tanpa *token* ini Laravel menolak *request* dengan *error* **419**.
- ***Header* Accept: application/json** → membuat Laravel mengembalikan *error* validasi sebagai JSON dengan kode **422** sehingga pesan *error* dapat ditampilkan di bawah *input*.
- **_method=PUT** → *form* HTML dan jQuery mengirim POST, lalu *field* _method membuat Laravel memperlakukannya sebagai PUT (*method spoofing*).
- **Escape HTML** → data dari *server* dimasukkan ke tabel menggunakan fungsi escapeHtml() agar teks seperti <script> tidak dijalankan oleh *browser* (mencegah XSS).
- **FormData** → objek JavaScript untuk mengirim isi *form* beserta *file* melalui AJAX. Opsi *processData: false* dan *contentType: false* membuat jQuery mengirim data apa adanya sebagai *multipart/form-data*, sama seperti *form* biasa yang memakai enctype="multipart/form-data".

### 8.4.4 Rancangan Aplikasi

Pada praktikum ini dibuat satu *project* bernama **pertemuan8_hmvc_blade** dengan pembagian sebagai berikut:

| Bagian | Letak | Jenis | Akses |
|---|---|---|---|
| *Register*, *Login*, *Logout*, *Dashboard* | Di luar HMVC (app/, routes/web.php) | *Form* biasa | Tamu / semua yang *login* |
| *Middleware role* | Di luar HMVC (app/Http/Middleware) | *Authorization* | - |
| UserManagement | Modules/UserManagement | CRUD *synchronous* | *Admin* |
| Pendaftaran | Modules/Pendaftaran | CRUD AJAX (5 item + upload berkas) | *Admin* dan *User* |

Modul Pendaftaran menyimpan lima item data, yaitu nama, asal sekolah, prodi tujuan, *email*, dan nomor HP, ditambah satu berkas pendukung (jpg, jpeg, png, atau pdf) yang dapat diunggah, dibuka, diganti, dan dihapus. Modul UserManagement mengelola nama, *email*, *role*, dan *password* pengguna.

### 8.4.5 Membangun Aplikasi HMVC dengan *Authentication*, *Authorization*, dan AJAX

Subbab ini membangun aplikasi dari awal. Tahap A sampai E berisi pembuatan kode, sedangkan seluruh pengujian melalui *browser* dilakukan pada Tahap F. Pengujian digabung di akhir karena *layout* aplikasi memanggil *route* dari kedua modul, sehingga halaman baru dapat dibuka setelah semua *route* tersedia.

**Tahap A: Menyiapkan *Project* dan Memasang nwidart/laravel-modules**

1. Buka terminal di *folder* tempat menyimpan *project* (misalnya C:\xampp\htdocs), pastikan Apache dan MySQL pada XAMPP/Laragon sudah berjalan, lalu buat *project* Laravel baru:

```
composer create-project laravel/laravel pertemuan8_hmvc_blade
cd pertemuan8_hmvc_blade
code .
```

Jika menggunakan Laravel *Installer*, dapat juga menggunakan perintah laravel new pertemuan8_hmvc_blade (pilih **None** ketika ditanya *starter kit* dan **Pest** ketika ditanya *framework testing*).

2. Buka phpMyAdmin (http://localhost/phpmyadmin), buat *database* baru bernama **hmvc_blade**, lalu ubah konfigurasi pada *file* .env:

```
APP_LOCALE=id

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=hmvc_blade
DB_USERNAME=root
DB_PASSWORD=
```

- APP_LOCALE=id → tanggal dan teks seperti "2 menit yang lalu" ditampilkan dalam Bahasa Indonesia.

- DB_CONNECTION=mysql → mengganti *database* bawaan SQLite menjadi MySQL/MariaDB pada XAMPP. Hapus tanda # di depan baris DB_HOST sampai DB_PASSWORD jika masih berupa komentar.

3. Izinkan *merge plugin*, lalu pasang *package* nwidart/laravel-modules:

```
composer config allow-plugins.wikimedia/composer-merge-plugin true
composer require nwidart/laravel-modules
```

- *Package* nwidart/laravel-modules membutuhkan wikimedia/composer-merge-plugin. Perintah composer config allow-plugins... memberi izin agar *plugin* tersebut boleh dijalankan oleh Composer. Jika langkah ini dilewati, Composer akan menanyakan izin secara interaktif (jawab y).

4. Publikasikan *file* konfigurasi dan *stub* milik package:

```
php artisan vendor:publish --
provider="Nwidart\Modules\LaravelModulesServiceProvider"
```

Perintah ini membuat *file* config/modules.php, *folder* stubs/nwidart-stubs (*template file* modul), dan *file* vite-module-loader.js.

5. Buka *file* composer.json, lalu tambahkan bagian **merge-plugin** di dalam extra agar class di dalam setiap modul ter-autoload:

```
    "extra": {
        "laravel": {
            "dont-discover": []
        },
        "merge-plugin": {
            "include": [
                "Modules/*/composer.json"
            ]
        }
    },
```

- Sejak versi 11, *autoload* "Modules\\": "Modules/" pada bagian psr-4 **tidak perlu** ditambahkan lagi. Setiap modul memiliki composer.json sendiri yang digabungkan melalui merge-plugin.
- Jalankan composer dump-autoload setiap kali selesai mengubah composer.json.

composer.json

```
77      "extra": {
78          "laravel": {
79              "dont-discover": []
80          },
81          "merge-plugin": {
82              "include": [
83                  "Modules/*/composer.json"
84              ]
85          }
86      },
87      "config": {
88          "optimize-autoloader": true,
89          "preferred-install": "dist",
90          "sort-packages": true,
91          "allow-plugins": {
92              "pestphp/pest-plugin": true,
93              "php-http/discovery": true,
94              "wikimedia/composer-merge-plugin": true
95          }
96      },
```

*Gambar 8.1 Konfigurasi merge-plugin pada composer.json*

6. Buat dua modul sekaligus, yaitu **UserManagement** dan **Pendaftaran**, lalu muat ulang autoload:

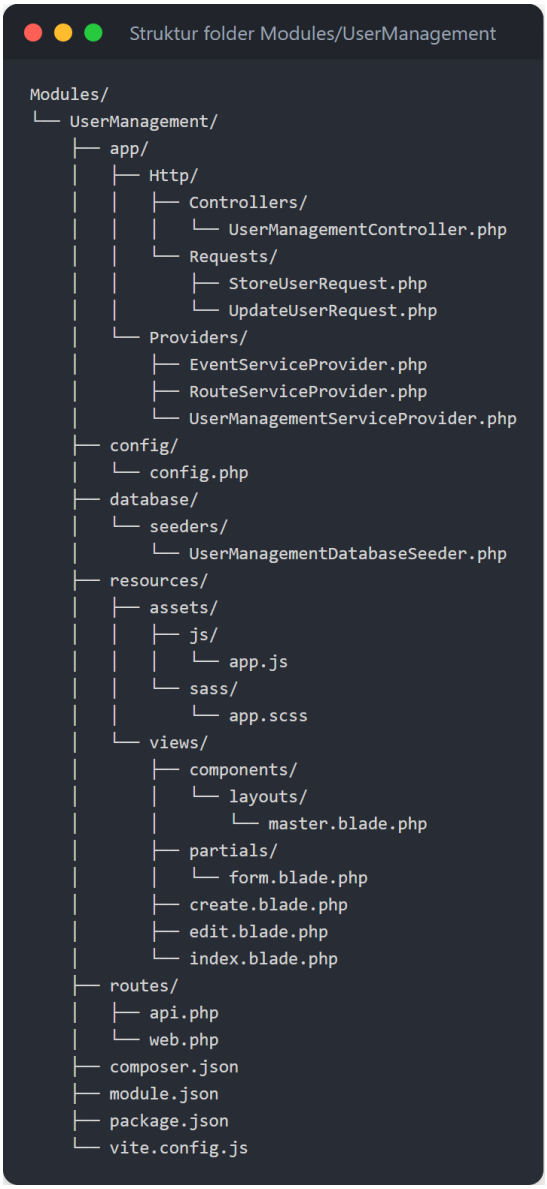
```
php artisan module:make UserManagement Pendaftaran
composer dump-autoload
```

Perintah module:make membuat *folder* Modules/UserManagement dan Modules/Pendaftaran lengkap dengan *service provider*, *controller*, *route*, *view*, serta *file* module.json dan composer.json milik modul. Nama modul juga dicatat pada *file* modules_statuses.json dengan status true (aktif).




*Gambar 8.2 Struktur folder Modules/Pendaftaran*

Struktur akhir modul Pendaftaran setelah seluruh tahap selesai. *File controller*, *request*, model, *factory*, *migration*, *route*, dan *view* berada di dalam *folder* modul.

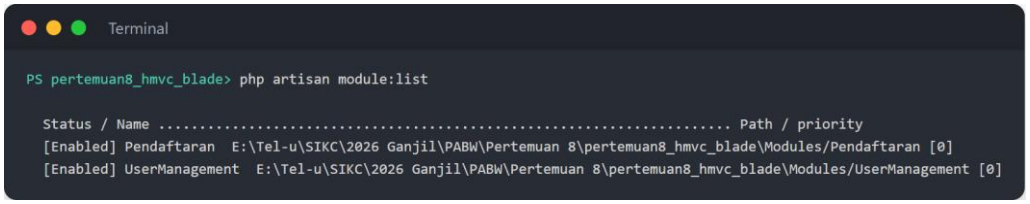



*Gambar 8.3 Struktur folder Modules/UserManagement*

Modul UserManagement tidak memiliki model sendiri karena menggunakan model App\Models\User milik aplikasi utama.

7. Periksa modul yang sudah terdaftar dengan perintah berikut:

```
php artisan module:list
```




*Gambar 8.4 Hasil php artisan module:list*

Kedua modul berstatus Enabled sehingga *route* dan *view*-nya dimuat oleh Laravel.

8. Kosongkan *file* routes/api.php pada **kedua** modul, yaitu Modules/UserManagement/routes/api.php dan Modules/Pendaftaran/routes/api.php:

```
<?php

// Modul ini tidak menyediakan endpoint API.
```

- *Route* API bawaan modul menggunakan *middleware* auth:sanctum, padahal Laravel Sanctum tidak dipasang pada *project* ini. Selain itu *route* tersebut menunjuk *method controller* yang tidak digunakan. Karena modul ini tidak menyediakan API, *file* cukup dikosongkan.

**Tahap B: *Role* Pengguna dan *Authentication* di Luar HMVC**

Fitur *register*, *login*, dan *logout* ditulis di *folder* app/ (di luar modul) karena digunakan oleh seluruh modul. Tabel *users* dan model *User* sudah tersedia dari *project* Laravel bawaan, sehingga kita hanya menambahkan kolom ***role***.

1. Buat *migration* untuk menambahkan kolom *role* pada tabel users:

```
php artisan make:migration add_role_to_users_table --table=users
```

Buka *file* database/migrations/xxxx_xx_xx_xxxxxx_add_role_to_users_table.php, lalu ganti isinya dengan kode berikut:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('role')->default('user')->after('email');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('role');
        });
    }
};
```

- string('role')->default('user') → setiap akun baru otomatis berperan sebagai *user* jika *role* tidak diisi.
- after('email') → kolom *role* diletakkan setelah kolom *email* (berlaku pada MySQL).

- dropColumn('role') pada *method* down() → menghapus kolom ketika *migration* di-rollback.

database/migrations/2026_11_01_071712_add_role_to_users_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('role')->default('user')->after('email');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('role');
        });
    }
};
```

*Gambar 8.5 Migration penambahan kolom role*

2. Buat enum untuk daftar role:

```
php artisan make:enum Enums/UserRole --string
```

Buka *file* app/Enums/UserRole.php, lalu ganti isinya dengan kode berikut:

```
<?php

namespace App\Enums;

enum UserRole: string
{
    case Admin = 'admin';
    case User = 'user';

    /**
     * Get the human readable label of the role.
     */
    public function label(): string
    {
        return match ($this) {
            self::Admin => 'Admin',
            self::User => 'User',
```

```
        };
    }
}
```

- enum UserRole: string → enum dengan nilai teks (*admin* dan *user*) yang tersimpan di kolom *role*.
- label() → mengembalikan nama *role* yang ramah dibaca untuk ditampilkan di halaman, misalnya pada *badge role*.

app/Enums/UserRole.php

```php
<?php

namespace App\Enums;

enum UserRole: string
{
    case Admin = 'admin';
    case User = 'user';

    /**
     * Get the human readable label of the role.
     */
    public function label(): string
    {
        return match ($this) {
            self::Admin => 'Admin',
            self::User => 'User',
        };
    }
}
```

*Gambar 8.6 Enum UserRole*

3. Buka *file* app/Models/User.php, lalu ganti isinya dengan kode berikut:

```php
<?php

namespace App\Models;

// use Illuminate\Contracts\Auth\MustVerifyEmail;
use App\Enums\UserRole;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasFactory, Notifiable;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
```

```
    protected $fillable = [
        'name',
        'email',
        'role',
        'password',
    ];

    /**
     * The attributes that should be hidden for serialization.
     *
     * @var list<string>
     */
    protected $hidden = [
        'password',
        'remember_token',
    ];

    /**
     * The model's default attribute values.
     *
     * @var array<string, mixed>
     */
    protected $attributes = [
        'role' => 'user',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
            'role' => UserRole::class,
        ];
    }

    /**
     * Determine whether the user has the admin role.
     */
    public function isAdmin(): bool
    {
        return $this->role === UserRole::Admin;
    }

    /**
     * Determine whether the user has one of the given roles.
     */
    public function hasRole(UserRole|string ...$roles): bool
    {
```

```
        foreach ($roles as $role) {
            if ($this->role === ($role instanceof UserRole ? $role :
UserRole::tryFrom($role))) {
                return true;
            }
        }

        return false;
    }
}
```

- 'role' pada $fillable → kolom *role* boleh diisi melalui User::create() dan update().
- $attributes = ['role' => 'user'] → nilai bawaan *role* pada objek *User* sebelum disimpan.
- 'role' => UserRole::class pada casts() → nilai *role* otomatis diubah menjadi enum UserRole, sehingga dapat dibandingkan dengan UserRole::Admin.
- isAdmin() → memeriksa apakah pengguna adalah *admin*. *Method* ini dipakai di *view* untuk menampilkan menu Kelola *User*.
- hasRole(...$roles) → memeriksa apakah *role* pengguna termasuk salah satu *role* yang diberikan. *Method* ini dipakai oleh *middleware role*.

app/Models/User.php

```php
<?php

namespace App\Models;

// use Illuminate\Contracts\Auth\MustVerifyEmail;
use App\Enums\UserRole;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasFactory, Notifiable;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'name',
        'email',
        'role',
        'password',
    ];

    /**
     * The attributes that should be hidden for serialization.
     *
     * @var list<string>
     */
    protected $hidden = [
        'password',
        'remember_token',
    ];

    /**
     * The model's default attribute values.
     *
     * @var array<string, mixed>
     */
    protected $attributes = [
        'role' => 'user',
    ];

    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
            'role' => UserRole::class,
        ];
    }

    /**
     * Determine whether the user has the admin role.
     */
    public function isAdmin(): bool
    {
        return $this->role === UserRole::Admin;
    }

    /**
     * Determine whether the user has one of the given roles.
     */
    public function hasRole(UserRole|string ...$roles): bool
    {
        foreach ($roles as $role) {
            if ($this->role === ($role instanceof UserRole ? $role : UserRole::tryFrom($role))) {
                return true;
            }
        }

        return false;
    }
}
```

*Gambar 8.7 Model User dengan role*

4. Buka *file* database/factories/UserFactory.php, lalu ganti isinya dengan kode berikut agar *factory* dapat membuat akun admin:

```
<?php
```

```
namespace Database\Factories;

use App\Enums\UserRole;
use App\Models\User;
use Illuminate\Database\Eloquent\Factories\Factory;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Str;

/**
 * @extends Factory<User>
 */
class UserFactory extends Factory
{
    /**
     * The current password being used by the factory.
     */
    protected static ?string $password;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'name' => fake()->name(),
            'email' => fake()->unique()->safeEmail(),
            'role' => UserRole::User,
            'email_verified_at' => now(),
            'password' => static::$password ??=
Hash::make('password'),
            'remember_token' => Str::random(10),
        ];
    }

    /**
     * Indicate that the user is an admin.
     */
    public function admin(): static
    {
        return $this->state(fn (array $attributes) => [
            'role' => UserRole::Admin,
        ]);
    }

    /**
     * Indicate that the model's email address should be unverified.
     */
    public function unverified(): static
    {
        return $this->state(fn (array $attributes) => [
            'email_verified_at' => null,
```

```
        ]);
    }
}
```

- 'role' => UserRole::User → akun buatan *factory* secara bawaan berperan sebagai *user*.
- admin() → *state factory* untuk membuat akun *admin*, misalnya User::factory()->admin()->create().

database/factories/UserFactory.php

```php
    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'name' => fake()->name(),
            'email' => fake()->unique()->safeEmail(),
            'role' => UserRole::User,
            'email_verified_at' => now(),
            'password' => static::$password ??= Hash::make('password'),
            'remember_token' => Str::random(10),
        ];
    }

    /**
     * Indicate that the user is an admin.
     */
    public function admin(): static
    {
        return $this->state(fn (array $attributes) => [
            'role' => UserRole::Admin,
        ]);
    }

    /**
     * Indicate that the model's email address should be unverified.
     */
    public function unverified(): static
    {
        return $this->state(fn (array $attributes) => [
            'email_verified_at' => null,
        ]);

```

*Gambar 8.8 State admin pada UserFactory*

5. Buka *file* database/seeders/DatabaseSeeder.php, lalu ganti isinya dengan kode berikut untuk membuat akun awal:

```
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
```

```
class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        User::factory()->admin()->create([
            'name' => 'Administrator',
            'email' => 'admin@example.com',
            'password' => 'password',
        ]);

        User::factory()->create([
            'name' => 'User Biasa',
            'email' => 'user@example.com',
            'password' => 'password',
        ]);
    }
}
```

- *Seeder* membuat satu akun ***admin*** (admin@example.com) dan satu akun ***user*** (user@example.com), keduanya dengan *password password*.
- *Password* ditulis biasa, lalu otomatis di-hash oleh *cast* 'password' => 'hashed' pada model *User* sebelum disimpan.

database/seeders/DatabaseSeeder.php

```php
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    /**
     * Seed the application's database.
     */
    public function run(): void
    {
        User::factory()->admin()->create([
            'name' => 'Administrator',
            'email' => 'admin@example.com',
            'password' => 'password',
        ]);

        User::factory()->create([
            'name' => 'User Biasa',
            'email' => 'user@example.com',
            'password' => 'password',
        ]);
    }
}
```

*Gambar 8.9 DatabaseSeeder dengan akun admin dan user*

6. Buat *Form Request* untuk validasi *login* dan register:

```
php artisan make:request Auth/LoginRequest
php artisan make:request Auth/RegisterRequest
```

Buka *file* app/Http/Requests/Auth/LoginRequest.php, lalu ganti isinya dengan kode berikut:

```php
<?php

namespace App\Http\Requests\Auth;

use Illuminate\Foundation\Http\FormRequest;

class LoginRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }
```

```
    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, array<int, string>>
     */
    public function rules(): array
    {
        return [
            'email' => ['required', 'string', 'email'],
            'password' => ['required', 'string'],
        ];
    }
}
```

Buka *file* app/Http/Requests/Auth/RegisterRequest.php, lalu ganti isinya dengan kode berikut:

```
<?php

namespace App\Http\Requests\Auth;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rules\Password;

class RegisterRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255',
'unique:users,email'],
            'password' => ['required', 'confirmed',
Password::defaults()],
        ];
    }
}
```

- *Form Request* memisahkan aturan validasi dari *controller*. Jika validasi gagal, Laravel otomatis kembali ke *form* beserta pesan *error* dan *input* lama.

- unique:users,*email* → *email* yang sama tidak dapat didaftarkan dua kali.
- confirmed → *input password* harus sama dengan *input* password_confirmation.
- Password::defaults() → aturan *password* bawaan Laravel (minimal 8 karakter).
- *Field* ***role*** **tidak ada** pada RegisterRequest, sehingga pengguna tidak dapat mendaftarkan dirinya sebagai *admin*.

app/Http/Requests/Auth/RegisterRequest.php

```php
<?php

namespace App\Http\Requests\Auth;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rules\Password;

class RegisterRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', 'unique:users,email'],
            'password' => ['required', 'confirmed', Password::defaults()],
        ];
    }
}
```

*Gambar 8.10 RegisterRequest*

7. Buat *controller authentication* dan dashboard:

```
php artisan make:controller Auth/LoginController
php artisan make:controller Auth/RegisterController
php artisan make:controller DashboardController --invokable
```

Buka *file* app/Http/Controllers/Auth/LoginController.php, lalu ganti isinya dengan kode berikut:

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
```

```
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\Str;
use Illuminate\Validation\ValidationException;
use Illuminate\View\View;

class LoginController extends Controller
{
    /**
     * Show the login form.
     */
    public function create(): View
    {
        return view('auth.login');
    }

    /**
     * Authenticate the user.
     */
    public function store(LoginRequest $request): RedirectResponse
    {
        $throttleKey = Str::transliterate(Str::lower($request-
>string('email')).'|'.$request->ip());

        if (RateLimiter::tooManyAttempts($throttleKey, 5)) {
            throw ValidationException::withMessages([
                'email' => 'Terlalu banyak percobaan login. Coba lagi
dalam '.RateLimiter::availableIn($throttleKey).' detik.',
            ]);
        }

        if (! Auth::attempt($request->validated(), $request-
>boolean('remember'))) {
            RateLimiter::hit($throttleKey);

            throw ValidationException::withMessages([
                'email' => 'Email atau password salah.',
            ]);
        }

        RateLimiter::clear($throttleKey);

        $request->session()->regenerate();

        return redirect()->intended(route('dashboard'));
    }

    /**
     * Log the user out.
     */
    public function destroy(Request $request): RedirectResponse
    {
        Auth::logout();
```

```
        $request->session()->invalidate();
        $request->session()->regenerateToken();

        return redirect()->route('login');
    }
}
```

- create() → menampilkan *form login*, store() → memproses *login*, dan destroy() → *logout*.
- RateLimiter::tooManyAttempts($throttleKey, 5) → membatasi percobaan *login* yang gagal maksimal 5 kali per kombinasi *email* dan alamat IP untuk mencegah tebak-tebakan *password* (*brute force*).
- Auth::attempt($request->validated(), $request->boolean('remember')) → mencocokkan *email* dan *password* dengan tabel *users*. Opsi remember (Ingat saya) membuat *login* bertahan lebih lama.
- $request->session()->regenerate() → membuat ID *session* baru setelah *login* untuk mencegah *session fixation*.
- redirect()->intended(route('dashboard')) → mengembalikan pengguna ke halaman yang tadi ingin dibuka sebelum diarahkan ke *login*.
- session()->invalidate() dan regenerateToken() pada *logout* → menghapus seluruh data *session* dan membuat *token* CSRF baru.

app/Http/Controllers/Auth/LoginController.php

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Support\Str;
use Illuminate\Validation\ValidationException;
use Illuminate\View\View;

class LoginController extends Controller
{
    /**
     * Show the login form.
     */
    public function create(): View
    {
        return view('auth.login');
    }

    /**
     * Authenticate the user.
     */
    public function store(LoginRequest $request): RedirectResponse
    {
        $throttleKey = Str::transliterate(Str::lower($request->string('email')).'|'.$request->ip());

        if (RateLimiter::tooManyAttempts($throttleKey, 5)) {
            throw ValidationException::withMessages([
                'email' => 'Terlalu banyak percobaan login. Coba lagi dalam '.RateLimiter::availableIn($throttleKey).' detik.',
            ]);
        }

        if (! Auth::attempt($request->validated(), $request->boolean('remember'))) {
            RateLimiter::hit($throttleKey);

            throw ValidationException::withMessages([
                'email' => 'Email atau password salah.',
            ]);
        }

        RateLimiter::clear($throttleKey);

        $request->session()->regenerate();

        return redirect()->intended(route('dashboard'));
    }

    /**
     * Log the user out.
     */
    public function destroy(Request $request): RedirectResponse
    {
        Auth::logout();

        $request->session()->invalidate();
        $request->session()->regenerateToken();

        return redirect()->route('login');
    }
}
```

*Gambar 8.11 LoginController*

Buka *file* app/Http/Controllers/Auth/RegisterController.php, lalu ganti isinya dengan kode berikut:

```
<?php

namespace App\Http\Controllers\Auth;
```

```
use App\Enums\UserRole;
use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\RegisterRequest;
use App\Models\User;
use Illuminate\Http\RedirectResponse;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

class RegisterController extends Controller
{
    /**
     * Show the registration form.
     */
    public function create(): View
    {
        return view('auth.register');
    }

    /**
     * Register a new user. Self-registered accounts always get the
"user" role.
     */
    public function store(RegisterRequest $request): RedirectResponse
    {
        $user = User::create([
            ...$request->safe()->only(['name', 'email', 'password']),
            'role' => UserRole::User,
        ]);

        Auth::login($user);

        $request->session()->regenerate();

        return redirect()->route('dashboard')->with('success',
'Registrasi berhasil. Selamat datang!');
    }
}
```

- $request->safe()->only([...]) → hanya mengambil nama, *email*, dan *password* yang sudah lolos validasi.
- 'role' => UserRole::User → akun hasil *register* **selalu** berperan sebagai *user*. Akun *admin* hanya dapat dibuat melalui *seeder* atau oleh *admin* di modul UserManagement.
- Auth::login($user) → pengguna yang baru mendaftar langsung masuk ke aplikasi.

app/Http/Controllers/Auth/RegisterController.php

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Enums\UserRole;
use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\RegisterRequest;
use App\Models\User;
use Illuminate\Http\RedirectResponse;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

class RegisterController extends Controller
{
    /**
     * Show the registration form.
     */
    public function create(): View
    {
        return view('auth.register');
    }

    /**
     * Register a new user. Self-registered accounts always get the "user" role.
     */
    public function store(RegisterRequest $request): RedirectResponse
    {
        $user = User::create([
            ...$request->safe()->only(['name', 'email', 'password']),
            'role' => UserRole::User,
        ]);

        Auth::login($user);

        $request->session()->regenerate();

        return redirect()->route('dashboard')->with('success', 'Registrasi berhasil. Selamat datang!');
    }
}
```

*Gambar 8.12 RegisterController*

Buka *file* app/Http/Controllers/DashboardController.php, lalu ganti isinya dengan kode berikut:

```php
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\View\View;
use Modules\Pendaftaran\Models\Daftar;

class DashboardController extends Controller
{
    /**
     * Show the dashboard with record counts per module.
     */
    public function __invoke(Request $request): View
```

```
    {
        return view('dashboard', [
            'totalPendaftar' => Daftar::count(),
            'totalUsers' => $request->user()->isAdmin() ?
User::count() : null,
        ]);
    }
}
```

- *Controller invokable* hanya memiliki satu *method* __invoke(), cocok untuk halaman yang hanya menampilkan satu tampilan.
- *Dashboard* menampilkan jumlah pendaftar dari model Daftar milik modul Pendaftaran (dibuat pada Tahap E). Jumlah *user* hanya dihitung untuk *admin*, sedangkan untuk *user* bernilai null.

8. Buka *file* routes/web.php, lalu ganti seluruh isinya dengan kode berikut:

```
<?php

use App\Http\Controllers\Auth\LoginController;
use App\Http\Controllers\Auth\RegisterController;
use App\Http\Controllers\DashboardController;
use Illuminate\Support\Facades\Route;

Route::redirect('/', '/dashboard');

Route::middleware('guest')->group(function () {
    Route::get('login', [LoginController::class, 'create'])-
>name('login');
    Route::post('login', [LoginController::class, 'store'])-
>name('login.store');
    Route::get('register', [RegisterController::class, 'create'])-
>name('register');
    Route::post('register', [RegisterController::class, 'store'])-
>name('register.store');
});

Route::middleware('auth')->group(function () {
    Route::get('dashboard', DashboardController::class)-
>name('dashboard');
    Route::post('logout', [LoginController::class, 'destroy'])-
>name('logout');
});
```

- Route::middleware('guest') → halaman *login* dan *register* hanya dapat dibuka oleh tamu.
- ->name('login') → nama *route login* wajib ada, karena *middleware* auth mengarahkan tamu ke *route* bernama *login*.
- Route::middleware('auth') → *dashboard* dan *logout* hanya dapat diakses setelah *login*. *Logout* menggunakan *method* POST agar terlindungi *token* CSRF.
- *Route* milik modul **tidak** ditulis di sini, karena setiap modul mendaftarkan *route*-nya sendiri pada *file* Modules/NamaModul/routes/web.php.

routes/web.php

```php
<?php

use App\Http\Controllers\Auth\LoginController;
use App\Http\Controllers\Auth\RegisterController;
use App\Http\Controllers\DashboardController;
use Illuminate\Support\Facades\Route;

Route::redirect('/', '/dashboard');

Route::middleware('guest')->group(function () {
    Route::get('login', [LoginController::class, 'create'])->name('login');
    Route::post('login', [LoginController::class, 'store'])->name('login.store');
    Route::get('register', [RegisterController::class, 'create'])->name('register');
    Route::post('register', [RegisterController::class, 'store'])->name('register.store');
});

Route::middleware('auth')->group(function () {
    Route::get('dashboard', DashboardController::class)->name('dashboard');
    Route::post('logout', [LoginController::class, 'destroy'])->name('logout');
});
```

*Gambar 8.13 Route authentication pada routes/web.php*

9. Buat *file* CSS tema di public/css/theme.css. *File* ini hanya berisi penyesuaian kecil warna dan tampilan di atas Bootstrap:

```css
:root {
    --bs-primary: #2563eb;
    --bs-primary-rgb: 37, 99, 235;
    --bs-link-color: #2563eb;
    --bs-link-color-rgb: 37, 99, 235;
    --bs-link-hover-color: #1d4ed8;
    --bs-border-radius: .5rem;
}

body {
    background: #f5f6f8;
    color: #212529;
}

.btn-primary {
    --bs-btn-bg: #2563eb;
    --bs-btn-border-color: #2563eb;
    --bs-btn-hover-bg: #1d4ed8;
    --bs-btn-hover-border-color: #1d4ed8;
    --bs-btn-active-bg: #1e40af;
    --bs-btn-active-border-color: #1e40af;
    --bs-btn-disabled-bg: #2563eb;
    --bs-btn-disabled-border-color: #2563eb;
}

.btn-outline-primary {
    --bs-btn-color: #2563eb;
    --bs-btn-border-color: #2563eb;
    --bs-btn-hover-bg: #2563eb;
```

```
    --bs-btn-hover-border-color: #2563eb;
    --bs-btn-active-bg: #1d4ed8;
}

.form-control:focus,
.form-select:focus {
    border-color: #93b4f5;
    box-shadow: 0 0 0 .2rem rgba(37, 99, 235, .15);
}

/* Navbar */
.navbar-brand .logo {
    width: 32px;
    height: 32px;
    border-radius: 8px;
    background: #2563eb;
    color: #fff;
    display: inline-flex;
    align-items: center;
    justify-content: center;
    font-size: 1rem;
}

.navbar .nav-link {
    color: #6c757d;
    border-radius: .5rem;
    padding: .45rem .8rem !important;
}

.navbar .nav-link:hover {
    color: #212529;
    background: #f1f3f5;
}

.navbar .nav-link.active {
    color: #2563eb;
    background: #eef3fe;
    font-weight: 600;
}

/* Kartu */
.card {
    border: 1px solid #e9ecef;
    border-radius: .75rem;
}

.card-header {
    background: #fff;
    border-bottom: 1px solid #e9ecef;
}

.icon-box {
    width: 44px;
```

```
    height: 44px;
    border-radius: 10px;
    display: inline-flex;
    align-items: center;
    justify-content: center;
    font-size: 1.25rem;
    flex-shrink: 0;
}

.icon-box.blue { background: #eef3fe; color: #2563eb; }
.icon-box.green { background: #e8f6ee; color: #198754; }
.icon-box.gray { background: #f1f3f5; color: #495057; }

/* Tabel */
.table {
    margin-bottom: 0;
}

.table thead th {
    font-size: .75rem;
    font-weight: 600;
    color: #6c757d;
    text-transform: uppercase;
    letter-spacing: .03em;
    background: #f8f9fa;
    white-space: nowrap;
}

.table td,
.table th {
    vertical-align: middle;
    padding: .75rem 1rem;
}

/* Data kosong */
.empty-state {
    text-align: center;
    padding: 3rem 1rem;
    color: #6c757d;
}

.empty-state i {
    font-size: 2.5rem;
    color: #adb5bd;
}

/* Halaman login / register */
.auth-logo {
    width: 48px;
    height: 48px;
    border-radius: 12px;
    background: #2563eb;
    color: #fff;
```

```
    display: inline-flex;
    align-items: center;
    justify-content: center;
    font-size: 1.4rem;
}
```

10. Buat *folder* resources/views/layouts/partials, lalu buat *file* resources/views/layouts/partials/head.blade.php yang berisi bagian <head> bersama:

```
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta name="csrf-token" content="{{ csrf_token() }}">

<title>@yield('title', 'Dashboard') - {{ config('app.name',
'Laravel') }}</title>

<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap
.min.css" rel="stylesheet">
<link href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.11.3/font/bootstrap-icons.min.css" rel="stylesheet">
<link href="{{ asset('css/theme.css') }}" rel="stylesheet">
@stack('styles')
```

- <meta name="csrf-token"> → menyimpan *token* CSRF yang nanti dibaca oleh jQuery untuk setiap *request* AJAX.
- @stack('styles') → tempat *view* lain menambahkan CSS khusus melalui @push('styles').

11. Buat *file* resources/views/layouts/partials/scripts.blade.php yang berisi JavaScript bersama dan *modal* konfirmasi hapus:

```
{{-- Modal konfirmasi yang dipakai bersama oleh seluruh halaman --}}
<div class="modal fade" id="confirmModal" tabindex="-1" aria-
labelledby="confirmModalLabel" aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered modal-sm">
        <div class="modal-content">
            <div class="modal-body text-center p-4">
                <i class="bi bi-exclamation-circle text-danger fs-
1"></i>
                <h6 class="mt-2 mb-1" id="confirmModalLabel">Hapus
data?</h6>
                <p class="text-muted small mb-0"
id="confirmModalMessage"></p>
            </div>
            <div class="modal-footer justify-content-center border-0
pt-0 pb-4">
                <button type="button" class="btn btn-light" data-bs-
dismiss="modal">Batal</button>
                <button type="button" class="btn btn-danger"
id="confirmModalYes">Ya, hapus</button>
            </div>
        </div>
    </div>
```

```
</div>

<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.b
undle.min.js"></script>
<script>
    /**
     * Tampilkan modal konfirmasi. Mengembalikan Promise yang
bernilai true jika user menekan "Ya, hapus".
     */
    window.confirmDelete = function (message) {
        const modalElement = document.getElementById('confirmModal');
        const modal =
bootstrap.Modal.getOrCreateInstance(modalElement);
        document.getElementById('confirmModalMessage').textContent =
message;

        return new Promise(function (resolve) {
            let confirmed = false;
            const yesButton =
document.getElementById('confirmModalYes');

            yesButton.onclick = function () {
                confirmed = true;
                modal.hide();
            };

            modalElement.addEventListener('hidden.bs.modal', function
() {
                resolve(confirmed);
            }, { once: true });

            modal.show();
        });
    };

    document.addEventListener('DOMContentLoaded', function () {
        // Form dengan atribut data-confirm meminta konfirmasi
sebelum dikirim
        document.querySelectorAll('form[data-
confirm]').forEach(function (form) {
            form.addEventListener('submit', function (event) {
                event.preventDefault();
                confirmDelete(form.dataset.confirm).then(function
(confirmed) {
                    if (confirmed) {
                        form.submit();
                    }
                });
            });
        });
```

```
        // Tombol tampilkan / sembunyikan password
        document.querySelectorAll('[data-toggle-
password]').forEach(function (button) {
            button.addEventListener('click', function () {
                const input =
document.getElementById(button.dataset.togglePassword);
                const isHidden = input.type === 'password';
                input.type = isHidden ? 'text' : 'password';
                button.querySelector('i').className = isHidden ? 'bi
bi-eye-slash' : 'bi bi-eye';
            });
        });

        // Pesan sukses hilang otomatis setelah 4 detik
        document.querySelectorAll('.alert-autohide').forEach(function
(alert) {
            setTimeout(function () {
                bootstrap.Alert.getOrCreateInstance(alert).close();
            }, 4000);
        });
    });
</script>
@stack('scripts')
```

- window.confirmDelete(*message*) → menampilkan *modal* konfirmasi Bootstrap dan mengembalikan *Promise* bernilai true jika pengguna menekan **Ya, hapus**. Fungsi ini dipakai oleh *form* biasa maupun oleh tombol hapus AJAX.
- *form*[data-confirm] → setiap *form* yang memiliki atribut data-confirm otomatis meminta konfirmasi sebelum dikirim.
- [data-toggle-password] → tombol mata untuk menampilkan atau menyembunyikan *password*.
- .alert-autohide → pesan berhasil otomatis tertutup setelah 4 detik.

resources/views/layouts/partials/scripts.blade.php

```
<script>
    /**
     * Tampilkan modal konfirmasi. Mengembalikan Promise yang bernilai true jika user menekan "Ya, hapus".
     */
    window.confirmDelete = function (message) {
        const modalElement = document.getElementById('confirmModal');
        const modal = bootstrap.Modal.getOrCreateInstance(modalElement);
        document.getElementById('confirmModalMessage').textContent = message;

        return new Promise(function (resolve) {
            let confirmed = false;
            const yesButton = document.getElementById('confirmModalYes');

            yesButton.onclick = function () {
                confirmed = true;
                modal.hide();
            };

            modalElement.addEventListener('hidden.bs.modal', function () {
                resolve(confirmed);
            }, { once: true });

            modal.show();
        });
    };

    document.addEventListener('DOMContentLoaded', function () {
        // Form dengan atribut data-confirm meminta konfirmasi sebelum dikirim
        document.querySelectorAll('form[data-confirm]').forEach(function (form) {
            form.addEventListener('submit', function (event) {
                event.preventDefault();
                confirmDelete(form.dataset.confirm).then(function (confirmed) {
                    if (confirmed) {
                        form.submit();
                    }
                });
            });
        });

        // Tombol tampilkan / sembunyikan password
        document.querySelectorAll('[data-toggle-password]').forEach(function (button) {
            button.addEventListener('click', function () {
                const input = document.getElementById(button.dataset.togglePassword);
                const isHidden = input.type === 'password';
                input.type = isHidden ? 'text' : 'password';
                button.querySelector('i').className = isHidden ? 'bi bi-eye-slash' : 'bi bi-eye';
            });
        });

        // Pesan sukses hilang otomatis setelah 4 detik
        document.querySelectorAll('.alert-autohide').forEach(function (alert) {
            setTimeout(function () {
                bootstrap.Alert.getOrCreateInstance(alert).close();
            }, 4000);
        });
    });
</script>
```

*Gambar 8.14 Fungsi confirmDelete dan script bersama*

12. Buat *file* resources/views/layouts/app.blade.php sebagai *layout* untuk halaman yang memerlukan login:

```
<!DOCTYPE html>
<html lang="{{ str_replace('_', '-', app()->getLocale()) }}">
<head>
    @include('layouts.partials.head')
```

```
</head>
<body>
    <nav class="navbar navbar-expand-lg bg-white border-bottom py-2">
        <div class="container">
            <a class="navbar-brand d-flex align-items-center gap-2
fw-semibold" href="{{ route('dashboard') }}">
                <span class="logo"><i class="bi bi-
mortarboard"></i></span>
                {{ config('app.name', 'Laravel') }}
            </a>
            <button class="navbar-toggler border-0" type="button"
data-bs-toggle="collapse" data-bs-target="#mainNavbar" aria-
controls="mainNavbar" aria-expanded="false" aria-label="Buka menu">
                <span class="navbar-toggler-icon"></span>
            </button>

            <div class="collapse navbar-collapse" id="mainNavbar">
                <ul class="navbar-nav me-auto ms-lg-4 gap-lg-1">
                    <li class="nav-item">
                        <a class="nav-link {{ request()-
>routeIs('dashboard') ? 'active' : '' }}" href="{{ route('dashboard')
}}">
                            <i class="bi bi-house me-1"></i>
Dashboard
                        </a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link {{ request()-
>routeIs('pendaftaran.*') ? 'active' : '' }}" href="{{
route('pendaftaran.index') }}">
                            <i class="bi bi-clipboard me-1"></i>
Pendaftaran
                        </a>
                    </li>
                    @if (auth()->user()->isAdmin())
                        <li class="nav-item">
                            <a class="nav-link {{ request()-
>routeIs('admin.users.*') ? 'active' : '' }}" href="{{
route('admin.users.index') }}">
                                <i class="bi bi-people me-1"></i>
Kelola User
                            </a>
                        </li>
                    @endif
                </ul>

                <div class="dropdown">
                    <button class="btn btn-light dropdown-toggle d-
flex align-items-center gap-2" type="button" data-bs-
toggle="dropdown" aria-expanded="false">
                        <i class="bi bi-person-circle"></i>
                        <span>{{ auth()->user()->name }}</span>
                    </button>
```

```
                    <ul class="dropdown-menu dropdown-menu-end">
                        <li class="px-3 py-2 small">
                            <div class="text-muted">{{ auth()-
>user()->email }}</div>
                            <span class="badge {{ auth()->user()-
>isAdmin() ? 'text-bg-primary' : 'text-bg-secondary' }} mt-1">{{
auth()->user()->role->label() }}</span>
                        </li>
                        <li><hr class="dropdown-divider"></li>
                        <li>
                            <form method="POST" action="{{
route('logout') }}">
                                @csrf
                                <button type="submit"
class="dropdown-item text-danger">
                                    <i class="bi bi-box-arrow-right
me-2"></i>Logout
                                </button>
                            </form>
                        </li>
                    </ul>
                </div>
            </div>
        </div>
    </nav>

    <main class="container py-4">
        @if (session('success'))
            <div class="alert alert-success alert-dismissible alert-
autohide fade show d-flex align-items-center gap-2" role="alert">
                <i class="bi bi-check-circle"></i>
                <span>{{ session('success') }}</span>
                <button type="button" class="btn-close" data-bs-
dismiss="alert" aria-label="Tutup"></button>
            </div>
        @endif

        @if (session('error'))
            <div class="alert alert-danger alert-dismissible fade
show d-flex align-items-center gap-2" role="alert">
                <i class="bi bi-exclamation-triangle"></i>
                <span>{{ session('error') }}</span>
                <button type="button" class="btn-close" data-bs-
dismiss="alert" aria-label="Tutup"></button>
            </div>
        @endif

        @yield('content')
    </main>

    @include('layouts.partials.scripts')
</body>
</html>
```

- @if (auth()->user()->isAdmin()) → menu **Kelola *User*** hanya ditampilkan untuk *admin*. Ini hanya menyembunyikan menu; perlindungan sebenarnya tetap dilakukan oleh *middleware role* pada *route*.
- request()->routeIs('pendaftaran.*') → memberi kelas active pada menu yang sedang dibuka.
- session('success') dan session('error') → menampilkan pesan dari *controller* sehingga tidak perlu ditulis ulang pada setiap *view*.

resources/views/layouts/app.blade.php

```blade
<!DOCTYPE html>
<html lang="{{ str_replace('_', '-', app()->getLocale()) }}">
<head>
    @include('layouts.partials.head')
</head>
<body>
    <nav class="navbar navbar-expand-lg bg-white border-bottom py-2">
        <div class="container">
            <a class="navbar-brand d-flex align-items-center gap-2 fw-semibold" href="{{ route('dashboard') }}">
                <span class="logo"><i class="bi bi-mortarboard"></i></span>
                {{ config('app.name', 'Laravel') }}
            </a>
            <button class="navbar-toggler border-0" type="button" data-bs-toggle="collapse" data-bs-target="#mainNavbar" aria-controls="mainNavbar" aria-expanded="false" aria-label="Buka menu">
                <span class="navbar-toggler-icon"></span>
            </button>

            <div class="collapse navbar-collapse" id="mainNavbar">
                <ul class="navbar-nav me-auto ms-lg-4 gap-lg-1">
                    <li class="nav-item">
                        <a class="nav-link {{ request()->routeIs('dashboard') ? 'active' : '' }}" href="{{ route('dashboard') }}">
                            <i class="bi bi-house me-1"></i> Dashboard
                        </a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link {{ request()->routeIs('pendaftaran.*') ? 'active' : '' }}" href="{{ route('pendaftaran.index') }}">
                            <i class="bi bi-clipboard me-1"></i> Pendaftaran
                        </a>
                    </li>
                    @if (auth()->user()->isAdmin())
                        <li class="nav-item">
                            <a class="nav-link {{ request()->routeIs('admin.users.*') ? 'active' : '' }}" href="{{ route('admin.users.index') }}">
                                <i class="bi bi-people me-1"></i> Kelola User
                            </a>
                        </li>
                    @endif
                </ul>

                <div class="dropdown">
                    <button class="btn btn-light dropdown-toggle d-flex align-items-center gap-2" type="button" data-bs-toggle="dropdown" aria-expanded="false">
                        <i class="bi bi-person-circle"></i>
                        <span>{{ auth()->user()->name }}</span>
                    </button>
                    <ul class="dropdown-menu dropdown-menu-end">
                        <li class="px-3 py-2 small">
                            <div class="text-muted">{{ auth()->user()->email }}</div>
                            <span class="badge {{ auth()->user()->isAdmin() ? 'text-bg-primary' : 'text-bg-secondary' }} mt-1">{{ auth()->user()->role->label() }}</span>
                        </li>
                        <li><hr class="dropdown-divider"></li>
                        <li>
                            <form method="POST" action="{{ route('logout') }}">
                                @csrf
                                <button type="submit" class="dropdown-item text-danger">
                                    <i class="bi bi-box-arrow-right me-2"></i>Logout
                                </button>
                            </form>
                        </li>
                    </ul>
                </div>
            </div>
        </div>
    </nav>
```

*Gambar 8.15 Navbar berbasis role pada layouts/app.blade.php*

13. Buat *file* resources/views/layouts/guest.blade.php sebagai *layout* halaman *login* dan register:

```blade
<!DOCTYPE html>
<html lang="{{ str_replace('_', '-', app()->getLocale()) }}">
<head>
    @include('layouts.partials.head')
</head>
<body>
    <div class="container">
        <div class="row justify-content-center align-items-center
min-vh-100 py-4">
            <div class="col-sm-10 col-md-7 col-lg-5 col-xl-4">
                <div class="text-center mb-4">
```

```
                    <span class="auth-logo mb-2"><i class="bi bi-
mortarboard"></i></span>
                    <h5 class="fw-semibold mb-0">{{
config('app.name', 'Laravel') }}</h5>
                </div>

                <div class="card shadow-sm">
                    <div class="card-body p-4">
                        @yield('content')
                    </div>
                </div>
            </div>
        </div>
    </div>

    @include('layouts.partials.scripts')
</body>
</html>
```

14. Buat *folder* resources/views/auth, lalu buat *file* resources/views/auth/login.blade.php:

```
@extends('layouts.guest')

@section('title', 'Login')

@section('content')
    <h5 class="mb-1">Masuk</h5>
    <p class="text-muted small mb-4">Gunakan email dan password akun
Anda.</p>

    <form method="POST" action="{{ route('login.store') }}">
        @csrf

        <div class="mb-3">
            <label for="email" class="form-label">Email</label>
            <input type="email" class="form-control @error('email')
is-invalid @enderror" id="email" name="email" value="{{ old('email')
}}" placeholder="nama@email.com" required autofocus
autocomplete="username">
            @error('email')
                <div class="invalid-feedback">{{ $message }}</div>
            @enderror
        </div>

        <div class="mb-3">
            <label for="password" class="form-label">Password</label>
            <div class="input-group has-validation">
                <input type="password" class="form-control
@error('password') is-invalid @enderror" id="password"
name="password" required autocomplete="current-password">
                <button type="button" class="btn btn-outline-
secondary" data-toggle-password="password" aria-label="Tampilkan
password">
                    <i class="bi bi-eye"></i>
```

```
                </button>
                @error('password')
                    <div class="invalid-feedback">{{ $message
}}</div>
                @enderror
            </div>
        </div>

        <div class="mb-4 form-check">
            <input type="checkbox" class="form-check-input"
id="remember" name="remember">
            <label class="form-check-label" for="remember">Ingat
saya</label>
        </div>

        <button type="submit" class="btn btn-primary w-
100">Masuk</button>
    </form>

    <p class="text-center small text-muted mt-4 mb-0">
        Belum punya akun? <a href="{{ route('register') }}">Daftar di
sini</a>
    </p>
@endsection
```

- @error('email') is-invalid @enderror → memberi garis merah pada *input* dan menampilkan pesan *error* di bawahnya.
- value="{{ old('email') }}" → mengisi kembali *email* ketika *login* gagal, sedangkan *password* tidak diisi ulang.




*Gambar 8.16 View auth/login.blade.php*

15. Buat *file* resources/views/auth/register.blade.php:

```
@extends('layouts.guest')
```

```
@section('title', 'Register')

@section('content')
    <h5 class="mb-1">Buat Akun</h5>
    <p class="text-muted small mb-4">Akun baru otomatis terdaftar
sebagai User.</p>

    <form method="POST" action="{{ route('register.store') }}">
        @csrf

        <div class="mb-3">
            <label for="name" class="form-label">Nama Lengkap</label>
            <input type="text" class="form-control @error('name') is-
invalid @enderror" id="name" name="name" value="{{ old('name') }}"
required autofocus autocomplete="name">
            @error('name')
                <div class="invalid-feedback">{{ $message }}</div>
            @enderror
        </div>

        <div class="mb-3">
            <label for="email" class="form-label">Email</label>
            <input type="email" class="form-control @error('email')
is-invalid @enderror" id="email" name="email" value="{{ old('email')
}}" placeholder="nama@email.com" required autocomplete="username">
            @error('email')
                <div class="invalid-feedback">{{ $message }}</div>
            @enderror
        </div>

        <div class="mb-3">
            <label for="password" class="form-label">Password</label>
            <div class="input-group has-validation">
                <input type="password" class="form-control
@error('password') is-invalid @enderror" id="password"
name="password" required autocomplete="new-password">
                <button type="button" class="btn btn-outline-
secondary" data-toggle-password="password" aria-label="Tampilkan
password">
                    <i class="bi bi-eye"></i>
                </button>
                @error('password')
                    <div class="invalid-feedback">{{ $message
}}</div>
                @enderror
            </div>
            <div class="form-text">Minimal 8 karakter.</div>
        </div>

        <div class="mb-4">
            <label for="password_confirmation" class="form-
label">Ulangi Password</label>
```

```
            <input type="password" class="form-control"
id="password_confirmation" name="password_confirmation" required
autocomplete="new-password">
        </div>

        <button type="submit" class="btn btn-primary w-
100">Daftar</button>
    </form>

    <p class="text-center small text-muted mt-4 mb-0">
        Sudah punya akun? <a href="{{ route('login') }}">Masuk di
sini</a>
    </p>
@endsection
```

- name="password_confirmation" → nama *input* harus diakhiri _confirmation agar aturan confirmed dapat membandingkannya dengan *input password*.

16. Buat *file* resources/views/dashboard.blade.php:

```
@extends('layouts.app')

@section('title', 'Dashboard')

@section('content')
    <div class="mb-4">
        <h4 class="mb-1">Halo, {{ auth()->user()->name }}</h4>
        <p class="text-muted mb-0">Anda login sebagai <strong>{{
auth()->user()->role->label() }}</strong>. Pilih menu di bawah untuk
mulai bekerja.</p>
    </div>

    <div class="row g-3">
        <div class="col-md-6 col-lg-4">
            <div class="card h-100">
                <div class="card-body d-flex align-items-center gap-
3">
                    <span class="icon-box green"><i class="bi bi-
clipboard-check"></i></span>
                    <div class="flex-grow-1">
                        <div class="text-muted small">Total
Pendaftar</div>
                        <div class="fs-4 fw-semibold">{{
$totalPendaftar }}</div>
                    </div>
                </div>
                <div class="card-footer bg-white border-top-0 pt-0">
                    <a href="{{ route('pendaftaran.index') }}"
class="small text-decoration-none">
                        Kelola pendaftaran <i class="bi bi-arrow-
right"></i>
                    </a>
                </div>
            </div>
```

```
        </div>

        @if (auth()->user()->isAdmin())
            <div class="col-md-6 col-lg-4">
                <div class="card h-100">
                    <div class="card-body d-flex align-items-center
gap-3">
                        <span class="icon-box blue"><i class="bi bi-
people"></i></span>
                        <div class="flex-grow-1">
                            <div class="text-muted small">Total
User</div>
                            <div class="fs-4 fw-semibold">{{
$totalUsers }}</div>
                        </div>
                    </div>
                    <div class="card-footer bg-white border-top-0 pt-
0">
                        <a href="{{ route('admin.users.index') }}"
class="small text-decoration-none">
                            Kelola User <i class="bi bi-arrow-
right"></i>
                        </a>
                    </div>
                </div>
            </div>
        @endif
    </div>
@endsection
```

**Tahap C: *Authorization* dengan *Middleware Role***

*Middleware* auth hanya memastikan pengguna sudah *login*. Untuk membedakan hak akses *admin* dan *user*, kita membuat *middleware* sendiri yang memeriksa kolom *role*.

1. Buat *middleware* dengan perintah berikut:

```
php artisan make:middleware EnsureUserHasRole
```

Buka *file* app/Http/Middleware/EnsureUserHasRole.php, lalu ganti isinya dengan kode berikut:

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserHasRole
{
    /**
     * Ensure the authenticated user has one of the given roles.
     *
```

```
     * Usage: ->middleware('role:admin') or -
>middleware('role:admin,user')
     *
     * @param  Closure(Request): (Response)  $next
     */
    public function handle(Request $request, Closure $next, string
...$roles): Response
    {
        $user = $request->user();

        if ($user === null || ! $user->hasRole(...$roles)) {
            abort(403, 'Anda tidak memiliki hak akses ke halaman
ini.');
        }

        return $next($request);
    }
}
```

- string ...$roles → menampung parameter *middleware*. Penulisan role:admin,*user* menghasilkan $roles = ['admin', 'user'].
- $user->hasRole(...$roles) → memeriksa apakah *role* pengguna termasuk salah satu *role* yang diizinkan.
- abort(403, ...) → menghentikan *request* dengan *error* **403 *Forbidden*** beserta pesan berbahasa Indonesia.
- $next($request) → meneruskan *request* ke *controller* jika *role* sesuai.

app/Http/Middleware/EnsureUserHasRole.php

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserHasRole
{
    /**
     * Ensure the authenticated user has one of the given roles.
     *
     * Usage: ->middleware('role:admin') or ->middleware('role:admin,user')
     *
     * @param  Closure(Request): (Response)  $next
     */
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        $user = $request->user();

        if ($user === null || ! $user->hasRole(...$roles)) {
            abort(403, 'Anda tidak memiliki hak akses ke halaman ini.');
        }

        return $next($request);
    }
}
```

*Gambar 8.17 Middleware EnsureUserHasRole*

2. Daftarkan *middleware* dengan nama alias ***role*** pada *file* bootstrap/app.php:

```php
<?php

use App\Http\Middleware\EnsureUserHasRole;
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->alias([
            'role' => EnsureUserHasRole::class,
        ]);

        $middleware->redirectGuestsTo(fn () => route('login'));
        $middleware->redirectUsersTo(fn () => route('dashboard'));
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
```

```
    })->create();
```

- $middleware->alias(['role' => EnsureUserHasRole::class]) → *middleware* dapat dipanggil dengan nama pendek *role* di *route*. Laravel 12 tidak lagi memiliki *file* app/Http/Kernel.php, sehingga pendaftaran *middleware* dilakukan di bootstrap/app.php.
- redirectGuestsTo(...) → tamu yang membuka halaman ber-*middleware* auth diarahkan ke *route login*.
- redirectUsersTo(...) → pengguna yang sudah *login* dan membuka halaman ber-*middleware guest* diarahkan ke *dashboard*.

bootstrap/app.php

```php
<?php

use App\Http\Middleware\EnsureUserHasRole;
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->alias([
            'role' => EnsureUserHasRole::class,
        ]);

        $middleware->redirectGuestsTo(fn () => route('login'));
        $middleware->redirectUsersTo(fn () => route('dashboard'));
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();
```

*Gambar 8.18 Alias middleware role pada bootstrap/app.php*

3. *Middleware role* digunakan pada *route* modul dengan cara berikut (diterapkan pada Tahap D dan E):

```
// Hanya admin
Route::middleware(['auth', 'role:admin'])->group(function () { ...
});

// Admin dan user
Route::middleware(['auth', 'role:admin,user'])->group(function () {
... });
```

- Urutan *middleware* penting: auth dijalankan lebih dulu agar tamu diarahkan ke *login*, baru kemudian *role* memeriksa hak akses pengguna yang sudah *login*.

**Tahap D: Modul UserManagement (CRUD *Synchronous* Khusus *Admin*)**

Modul ini memungkinkan *admin* menambah, mengubah, dan menghapus akun pengguna. Setiap aksi dikirim melalui *form* biasa sehingga halaman dimuat ulang (*synchronous*).

1. Buat dua *Form Request* di dalam modul UserManagement:

```
php artisan module:make-request StoreUserRequest UserManagement
php artisan module:make-request UpdateUserRequest UserManagement
```

Buka *file* Modules/UserManagement/app/Http/Requests/StoreUserRequest.php, lalu ganti isinya dengan kode berikut:

```
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class StoreUserRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255',
'unique:users,email'],
            'role' => ['required', Rule::enum(UserRole::class)],
            'password' => ['required', 'confirmed',
Password::defaults()],
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }
}
```

- Rule::enum(UserRole::class) → *role* hanya boleh bernilai *admin* atau *user*. Nilai lain seperti superadmin ditolak.
- authorize() → lapisan pengaman kedua: *request* ditolak (403) jika yang mengirim bukan *admin*, walaupun *route* sudah dilindungi *middleware*.

Modules/UserManagement/app/Http/Requests/StoreUserRequest.php

```php
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class StoreUserRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', 'unique:users,email'],
            'role' => ['required', Rule::enum(UserRole::class)],
            'password' => ['required', 'confirmed', Password::defaults()],
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }
}
```

*Gambar 8.19 StoreUserRequest*

Buka *file* Modules/UserManagement/app/Http/Requests/UpdateUserRequest.php, lalu ganti isinya dengan kode berikut:

```
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class UpdateUserRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * An admin cannot demote their own account, so they never lock
themselves out.
     *
```

```
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $user = $this->route('user');
        $isEditingOwnAccount = $user?->is($this->user()) ?? false;

        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255',
Rule::unique('users', 'email')->ignore($user?->id)],
            'role' => [
                'required',
                Rule::enum(UserRole::class),
                Rule::when($isEditingOwnAccount,
Rule::in([UserRole::Admin->value])),
            ],
            'password' => ['nullable', 'confirmed',
Password::defaults()],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'role.in' => 'Anda tidak dapat menurunkan role akun Anda
sendiri.',
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }
}
```

- Rule::unique('users', 'email')->ignore($user?->id) → *email* boleh sama dengan *email* milik *user* itu sendiri, tetapi tidak boleh sama dengan *user* lain.
- Rule::when($isEditingOwnAccount, Rule::in(['admin'])) → *admin* tidak dapat menurunkan *role* akunnya sendiri menjadi *user*, agar tidak terkunci dari menu *admin*.
- 'password' => ['nullable', ...] → *password* boleh dikosongkan ketika edit; *password* lama tetap digunakan.

Modules/UserManagement/app/Http/Requests/UpdateUserRequest.php

```php
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class UpdateUserRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * An admin cannot demote their own account, so they never lock themselves out.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $user = $this->route('user');
        $isEditingOwnAccount = $user?->is($this->user()) ?? false;

        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', Rule::unique('users', 'email')->ignore($user?->id)],
            'role' => [
                'required',
                Rule::enum(UserRole::class),
                Rule::when($isEditingOwnAccount, Rule::in([UserRole::Admin->value])),
            ],
            'password' => ['nullable', 'confirmed', Password::defaults()],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'role.in' => 'Anda tidak dapat menurunkan role akun Anda sendiri.',
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return $this->user()?->isAdmin() ?? false;
    }
}
```

*Gambar 8.20 UpdateUserRequest*

2. Buka *file* Modules/UserManagement/app/Http/Controllers/UserManagementController.php, lalu ganti isinya dengan kode berikut:

```php
<?php

namespace Modules\UserManagement\Http\Controllers;

use App\Enums\UserRole;
use App\Http\Controllers\Controller;
use App\Models\User;
```

```
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\View\View;
use Modules\UserManagement\Http\Requests\StoreUserRequest;
use Modules\UserManagement\Http\Requests\UpdateUserRequest;

class UserManagementController extends Controller
{
    /**
     * Display a listing of users, optionally filtered by keyword and
role.
     */
    public function index(Request $request): View
    {
        $search = $request->string('search')->trim()->toString();
        $role = UserRole::tryFrom($request->string('role')-
>toString());

        $users = User::query()
            ->when($search !== '', function (Builder $query) use
($search) {
                $query->where(function (Builder $query) use ($search)
{
                    $query->where('name', 'like', "%{$search}%")
                        ->orWhere('email', 'like', "%{$search}%");
                });
            })
            ->when($role, fn (Builder $query) => $query-
>where('role', $role))
            ->orderBy('name')
            ->paginate(10)
            ->withQueryString();

        return view('usermanagement::index', [
            'users' => $users,
            'search' => $search,
            'selectedRole' => $role,
            'roles' => UserRole::cases(),
        ]);
    }

    /**
     * Show the form for creating a new user.
     */
    public function create(): View
    {
        return view('usermanagement::create', ['roles' =>
UserRole::cases()]);
    }

    /**
     * Store a newly created user.
```

```
     */
    public function store(StoreUserRequest $request):
RedirectResponse
    {
        User::create($request->validated());

        return redirect()->route('admin.users.index')-
>with('success', 'User berhasil ditambahkan.');
    }

    /**
     * Show the form for editing the specified user.
     */
    public function edit(User $user): View
    {
        return view('usermanagement::edit', [
            'user' => $user,
            'roles' => UserRole::cases(),
        ]);
    }

    /**
     * Update the specified user. The password is only changed when
filled in.
     */
    public function update(UpdateUserRequest $request, User $user):
RedirectResponse
    {
        $attributes = $request->validated();

        if (blank($attributes['password'] ?? null)) {
            unset($attributes['password']);
        }

        $user->update($attributes);

        return redirect()->route('admin.users.index')-
>with('success', 'User berhasil diperbarui.');
    }

    /**
     * Remove the specified user. An admin cannot delete their own
account.
     */
    public function destroy(Request $request, User $user):
RedirectResponse
    {
        if ($user->is($request->user())) {
            return redirect()->route('admin.users.index')-
>with('error', 'Anda tidak dapat menghapus akun Anda sendiri.');
        }

        $user->delete();
```

```
        return redirect()->route('admin.users.index')-
>with('success', 'User berhasil dihapus.');
    }
}
```

- index() → menampilkan daftar *user* dengan pencarian (*search*) berdasarkan nama atau *email*, filter *role*, dan *pagination* 10 data per halaman. withQueryString() menjaga kata kunci pencarian saat berpindah halaman.
- view('usermanagement::index') → memanggil *view* dari dalam modul menggunakan *namespace* modul.
- store() dan update() → hanya menyimpan data hasil validasi ($request->validated()). Pada update, *password* dihapus dari data jika dikosongkan.
- destroy() → *admin* tidak dapat menghapus akunnya sendiri; aplikasi kembali ke daftar *user* dengan pesan *error*.
- *User* $user → *route* model *binding*; Laravel otomatis mengambil data *user* berdasarkan id pada URL dan menampilkan 404 jika tidak ditemukan.

Modules/UserManagement/app/Http/Controllers/UserManagementController.php

```php
<?php

namespace Modules\UserManagement\Http\Controllers;

use App\Enums\UserRole;
use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\View\View;
use Modules\UserManagement\Http\Requests\StoreUserRequest;
use Modules\UserManagement\Http\Requests\UpdateUserRequest;

class UserManagementController extends Controller
{
    /**
     * Display a listing of users, optionally filtered by keyword and role.
     */
    public function index(Request $request): View
    {
        $search = $request->string('search')->trim()->toString();
        $role = UserRole::tryFrom($request->string('role')->toString());

        $users = User::query()
            ->when($search !== '', function (Builder $query) use ($search) {
                $query->where(function (Builder $query) use ($search) {
                    $query->where('name', 'like', "%{$search}%")
                        ->orWhere('email', 'like', "%{$search}%");
                });
            })
            ->when($role, fn (Builder $query) => $query->where('role', $role))
            ->orderBy('name')
            ->paginate(10)
            ->withQueryString();

        return view('usermanagement::index', [
            'users' => $users,
            'search' => $search,
            'selectedRole' => $role,
            'roles' => UserRole::cases(),
        ]);
    }

    /**
     * Show the form for creating a new user.
     */
    public function create(): View
    {
        return view('usermanagement::create', ['roles' => UserRole::cases()]);
    }

    /**
     * Store a newly created user.
     */
    public function store(StoreUserRequest $request): RedirectResponse
    {
        User::create($request->validated());

        return redirect()->route('admin.users.index')->with('success', 'User berhasil ditambahkan.');
    }

    /**
     * Show the form for editing the specified user.
```

*Gambar 8.21 UserManagementController bagian 1 (index, create, store)*

Modules/UserManagement/app/Http/Controllers/UserManagementController.php

```php
     */
    public function edit(User $user): View
    {
        return view('usermanagement::edit', [
            'user' => $user,
            'roles' => UserRole::cases(),
        ]);
    }

    /**
     * Update the specified user. The password is only changed when filled in.
     */
    public function update(UpdateUserRequest $request, User $user): RedirectResponse
    {
        $attributes = $request->validated();

        if (blank($attributes['password'] ?? null)) {
            unset($attributes['password']);
        }

        $user->update($attributes);

        return redirect()->route('admin.users.index')->with('success', 'User berhasil diperbarui.');
    }

    /**
     * Remove the specified user. An admin cannot delete their own account.
     */
    public function destroy(Request $request, User $user): RedirectResponse
    {
        if ($user->is($request->user())) {
            return redirect()->route('admin.users.index')->with('error', 'Anda tidak dapat menghapus akun Anda sendiri.');
        }

        $user->delete();

        return redirect()->route('admin.users.index')->with('success', 'User berhasil dihapus.');
    }
}
```

*Gambar 8.22 UserManagementController bagian 2 (edit, update, destroy)*

3. Buka *file* Modules/UserManagement/routes/web.php, lalu ganti isinya dengan kode berikut:

```
<?php

use Illuminate\Support\Facades\Route;
use Modules\UserManagement\Http\Controllers\UserManagementController;

Route::middleware(['auth', 'role:admin'])
    ->prefix('admin')
    ->name('admin.')
    ->group(function () {
        Route::resource('users', UserManagementController::class)-
>except('show');
    });
```

- middleware(['auth', 'role:admin']) → seluruh *route* modul ini hanya untuk *admin*.
- prefix('admin') dan name('admin.') → URL menjadi /admin/users dan nama *route* menjadi admin.users.index, admin.users.create, dan seterusnya.
- Route::resource(...)->except('show') → membuat enam *route* CRUD sekaligus tanpa halaman detail.

Modules/UserManagement/routes/web.php

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\UserManagement\Http\Controllers\UserManagementController;

Route::middleware(['auth', 'role:admin'])
    ->prefix('admin')
    ->name('admin.')
    ->group(function () {
        Route::resource('users', UserManagementController::class)->except('show');
    });
```

*Gambar 8.23 Route modul UserManagement*

4. Buka *file* Modules/UserManagement/resources/views/index.blade.php, lalu ganti isinya dengan kode berikut:

```
@extends('layouts.app')

@section('title', 'Kelola User')

@section('content')
    <div class="d-flex flex-wrap justify-content-between align-items-
center gap-2 mb-4">
        <div>
            <h4 class="mb-1">Kelola User</h4>
            <p class="text-muted mb-0 small">Tambah, ubah, dan hapus
akun pengguna aplikasi.</p>
        </div>
        <a href="{{ route('admin.users.create') }}" class="btn btn-
primary">
            <i class="bi bi-person-plus me-1"></i> Tambah User
        </a>
    </div>

    <div class="card">
        <div class="card-header py-3">
            <form method="GET" action="{{ route('admin.users.index')
}}" class="row g-2 align-items-center">
                <div class="col-sm-6 col-lg-4">
                    <div class="input-group">
                        <span class="input-group-text bg-white"><i
class="bi bi-search"></i></span>
                        <input type="search" name="search" value="{{
$search }}" class="form-control border-start-0" placeholder="Cari
nama atau email...">
                    </div>
                </div>
                <div class="col-sm-3 col-lg-2">
                    <select name="role" class="form-select"
onchange="this.form.submit()" aria-label="Filter role">
                        <option value="">Semua role</option>
                        @foreach ($roles as $role)
```

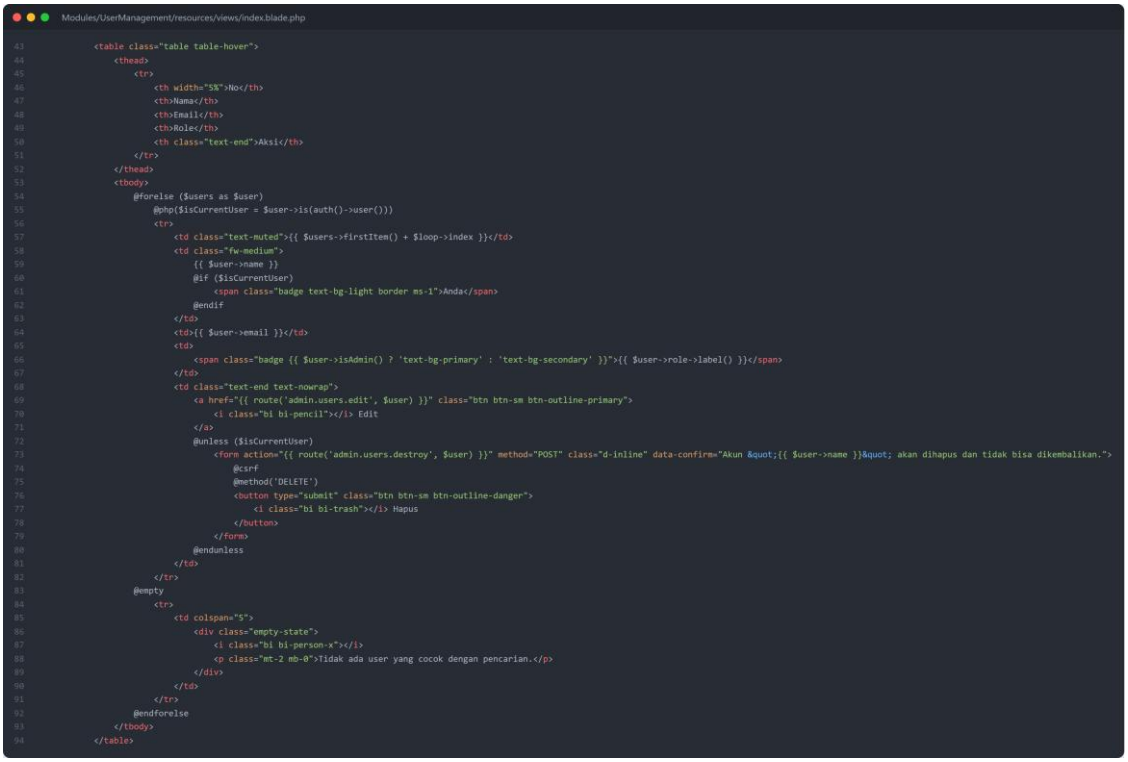
```
                            <option value="{{ $role->value }}"
@selected($selectedRole === $role)>{{ $role->label() }}</option>
                        @endforeach
                    </select>
                </div>
                <div class="col-auto">
                    <button type="submit" class="btn btn-light
border">Cari</button>
                    @if ($search !== '' || $selectedRole)
                        <a href="{{ route('admin.users.index') }}"
class="btn btn-link text-decoration-none">Reset</a>
                    @endif
                </div>
            </form>
        </div>

        <div class="table-responsive">
            <table class="table table-hover">
                <thead>
                    <tr>
                        <th width="5%">No</th>
                        <th>Nama</th>
                        <th>Email</th>
                        <th>Role</th>
                        <th class="text-end">Aksi</th>
                    </tr>
                </thead>
                <tbody>
                    @forelse ($users as $user)
                        @php($isCurrentUser = $user->is(auth()-
>user()))
                        <tr>
                            <td class="text-muted">{{ $users-
>firstItem() + $loop->index }}</td>
                            <td class="fw-medium">
                                {{ $user->name }}
                                @if ($isCurrentUser)
                                    <span class="badge text-bg-light
border ms-1">Anda</span>
                                @endif
                            </td>
                            <td>{{ $user->email }}</td>
                            <td>
                                <span class="badge {{ $user-
>isAdmin() ? 'text-bg-primary' : 'text-bg-secondary' }}">{{ $user-
>role->label() }}</span>
                            </td>
                            <td class="text-end text-nowrap">
                                <a href="{{ route('admin.users.edit',
$user) }}" class="btn btn-sm btn-outline-primary">
                                    <i class="bi bi-pencil"></i> Edit
                                </a>
                                @unless ($isCurrentUser)
```

```
                                    <form action="{{
route('admin.users.destroy', $user) }}" method="POST" class="d-
inline" data-confirm="Akun &amp;quot;{{ $user->name }}&amp;quot; akan dihapus
dan tidak bisa dikembalikan.">
                                        @csrf
                                        @method('DELETE')
                                        <button type="submit"
class="btn btn-sm btn-outline-danger">
                                            <i class="bi bi-
trash"></i> Hapus
                                        </button>
                                    </form>
                                @endunless
                            </td>
                        </tr>
                    @empty
                        <tr>
                            <td colspan="5">
                                <div class="empty-state">
                                    <i class="bi bi-person-x"></i>
                                    <p class="mt-2 mb-0">Tidak ada
user yang cocok dengan pencarian.</p>
                                </div>
                            </td>
                        </tr>
                    @endforelse
                </tbody>
            </table>
        </div>

        @if ($users->hasPages())
            <div class="card-footer bg-white d-flex flex-wrap
justify-content-between align-items-center gap-2">
                <small class="text-muted">Menampilkan {{ $users-
>firstItem() }}-{{ $users->lastItem() }} dari {{ $users->total() }}
user</small>
                {{ $users->links('pagination::bootstrap-4') }}
            </div>
        @endif
    </div>
@endsection
```

- @extends('layouts.app') → *view* modul tetap memakai *layout* aplikasi utama, sehingga tampilan seluruh modul seragam.
- {{ $users->firstItem() + $loop->index }} → nomor urut tetap berlanjut pada halaman *pagination* berikutnya.
- data-confirm="..." → *form* hapus otomatis meminta konfirmasi melalui *modal* sebelum dikirim. Tombol Hapus tidak ditampilkan pada baris akun yang sedang *login*.
- onchange="this.form.submit()" pada filter *role* → daftar langsung diperbarui ketika *role* dipilih.




*Gambar 8.24 Tabel daftar user pada view index*

5. Buat *folder* Modules/UserManagement/resources/views/partials, lalu buat *file* partials/form.blade.php yang dipakai bersama oleh *form* tambah dan edit:

```
<div class="mb-3">
    <label for="name" class="form-label">Nama Lengkap <span
class="text-danger">*</span></label>
    <input type="text" class="form-control @error('name') is-invalid
@enderror" id="name" name="name" value="{{ old('name', $user?->name)
}}" required>
    @error('name')
        <div class="invalid-feedback">{{ $message }}</div>
    @enderror
</div>

<div class="mb-3">
    <label for="email" class="form-label">Email <span class="text-
danger">*</span></label>
    <input type="email" class="form-control @error('email') is-
invalid @enderror" id="email" name="email" value="{{ old('email',
$user?->email) }}" placeholder="nama@email.com" required>
    @error('email')
        <div class="invalid-feedback">{{ $message }}</div>
    @enderror
</div>

<div class="mb-3">
    <label for="role" class="form-label">Role <span class="text-
danger">*</span></label>
    <select class="form-select @error('role') is-invalid @enderror"
id="role" name="role" required>
```

```
        @foreach ($roles as $role)
            <option value="{{ $role->value }}" @selected(old('role',
$user?->role->value ?? 'user') === $role->value)>{{ $role->label()
}}</option>
        @endforeach
    </select>
    @error('role')
        <div class="invalid-feedback">{{ $message }}</div>
    @enderror
    <div class="form-text">Admin dapat mengelola user. User hanya
dapat mengakses Pendaftaran.</div>
</div>

<hr class="my-4">

<div class="row g-3 mb-3">
    <div class="col-md-6">
        <label for="password" class="form-label">
            Password @unless ($user)<span class="text-
danger">*</span>@endunless
        </label>
        <div class="input-group has-validation">
            <input type="password" class="form-control
@error('password') is-invalid @enderror" id="password"
name="password" autocomplete="new-password" @required(! $user)>
            <button type="button" class="btn btn-outline-secondary"
data-toggle-password="password" aria-label="Tampilkan password">
                <i class="bi bi-eye"></i>
            </button>
            @error('password')
                <div class="invalid-feedback">{{ $message }}</div>
            @enderror
        </div>
        <div class="form-text">
            {{ $user ? 'Kosongkan jika tidak ingin mengubah
password.' : 'Minimal 8 karakter.' }}
        </div>
    </div>

    <div class="col-md-6">
        <label for="password_confirmation" class="form-label">Ulangi
Password</label>
        <input type="password" class="form-control"
id="password_confirmation" name="password_confirmation"
autocomplete="new-password" @required(! $user)>
    </div>
</div>
```

- old('name', $user?->name) → pada *form* tambah *input* kosong ($user bernilai null), sedangkan pada *form* edit *input* berisi data lama.
- @required(! $user) → *password* wajib diisi saat menambah *user*, tetapi opsional saat edit.

Modules/UserManagement/resources/views/partials/form.blade.php

```blade
<div class="mb-3">
    <label for="name" class="form-label">Nama Lengkap <span class="text-danger">*</span></label>
    <input type="text" class="form-control @error('name') is-invalid @enderror" id="name" name="name" value="{{ old('name', $user?->name) }}" required>
    @error('name')
        <div class="invalid-feedback">{{ $message }}</div>
    @enderror
</div>

<div class="mb-3">
    <label for="email" class="form-label">Email <span class="text-danger">*</span></label>
    <input type="email" class="form-control @error('email') is-invalid @enderror" id="email" name="email" value="{{ old('email', $user?->email) }}" placeholder="nama@email.com" required>
    @error('email')
        <div class="invalid-feedback">{{ $message }}</div>
    @enderror
</div>

<div class="mb-3">
    <label for="role" class="form-label">Role <span class="text-danger">*</span></label>
    <select class="form-select @error('role') is-invalid @enderror" id="role" name="role" required>
        @foreach ($roles as $role)
            <option value="{{ $role->value }}" @selected(old('role', $user?->role->value ?? 'user') === $role->value)>{{ $role->label() }}</option>
        @endforeach
    </select>
    @error('role')
        <div class="invalid-feedback">{{ $message }}</div>
    @enderror
    <div class="form-text">Admin dapat mengelola user. User hanya dapat mengakses Pendaftaran.</div>
</div>

<hr class="my-4">

<div class="row g-3 mb-3">
    <div class="col-md-6">
        <label for="password" class="form-label">
            Password @unless ($user)<span class="text-danger">*</span>@endunless
        </label>
        <div class="input-group has-validation">
            <input type="password" class="form-control @error('password') is-invalid @enderror" id="password" name="password" autocomplete="new-password" @required(! $user)>
            <button type="button" class="btn btn-outline-secondary" data-toggle-password="password" aria-label="Tampilkan password">
                <i class="bi bi-eye"></i>
            </button>
            @error('password')
                <div class="invalid-feedback">{{ $message }}</div>
            @enderror
        </div>
        <div class="form-text">
            {{ $user ? 'Kosongkan jika tidak ingin mengubah password.' : 'Minimal 8 karakter.' }}
        </div>
    </div>

    <div class="col-md-6">
        <label for="password_confirmation" class="form-label">Ulangi Password</label>
        <input type="password" class="form-control" id="password_confirmation" name="password_confirmation" autocomplete="new-password" @required(! $user)>
    </div>
</div>
```

*Gambar 8.25 Partial form user*

6. Buat *file* Modules/UserManagement/resources/views/create.blade.php:

```
@extends('layouts.app')

@section('title', 'Tambah User')

@section('content')
    <div class="row justify-content-center">
        <div class="col-lg-7">
            <a href="{{ route('admin.users.index') }}" class="text-
decoration-none small">
                <i class="bi bi-arrow-left"></i> Kembali ke daftar
user
            </a>
            <h4 class="mt-2 mb-4">Tambah User</h4>

            <div class="card">
                <div class="card-body p-4">
                    <form action="{{ route('admin.users.store') }}"
method="POST">
                        @csrf
                        @include('usermanagement::partials.form',
['user' => null])

                        <div class="d-flex justify-content-end gap-
2">
```

```
                            <a href="{{ route('admin.users.index')
}}" class="btn btn-light">Batal</a>
                            <button type="submit" class="btn btn-
primary">Simpan</button>
                        </div>
                    </form>
                </div>
            </div>
        </div>
    </div>
@endsection
```

7. Buat *file* Modules/UserManagement/resources/views/edit.blade.php:

```
@extends('layouts.app')

@section('title', 'Edit User')

@section('content')
    <div class="row justify-content-center">
        <div class="col-lg-7">
            <a href="{{ route('admin.users.index') }}" class="text-
decoration-none small">
                <i class="bi bi-arrow-left"></i> Kembali ke daftar
user
            </a>
            <h4 class="mt-2 mb-4">Edit User</h4>

            <div class="card">
                <div class="card-body p-4">
                    <form action="{{ route('admin.users.update',
$user) }}" method="POST">
                        @csrf
                        @method('PUT')
                        @include('usermanagement::partials.form',
['user' => $user])

                        <div class="d-flex justify-content-end gap-
2">
                            <a href="{{ route('admin.users.index')
}}" class="btn btn-light">Batal</a>
                            <button type="submit" class="btn btn-
primary">Simpan Perubahan</button>
                        </div>
                    </form>
                </div>
            </div>
        </div>
    </div>
@endsection
```

- @method('PUT') → *form* dikirim sebagai *request* PUT sehingga diproses oleh *method* update().

**Tahap E: Modul Pendaftaran (CRUD AJAX)**

Modul Pendaftaran mengelola data calon mahasiswa tanpa memuat ulang halaman. *Controller* tidak lagi mengembalikan *view* atau *redirect*, melainkan data JSON yang diolah oleh jQuery. Pada tahap ini modul juga dilengkapi fitur *upload* berkas pendukung yang dapat dibuka kembali melalui tautan pada tabel.

1. Buat model **Daftar** beserta *migration* di dalam modul Pendaftaran:

```
php artisan module:make-model Daftar Pendaftaran --migration
```

Buka *file* Modules/Pendaftaran/database/migrations/xxxx_xx_xx_xxxxxx_create_daftars_table.php, lalu ganti isinya dengan kode berikut:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('daftars', function (Blueprint $table) {
            $table->id();
            $table->string('nama');
            $table->string('asal_sekolah');
            $table->string('prodi_tujuan');
            $table->string('email')->unique();
            $table->string('no_hp', 20);
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('daftars');
    }
};
```

- Tabel daftars menyimpan lima item data: nama, asal_sekolah, prodi_tujuan, *email* (unik), dan no_hp.
- string('no_hp', 20) → nomor HP disimpan sebagai teks agar angka 0 di depan tidak hilang.

Modules/Pendaftaran/database/migrations/2026_11_01_071637_create_daftars_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('daftars', function (Blueprint $table) {
            $table->id();
            $table->string('nama');
            $table->string('asal_sekolah');
            $table->string('prodi_tujuan');
            $table->string('email')->unique();
            $table->string('no_hp', 20);
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('daftars');
    }
};
```

*Gambar 8.26 Migration tabel daftars*

2. Jalankan *migration*. *Migration* milik modul ikut dijalankan oleh perintah migrate biasa:

```
php artisan migrate

# atau khusus satu modul
php artisan module:migrate Pendaftaran
```

3. Tambahkan kolom untuk menyimpan berkas dengan membuat *migration* baru di dalam modul Pendaftaran:

```
php artisan module:make-migration add_berkas_to_daftars_table
Pendaftaran
```

Buka *file* Modules/Pendaftaran/database/migrations/xxxx_xx_xx_xxxxxx_add_berkas_to_daftars_table.php, lalu ganti isinya dengan kode berikut:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
```

```
return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('daftars', function (Blueprint $table) {
            $table->string('berkas')->nullable()->after('no_hp');
            $table->string('nama_berkas')->nullable()-
>after('berkas');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('daftars', function (Blueprint $table) {
            $table->dropColumn(['berkas', 'nama_berkas']);
        });
    }
};
```

- berkas → menyimpan *path* berkas hasil store(), misalnya berkas_pendaftaran/Xy12AbCd.pdf. Isi *file* tetap disimpan di *folder storage*, bukan di *database*.
- nama_berkas → menyimpan nama asli *file* (misalnya rapor_budi.png) agar mudah dikenali saat ditampilkan pada tabel.
- nullable() → kolom boleh kosong, sehingga data yang dibuat sebelum fitur *upload* ditambahkan tetap valid.

Modules/Pendaftaran/database/migrations/2026_11_01_091043_add_berkas_to_daftars_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('daftars', function (Blueprint $table) {
            $table->string('berkas')->nullable()->after('no_hp');
            $table->string('nama_berkas')->nullable()->after('berkas');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('daftars', function (Blueprint $table) {
            $table->dropColumn(['berkas', 'nama_berkas']);
        });
    }
};
```

*Gambar 8.27 Migration penambahan kolom berkas*

4. Jalankan *migration*, lalu buat *symbolic link* agar berkas di *folder storage* dapat dibuka dari *browser*:

```
php artisan migrate
php artisan storage:link
```

- storage:link → membuat *folder* pintasan public/storage yang mengarah ke storage/app/public, sehingga berkas dapat dibuka melalui asset('storage/' . $path). Perintah ini cukup dijalankan satu kali untuk setiap *project*. Jika muncul pesan *The [public/storage] link already exists*, berarti *link* sudah pernah dibuat.

Terminal

```
PS pertemuan8_hmvc_blade> php artisan storage:link


   INFO  The [E:\Tel-u\SIKC6 Ganjil\PABW\Pertemuan 8\pertemuan8_hmvc_blade\public\storage] link has been connecte
d to [E:\Tel-u\SIKC6 Ganjil\PABW\Pertemuan 8\pertemuan8_hmvc_blade\storage\app/public].
```

*Gambar 8.28 Hasil php artisan storage:link*

5. Buat *factory* untuk model Daftar (digunakan pada pengujian otomatis):

```
php artisan module:make-factory Daftar Pendaftaran
```

Buka *file* Modules/Pendaftaran/database/factories/DaftarFactory.php, lalu ganti isinya dengan kode berikut:

```php
<?php

namespace Modules\Pendaftaran\Database\Factories;

use Illuminate\Database\Eloquent\Factories\Factory;
use Modules\Pendaftaran\Models\Daftar;

/**
 * @extends Factory<Daftar>
 */
class DaftarFactory extends Factory
{
    /**
     * The name of the factory's corresponding model.
     */
    protected $model = Daftar::class;

    /**
     * Define the model's default state.
     *
     * @return array<string, mixed>
     */
    public function definition(): array
    {
        return [
            'nama' => fake()->name(),
            'asal_sekolah' => 'SMA '.fake()->city(),
            'prodi_tujuan' => fake()->randomElement(['Informatika',
'Sistem Informasi', 'Teknik Elektro']),
            'email' => fake()->unique()->safeEmail(),
            'no_hp' => fake()->numerify('08##########'),
        ];
    }
}
```

6. Buka *file* Modules/Pendaftaran/app/Models/Daftar.php, lalu ganti isinya dengan kode berikut:

```php
<?php

namespace Modules\Pendaftaran\Models;

use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Modules\Pendaftaran\Database\Factories\DaftarFactory;

class Daftar extends Model
{
    /** @use HasFactory<DaftarFactory> */
    use HasFactory;

    /**
```

```
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'nama',
        'asal_sekolah',
        'prodi_tujuan',
        'email',
        'no_hp',
        'berkas',
        'nama_berkas',
    ];

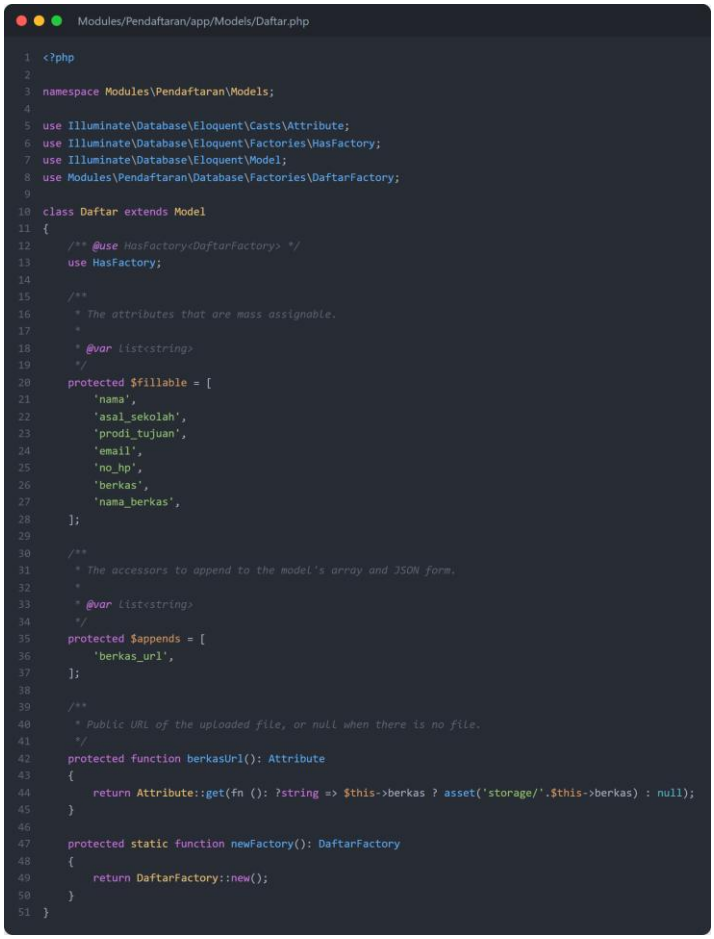
    /**
     * The accessors to append to the model's array and JSON form.
     *
     * @var list<string>
     */
    protected $appends = [
        'berkas_url',
    ];

    /**
     * Public URL of the uploaded file, or null when there is no
file.
     */
    protected function berkasUrl(): Attribute
    {
        return Attribute::get(fn (): ?string => $this->berkas ?
asset('storage/'.$this->berkas) : null);
    }

    protected static function newFactory(): DaftarFactory
    {
        return DaftarFactory::new();
    }
}
```

- $fillable → lima kolom data ditambah berkas dan nama_berkas yang boleh diisi melalui Daftar::create() dan update().
- newFactory() → menghubungkan model modul dengan *factory*-nya, karena *factory* modul berada di *namespace* yang berbeda dari *factory* aplikasi utama.
- $appends dan berkasUrl() → menambahkan atribut berkas_url ke data JSON. Nilainya dibuat dengan asset('storage/' . $this->berkas) sehingga JavaScript dapat langsung membuat tautan untuk membuka berkas.

Modules/Pendaftaran/app/Models/Daftar.php



```php
<?php

namespace Modules\Pendaftaran\Models;

use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Modules\Pendaftaran\Database\Factories\DaftarFactory;

class Daftar extends Model
{
    /** @use HasFactory<DaftarFactory> */
    use HasFactory;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    protected $fillable = [
        'nama',
        'asal_sekolah',
        'prodi_tujuan',
        'email',
        'no_hp',
        'berkas',
        'nama_berkas',
    ];

    /**
     * The accessors to append to the model's array and JSON form.
     *
     * @var list<string>
     */
    protected $appends = [
        'berkas_url',
    ];

    /**
     * Public URL of the uploaded file, or null when there is no file.
     */
    protected function berkasUrl(): Attribute
    {
        return Attribute::get(fn (): ?string => $this->berkas ? asset('storage/'.$this->berkas) : null);
    }

    protected static function newFactory(): DaftarFactory
    {
        return DaftarFactory::new();
    }
}
```

*Gambar 8.29 Model Daftar*

7. Buat *Form Request* untuk validasi data pendaftaran:

```
php artisan module:make-request DaftarRequest Pendaftaran
```

Buka *file* Modules/Pendaftaran/app/Http/Requests/DaftarRequest.php, lalu ganti isinya dengan kode berikut:

```php
<?php

namespace Modules\Pendaftaran\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class DaftarRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * The file is required when creating, and optional when updating
(keep the old file).
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
```

```
        $daftar = $this->route('daftar');

        return [
            'nama' => ['required', 'string', 'max:255'],
            'asal_sekolah' => ['required', 'string', 'max:255'],
            'prodi_tujuan' => ['required', 'string', 'max:255'],
            'email' => ['required', 'email', 'max:255',
Rule::unique('daftars', 'email')->ignore($daftar?->id)],
            'no_hp' => ['required', 'string', 'regex:/^[0-
9+]{8,20}$/'],
            'berkas' => [$daftar ? 'nullable' : 'required', 'file',
'mimes:jpg,jpeg,png,pdf', 'max:2048'],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'no_hp.regex' => 'No HP hanya boleh berisi angka (8-20
digit).',
            'berkas.required' => 'Berkas wajib diunggah.',
            'berkas.mimes' => 'Berkas harus berformat jpg, jpeg, png,
atau pdf.',
            'berkas.max' => 'Ukuran berkas maksimal 2 MB.',
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }
}
```

- Satu *request* dipakai untuk tambah dan edit. ignore($this->route('daftar')?->id) membuat *email* milik data yang sedang diedit tidak dianggap duplikat.
- regex:/^[0-9+]{8,20}$/ → nomor HP hanya boleh berisi angka (dan tanda +) sepanjang 8–20 karakter. Pesan *error*-nya diganti melalui *method* messages().
- Karena *request* AJAX mengirim *header* Accept: application/json, *error* validasi dikembalikan sebagai JSON dengan kode **422**, bukan *redirect*.
- berkas → wajib diunggah saat menambah data (required) dan boleh dikosongkan saat edit (nullable) karena berkas lama tetap digunakan. Aturan file, mimes:jpg,jpeg,png,pdf, dan max:2048 hanya menerima gambar atau PDF berukuran maksimal 2 MB, sehingga *file* berbahaya seperti .php ikut ditolak.

- messages() → mengganti pesan *error* berkas menjadi Bahasa Indonesia, misalnya *Berkas harus berformat jpg, jpeg, png, atau pdf*.

Modules/Pendaftaran/app/Http/Requests/DaftarRequest.php

```php
<?php

namespace Modules\Pendaftaran\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;

class DaftarRequest extends FormRequest
{
    /**
     * Get the validation rules that apply to the request.
     *
     * The file is required when creating, and optional when updating (keep the old file).
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        $daftar = $this->route('daftar');

        return [
            'nama' => ['required', 'string', 'max:255'],
            'asal_sekolah' => ['required', 'string', 'max:255'],
            'prodi_tujuan' => ['required', 'string', 'max:255'],
            'email' => ['required', 'email', 'max:255', Rule::unique('daftars', 'email')->ignore($daftar?->id)],
            'no_hp' => ['required', 'string', 'regex:/^[0-9+]{8,20}$/'],
            'berkas' => [$daftar ? 'nullable' : 'required', 'file', 'mimes:jpg,jpeg,png,pdf', 'max:2048'],
        ];
    }

    /**
     * Get custom messages for validator errors.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'no_hp.regex' => 'No HP hanya boleh berisi angka (8-20 digit).',
            'berkas.required' => 'Berkas wajib diunggah.',
            'berkas.mimes' => 'Berkas harus berformat jpg, jpeg, png, atau pdf.',
            'berkas.max' => 'Ukuran berkas maksimal 2 MB.',
        ];
    }

    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return true;
    }
}
```

*Gambar 8.30 DaftarRequest*

8. Buka *file* Modules/Pendaftaran/app/Http/Controllers/PendaftaranController.php, lalu ganti isinya dengan kode berikut:

```
<?php

namespace Modules\Pendaftaran\Http\Controllers;

use App\Http\Controllers\Controller;
```

```
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\Storage;
use Illuminate\View\View;
use Modules\Pendaftaran\Http\Requests\DaftarRequest;
use Modules\Pendaftaran\Models\Daftar;

class PendaftaranController extends Controller
{
    /**
     * Display the page. Data is loaded via AJAX.
     */
    public function index(): View
    {
        return view('pendaftaran::index');
    }

    /**
     * Return all registrations as JSON.
     */
    public function getData(): JsonResponse
    {
        return response()->json(Daftar::latest()->get());
    }

    /**
     * Store a newly created registration together with its uploaded
file.
     */
    public function store(DaftarRequest $request): JsonResponse
    {
        $data = $request->safe()->except('berkas');

        $file = $request->file('berkas');
        $data['berkas'] = $file->store('berkas_pendaftaran',
'public');
        $data['nama_berkas'] = $file->getClientOriginalName();

        Daftar::create($data);

        return response()->json(['success' => 'Data berhasil
disimpan.']);
    }

    /**
     * Return a single registration for the edit form.
     */
    public function edit(Daftar $daftar): JsonResponse
    {
        return response()->json($daftar);
    }

    /**
```

```
     * Update the specified registration. The old file is replaced
only when a new one is uploaded.
     */
    public function update(DaftarRequest $request, Daftar $daftar):
JsonResponse
    {
        $data = $request->safe()->except('berkas');

        if ($request->hasFile('berkas')) {
            if ($daftar->berkas) {
                Storage::disk('public')->delete($daftar->berkas);
            }

            $file = $request->file('berkas');
            $data['berkas'] = $file->store('berkas_pendaftaran',
'public');
            $data['nama_berkas'] = $file->getClientOriginalName();
        }

        $daftar->update($data);

        return response()->json(['success' => 'Data berhasil
diperbarui.']);
    }

    /**
     * Remove the specified registration and its uploaded file.
     */
    public function destroy(Daftar $daftar): JsonResponse
    {
        if ($daftar->berkas) {
            Storage::disk('public')->delete($daftar->berkas);
        }

        $daftar->delete();

        return response()->json(['success' => 'Data berhasil
dihapus.']);
    }
}
```

- index() → satu-satunya *method* yang mengembalikan *view*. Halaman dimuat sekali, lalu data diambil melalui AJAX.
- getData() → mengembalikan seluruh data pendaftar dalam bentuk JSON, diurutkan dari yang terbaru.
- store(), update(), dan destroy() → mengembalikan JSON berisi pesan success yang ditampilkan oleh JavaScript.
- edit() → mengembalikan satu data dalam JSON untuk mengisi *form* di dalam *modal*.
- $request->safe()->except('berkas') → mengambil data hasil validasi tanpa objek *file*, karena yang disimpan ke *database* hanya *path*-nya.

- $file->store('berkas_pendaftaran', 'public') → menyimpan berkas ke storage/app/public/berkas_pendaftaran dengan nama acak yang unik lalu mengembalikan *path*-nya, sedangkan getClientOriginalName() mengambil nama asli *file*.
- Storage::disk('public')->delete(...) → menghapus berkas lama ketika berkas diganti pada update() dan ketika data dihapus pada destroy(), agar tidak ada *file* sampah yang menumpuk di *server*.
- Daftar $daftar → *route* model *binding*; jika id tidak ditemukan, Laravel otomatis mengembalikan *error* 404.

Modules/Pendaftaran/app/Http/Controllers/PendaftaranController.php

```php
<?php

namespace Modules\Pendaftaran\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\Storage;
use Illuminate\View\View;
use Modules\Pendaftaran\Http\Requests\DaftarRequest;
use Modules\Pendaftaran\Models\Daftar;

class PendaftaranController extends Controller
{
    /**
     * Display the page. Data is loaded via AJAX.
     */
    public function index(): View
    {
        return view('pendaftaran::index');
    }

    /**
     * Return all registrations as JSON.
     */
    public function getData(): JsonResponse
    {
        return response()->json(Daftar::latest()->get());
    }

    /**
     * Store a newly created registration together with its uploaded file.
     */
    public function store(DaftarRequest $request): JsonResponse
    {
        $data = $request->safe()->except('berkas');

        $file = $request->file('berkas');
        $data['berkas'] = $file->store('berkas_pendaftaran', 'public');
        $data['nama_berkas'] = $file->getClientOriginalName();

        Daftar::create($data);

        return response()->json(['success' => 'Data berhasil disimpan.']);
    }

    /**
     * Return a single registration for the edit form.
     */
    public function edit(Daftar $daftar): JsonResponse
    {
        return response()->json($daftar);
    }

    /**
     * Update the specified registration. The old file is replaced only when a new one is uploaded.
     */
    public function update(DaftarRequest $request, Daftar $daftar): JsonResponse
    {
        $data = $request->safe()->except('berkas');

        if ($request->hasFile('berkas')) {
            if ($daftar->berkas) {
                Storage::disk('public')->delete($daftar->berkas);
            }

            $file = $request->file('berkas');
            $data['berkas'] = $file->store('berkas_pendaftaran', 'public');
            $data['nama_berkas'] = $file->getClientOriginalName();
        }

        $daftar->update($data);

        return response()->json(['success' => 'Data berhasil diperbarui.']);
    }

    /**
     * Remove the specified registration and its uploaded file.
     */
    public function destroy(Daftar $daftar): JsonResponse
    {
        if ($daftar->berkas) {
            Storage::disk('public')->delete($daftar->berkas);
        }

        $daftar->delete();

        return response()->json(['success' => 'Data berhasil dihapus.']);
    }
}
```

*Gambar 8.31 PendaftaranController dengan respons JSON dan upload berkas*

9. Buka *file* Modules/Pendaftaran/routes/web.php, lalu ganti isinya dengan kode berikut:

```
<?php

use Illuminate\Support\Facades\Route;
```

```
use Modules\Pendaftaran\Http\Controllers\PendaftaranController;

Route::middleware(['auth', 'role:admin,user'])->group(function () {
    Route::get('pendaftaran', [PendaftaranController::class,
'index'])->name('pendaftaran.index');
    Route::get('pendaftaran/data', [PendaftaranController::class,
'getData'])->name('pendaftaran.data');
    Route::post('pendaftaran', [PendaftaranController::class,
'store'])->name('pendaftaran.store');
    Route::get('pendaftaran/{daftar}/edit',
[PendaftaranController::class, 'edit'])->name('pendaftaran.edit');
    Route::put('pendaftaran/{daftar}', [PendaftaranController::class,
'update'])->name('pendaftaran.update');
    Route::delete('pendaftaran/{daftar}',
[PendaftaranController::class, 'destroy'])-
>name('pendaftaran.destroy');
});
```

- middleware(['auth', 'role:admin,user']) → modul dapat diakses oleh *admin* maupun *user* yang sudah *login*.
- *Route* pendaftaran/data ditulis **sebelum** *route* pendaftaran/{daftar} agar kata data tidak dianggap sebagai id.
- Edit dan update dipisah: GET pendaftaran/{daftar}/edit mengambil data, sedangkan PUT pendaftaran/{daftar} menyimpan perubahan.

Modules/Pendaftaran/routes/web.php

```
1  <?php
2
3  use Illuminate\Support\Facades\Route;
4  use Modules\Pendaftaran\Http\Controllers\PendaftaranController;
5
6  Route::middleware(['auth', 'role:admin,user'])->group(function () {
7      Route::get('pendaftaran', [PendaftaranController::class, 'index'])->name('pendaftaran.index');
8      Route::get('pendaftaran/data', [PendaftaranController::class, 'getData'])->name('pendaftaran.data');
9      Route::post('pendaftaran', [PendaftaranController::class, 'store'])->name('pendaftaran.store');
10     Route::get('pendaftaran/{daftar}/edit', [PendaftaranController::class, 'edit'])->name('pendaftaran.edit');
11     Route::put('pendaftaran/{daftar}', [PendaftaranController::class, 'update'])->name('pendaftaran.update');
12     Route::delete('pendaftaran/{daftar}', [PendaftaranController::class, 'destroy'])->name('pendaftaran.destroy');
13 });
```

*Gambar 8.32 Route modul Pendaftaran*

10. Buka *file* Modules/Pendaftaran/resources/views/index.blade.php, lalu ganti seluruh isinya dengan kode berikut:

```
@extends('layouts.app')

@section('title', 'Pendaftaran')

@section('content')
    <div class="d-flex flex-wrap justify-content-between align-items-
center gap-2 mb-4">
        <div>
            <h4 class="mb-1">Pendaftaran</h4>
            <p class="text-muted mb-0 small">Data tersimpan tanpa
memuat ulang halaman.</p>
```

```
        </div>
        <button type="button" class="btn btn-primary" id="addBtn">
            <i class="bi bi-plus-lg me-1"></i> Tambah Pendaftar
        </button>
    </div>

    <div id="alert-container"></div>

    <div class="card">
        <div class="card-header d-flex flex-wrap justify-content-
between align-items-center gap-2 py-3">
            <span class="fw-semibold">
                Daftar Pendaftar <span class="badge text-bg-light
border ms-1" id="total-count">0</span>
            </span>
            <div class="input-group" style="max-width: 300px">
                <span class="input-group-text bg-white"><i class="bi
bi-search"></i></span>
                <input type="search" id="search" class="form-control
border-start-0" placeholder="Cari pendaftar...">
            </div>
        </div>

        <div class="table-responsive">
            <table class="table table-hover" id="pendaftaran-table">
                <thead>
                    <tr>
                        <th width="5%">No</th>
                        <th>Nama</th>
                        <th>Asal Sekolah</th>
                        <th>Prodi Tujuan</th>
                        <th>Kontak</th>
                        <th>Berkas</th>
                        <th class="text-end">Aksi</th>
                    </tr>
                </thead>
                <tbody>
                    <tr>
                        <td colspan="7" class="text-center text-muted
py-5">
                            <div class="spinner-border spinner-
border-sm me-2" role="status"></div> Memuat data...
                        </td>
                    </tr>
                </tbody>
            </table>
        </div>
    </div>

    <!-- Modal form -->
    <div class="modal fade" id="pendaftaranModal" tabindex="-1" aria-
labelledby="pendaftaranModalLabel" aria-hidden="true">
        <div class="modal-dialog modal-dialog-centered">
```

```
            <div class="modal-content">
                <div class="modal-header">
                    <h5 class="modal-title"
id="pendaftaranModalLabel">Tambah Pendaftar</h5>
                    <button type="button" class="btn-close" data-bs-
dismiss="modal" aria-label="Tutup"></button>
                </div>
                <div class="modal-body">
                    <form id="pendaftaranForm">
                        @csrf
                        <input type="hidden" name="id" id="id">

                        @foreach ([
                            'nama' => ['label' => 'Nama Lengkap',
'type' => 'text', 'placeholder' => 'Nama pendaftar'],
                            'asal_sekolah' => ['label' => 'Asal
Sekolah', 'type' => 'text', 'placeholder' => 'Contoh: SMA Negeri 1
Bandung'],
                            'prodi_tujuan' => ['label' => 'Prodi
Tujuan', 'type' => 'text', 'placeholder' => 'Contoh: Informatika'],
                            'email' => ['label' => 'Email', 'type' =>
'email', 'placeholder' => 'nama@email.com'],
                            'no_hp' => ['label' => 'No HP', 'type' =>
'tel', 'placeholder' => 'Contoh: 081234567890'],
                        ] as $name => $field)
                            <div class="mb-3">
                                <label for="{{ $name }}" class="form-
label">{{ $field['label'] }} <span class="text-
danger">*</span></label>
                                <input type="{{ $field['type'] }}"
class="form-control" id="{{ $name }}" name="{{ $name }}"
placeholder="{{ $field['placeholder'] }}" required>
                                <div class="invalid-feedback" id="{{
$name }}-error"></div>
                            </div>
                        @endforeach

                        <div class="mb-3">
                            <label for="berkas" class="form-label">
                                Berkas Pendukung <span class="text-
danger" id="berkas-required">*</span>
                            </label>
                            <div class="small mb-2 d-none"
id="berkas-current">
                                Berkas saat ini: <a href="#"
target="_blank" rel="noopener" id="berkas-current-link"></a>
                            </div>
                            <input type="file" class="form-control"
id="berkas" name="berkas" accept=".jpg,.jpeg,.png,.pdf">
                            <div class="invalid-feedback" id="berkas-
error"></div>
                            <div class="form-text" id="berkas-
help">Format jpg, jpeg, png, atau pdf. Maksimal 2 MB.</div>
```

```
                        </div>
                    </form>
                </div>
                <div class="modal-footer">
                    <button type="button" class="btn btn-light" data-
bs-dismiss="modal">Batal</button>
                    <button type="button" class="btn btn-primary"
id="saveBtn">
                        <span class="spinner-border spinner-border-sm
me-1 d-none" role="status" aria-hidden="true"></span>
                        <span class="submit-text">Simpan</span>
                    </button>
                </div>
            </div>
        </div>
    </div>
@endsection

@push('scripts')
<script>
    $(document).ready(function () {
        const urls = {
            data: "{{ route('pendaftaran.data') }}",
            store: "{{ route('pendaftaran.store') }}",
            edit: "{{ route('pendaftaran.edit', ':id') }}",
            update: "{{ route('pendaftaran.update', ':id') }}",
            destroy: "{{ route('pendaftaran.destroy', ':id') }}",
        };

        let records = [];

        // Set CSRF token untuk semua request AJAX
        $.ajaxSetup({
            headers: {
                'X-CSRF-TOKEN': $('meta[name="csrf-
token"]').attr('content'),
                'Accept': 'application/json'
            }
        });

        // Escape karakter HTML (termasuk tanda kutip untuk atribut)
agar data user tidak dieksekusi sebagai script (XSS)
        function escapeHtml(value) {
            return String(value ?? '')
                .replace(/&/g, '&amp;')
                .replace(/</g, '&lt;')
                .replace(/>/g, '&gt;')
                .replace(/"/g, '&quot;')
                .replace(/'/g, '&#039;');
        }

        // Link untuk membuka berkas di tab baru
        function fileLink(item) {
```

```
            if (! item.berkas_url) {
                return '<span class="text-muted">-</span>';
            }

            const isPdf = (item.nama_berkas ||
item.berkas).toLowerCase().endsWith('.pdf');
            const icon = isPdf ? 'bi-file-earmark-pdf text-danger' :
'bi-file-earmark-image text-primary';

            return `<a href="${escapeHtml(item.berkas_url)}"
target="_blank" rel="noopener" class="small text-decoration-none">
                        <i class="bi ${icon} me-
1"></i>${escapeHtml(item.nama_berkas || 'Lihat berkas')}
                    </a>`;
        }

        // Mode input berkas: wajib saat tambah, opsional saat edit
        function setFileMode(item = null) {
            $('#berkas').val('');
            $('#berkas-required').toggleClass('d-none', item !==
null);
            $('#berkas-current').toggleClass('d-none', !
item?.berkas_url);
            $('#berkas-help').text(item
                ? 'Kosongkan jika tidak ingin mengganti berkas.
Format jpg, jpeg, png, atau pdf. Maksimal 2 MB.'
                : 'Format jpg, jpeg, png, atau pdf. Maksimal 2 MB.');

            if (item?.berkas_url) {
                $('#berkas-current-link').attr('href',
item.berkas_url).text(item.nama_berkas || 'Lihat berkas');
            }
        }

        function showAlert(message, type = 'success') {
            const icon = type === 'success' ? 'bi-check-circle' :
'bi-exclamation-triangle';
            const alert = $(`
                <div class="alert alert-${type} alert-dismissible
fade show d-flex align-items-center gap-2" role="alert">
                    <i class="bi ${icon}"></i>
                    <span>${escapeHtml(message)}</span>
                    <button type="button" class="btn-close" data-bs-
dismiss="alert" aria-label="Tutup"></button>
                </div>
            `);

            $('#alert-container').html(alert);
            setTimeout(() =>
bootstrap.Alert.getOrCreateInstance(alert[0]).close(), 4000);
        }

        function clearErrors() {
```

```
            $('#pendaftaranForm .invalid-feedback').text('');
            $('#pendaftaranForm .is-invalid').removeClass('is-
invalid');
        }

        function emptyRow(icon, text) {
            return `
                <tr>
                    <td colspan="7">
                        <div class="empty-state">
                            <i class="bi ${icon}"></i>
                            <p class="mt-2 mb-0">${text}</p>
                        </div>
                    </td>
                </tr>`;
        }

        // Tampilkan data ke tabel sesuai kata kunci pencarian
        function renderTable() {
            const keyword = $('#search').val().trim().toLowerCase();
            const rows = records.filter((item) =>
                [item.nama, item.asal_sekolah, item.prodi_tujuan,
item.email, item.no_hp]
                    .some((value) => (value ||
'').toLowerCase().includes(keyword))
            );

            $('#total-count').text(records.length);

            if (records.length === 0) {
                $('#pendaftaran-table tbody').html(emptyRow('bi-
inbox', 'Belum ada pendaftar. Klik <strong>Tambah Pendaftar</strong>
untuk memulai.'));
                return;
            }

            if (rows.length === 0) {
                $('#pendaftaran-table tbody').html(emptyRow('bi-
search', 'Tidak ada pendaftar yang cocok dengan pencarian.'));
                return;
            }

            const html = rows.map((item, index) => `
                <tr>
                    <td class="text-muted">${index + 1}</td>
                    <td class="fw-
medium">${escapeHtml(item.nama)}</td>
                    <td>${escapeHtml(item.asal_sekolah)}</td>
                    <td><span class="badge text-bg-light
border">${escapeHtml(item.prodi_tujuan)}</span></td>
                    <td class="small">
                        <div>${escapeHtml(item.email)}</div>
```

```
                        <div class="text-
muted">${escapeHtml(item.no_hp)}</div>
                    </td>
                    <td>${fileLink(item)}</td>
                    <td class="text-end text-nowrap">
                        <button data-id="${item.id}" class="edit btn
btn-sm btn-outline-primary">
                            <i class="bi bi-pencil"></i> Edit
                        </button>
                        <button data-id="${item.id}" data-
nama="${escapeHtml(item.nama)}" class="delete btn btn-sm btn-outline-
danger">
                            <i class="bi bi-trash"></i> Hapus
                        </button>
                    </td>
                </tr>
            `).join('');

            $('#pendaftaran-table tbody').html(html);
        }

        // Muat data dari server
        function loadData() {
            $.ajax({
                url: urls.data,
                type: 'GET',
                dataType: 'json',
                success: function (data) {
                    records = data;
                    renderTable();
                },
                error: function (xhr) {
                    console.error('Error:', xhr.responseText);
                    $('#pendaftaran-table tbody').html(emptyRow('bi-
wifi-off', 'Gagal memuat data. Silakan refresh halaman.'));
                }
            });
        }

        loadData();
        $('#search').on('input', renderTable);

        const pendaftaranModal = new
bootstrap.Modal(document.getElementById('pendaftaranModal'));

        $('#pendaftaranModal').on('shown.bs.modal', function () {
            $('#nama').trigger('focus');
        });

        $('#pendaftaranModal').on('hidden.bs.modal', function () {
            $('#pendaftaranForm').trigger('reset');
            $('#id').val('');
            clearErrors();
```

```
            setFileMode();
        });

        // Tambah data
        $('#addBtn').click(function () {
            $('#pendaftaranModalLabel').text('Tambah Pendaftar');
            setFileMode();
            pendaftaranModal.show();
        });

        // Tekan Enter di form = simpan
        $('#pendaftaranForm').on('submit', function (e) {
            e.preventDefault();
            $('#saveBtn').trigger('click');
        });

        // Simpan data (create / update)
        $('#saveBtn').click(function () {
            const btn = $(this);
            const id = $('#id').val();

            btn.prop('disabled', true);
            btn.find('.submit-text').text('Menyimpan...');
            btn.find('.spinner-border').removeClass('d-none');
            clearErrors();

            // FormData dipakai (bukan serialize) agar file ikut
terkirim
            const formData = new
FormData(document.getElementById('pendaftaranForm'));
            if (id) {
                formData.append('_method', 'PUT');
            }

            $.ajax({
                url: id ? urls.update.replace(':id', id) :
urls.store,
                type: 'POST',
                data: formData,
                processData: false,
                contentType: false,
                dataType: 'json',
                success: function (response) {
                    pendaftaranModal.hide();
                    loadData();
                    showAlert(response.success);
                },
                error: function (xhr) {
                    if (xhr.status === 422) {
                        $.each(xhr.responseJSON.errors, function
(key, value) {
                            $(`#${key}`).addClass('is-invalid');
                            $(`#${key}-error`).text(value[0]);
```

```
                        });
                        $('#pendaftaranForm .is-
invalid:first').focus();
                    } else {
                        console.error('Error:', xhr.responseText);
                        pendaftaranModal.hide();
                        showAlert('Terjadi kesalahan. Silakan coba
lagi.', 'danger');
                    }
                },
                complete: function () {
                    btn.prop('disabled', false);
                    btn.find('.submit-text').text('Simpan');
                    btn.find('.spinner-border').addClass('d-none');
                }
            });
        });

        // Edit data
        $(document).on('click', '.edit', function () {
            const id = $(this).data('id');

            $.get(urls.edit.replace(':id', id), function (data) {
                $('#pendaftaranModalLabel').text('Edit Pendaftar');
                $('#id').val(data.id);
                $('#nama').val(data.nama);
                $('#asal_sekolah').val(data.asal_sekolah);
                $('#prodi_tujuan').val(data.prodi_tujuan);
                $('#email').val(data.email);
                $('#no_hp').val(data.no_hp);
                setFileMode(data);
                pendaftaranModal.show();
            }).fail(function () {
                showAlert('Gagal memuat data untuk diedit.',
'danger');
            });
        });

        // Hapus data
        $(document).on('click', '.delete', function () {
            const id = $(this).data('id');
            const nama = $(this).data('nama');

            confirmDelete(`Data pendaftar "${nama}" akan dihapus dan
tidak bisa dikembalikan.`).then(function (confirmed) {
                if (! confirmed) {
                    return;
                }

                $.ajax({
                    type: 'DELETE',
                    url: urls.destroy.replace(':id', id),
                    dataType: 'json',
```

```
                    success: function (response) {
                        loadData();
                        showAlert(response.success);
                    },
                    error: function () {
                        showAlert('Gagal menghapus data. Silakan coba
lagi.', 'danger');
                    }
                });
            });
        });
    });
</script>
@endpush
```

Kode di atas cukup panjang, sehingga penjelasannya dibagi menjadi beberapa bagian berikut.

Modules/Pendaftaran/resources/views/index.blade.php

```blade
<!-- Modal form -->
<div class="modal fade" id="pendaftaranModal" tabindex="-1" aria-labelledby="pendaftaranModalLabel" aria-hidden="true">
    <div class="modal-dialog modal-dialog-centered">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title" id="pendaftaranModalLabel">Tambah Pendaftar</h5>
                <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Tutup"></button>
            </div>
            <div class="modal-body">
                <form id="pendaftaranForm">
                    @csrf
                    <input type="hidden" name="id" id="id">

                    @foreach ([
                        'nama' => ['label' => 'Nama Lengkap', 'type' => 'text', 'placeholder' => 'Nama pendaftar'],
                        'asal_sekolah' => ['label' => 'Asal Sekolah', 'type' => 'text', 'placeholder' => 'Contoh: SMA Negeri 1 Bandung'],
                        'prodi_tujuan' => ['label' => 'Prodi Tujuan', 'type' => 'text', 'placeholder' => 'Contoh: Informatika'],
                        'email' => ['label' => 'Email', 'type' => 'email', 'placeholder' => 'nama@email.com'],
                        'no_hp' => ['label' => 'No HP', 'type' => 'tel', 'placeholder' => 'Contoh: 081234567890'],
                    ] as $name => $field)
                        <div class="mb-3">
                            <label for="{{ $name }}" class="form-label">{{ $field['label'] }} <span class="text-danger">*</span></label>
                            <input type="{{ $field['type'] }}" class="form-control" id="{{ $name }}" name="{{ $name }}" placeholder="{{ $field['placeholder'] }}" required>
                            <div class="invalid-feedback" id="{{ $name }}-error"></div>
                        </div>
                    @endforeach

                    <div class="mb-3">
                        <label for="berkas" class="form-label">
                            Berkas Pendukung <span class="text-danger" id="berkas-required">*</span>
                        </label>
                        <div class="small mb-2 d-none" id="berkas-current">
                            Berkas saat ini: <a href="#" target="_blank" rel="noopener" id="berkas-current-link"></a>
                        </div>
                        <input type="file" class="form-control" id="berkas" name="berkas" accept=".jpg,.jpeg,.png,.pdf">
                        <div class="invalid-feedback" id="berkas-error"></div>
                        <div class="form-text" id="berkas-help">Format jpg, jpeg, png, atau pdf. Maksimal 2 MB.</div>
                    </div>
                </form>
            </div>
            <div class="modal-footer">
                <button type="button" class="btn btn-light" data-bs-dismiss="modal">Batal</button>
                <button type="button" class="btn btn-primary" id="saveBtn">
                    <span class="spinner-border spinner-border-sm me-1 d-none" role="status" aria-hidden="true"></span>
                    <span class="submit-text">Simpan</span>
                </button>
            </div>
        </div>
    </div>
</div>
@endsection
```

*Gambar 8.33 Modal form tambah/edit pendaftar*

*Form* tambah dan edit menggunakan satu *modal* yang sama. *Input* hidden id bernilai kosong saat menambah dan berisi id data saat edit. Elemen  disiapkan untuk menampilkan pesan *error* dari *server*. Input berkas memakai accept=".jpg,.jpeg,.png,.pdf" agar jendela pemilihan *file* hanya menampilkan format yang diizinkan, sedangkan elemen #berkas-current menampilkan tautan berkas saat ini ketika data diedit.

Modules/Pendaftaran/resources/views/index.blade.php

```blade
@push('scripts')
<script>
    $(document).ready(function () {
        const urls = {
            data: "{{ route('pendaftaran.data') }}",
            store: "{{ route('pendaftaran.store') }}",
            edit: "{{ route('pendaftaran.edit', ':id') }}",
            update: "{{ route('pendaftaran.update', ':id') }}",
            destroy: "{{ route('pendaftaran.destroy', ':id') }}",
        };

        let records = [];

        // Set CSRF token untuk semua request AJAX
        $.ajaxSetup({
            headers: {
                'X-CSRF-TOKEN': $('meta[name="csrf-token"]').attr('content'),
                'Accept': 'application/json'
            }
        });

        // Escape karakter HTML (termasuk tanda kutip untuk atribut) agar data user tidak dieksekusi sebagai script (XSS)
        function escapeHtml(value) {
            return String(value ?? '')
                .replace(/&/g, '&amp;')
                .replace(/</g, '&lt;')
                .replace(/>/g, '&gt;')
                .replace(/"/g, '&quot;')
                .replace(/'/g, '&#039;');
        }

        // Link untuk membuka berkas di tab baru
        function fileLink(item) {
            if (! item.berkas_url) {
                return '<span class="text-muted">-</span>';
            }

            const isPdf = (item.nama_berkas || item.berkas).toLowerCase().endsWith('.pdf');
            const icon = isPdf ? 'bi-file-earmark-pdf text-danger' : 'bi-file-earmark-image text-primary';

            return `<a href="${escapeHtml(item.berkas_url)}" target="_blank" rel="noopener" class="small text-decoration-none">
                        <i class="bi ${icon} me-1"></i>${escapeHtml(item.nama_berkas || 'Lihat berkas')}
                    </a>`;
        }

        // Mode input berkas: wajib saat tambah, opsional saat edit
        function setFileMode(item = null) {
            $('#berkas').val('');
            $('#berkas-required').toggleClass('d-none', item !== null);
            $('#berkas-current').toggleClass('d-none', ! item?.berkas_url);
            $('#berkas-help').text(item
                ? 'Kosongkan jika tidak ingin mengganti berkas. Format jpg, jpeg, png, atau pdf. Maksimal 2 MB.'
                : 'Format jpg, jpeg, png, atau pdf. Maksimal 2 MB.');

            if (item?.berkas_url) {
                $('#berkas-current-link').attr('href', item.berkas_url).text(item.nama_berkas || 'Lihat berkas');
            }
        }
```

*Gambar 8.34 Pengaturan AJAX, escapeHtml(), fileLink(), dan setFileMode()*

Modules/Pendaftaran/resources/views/index.blade.php

```
        function showAlert(message, type = 'success') {
            const icon = type === 'success' ? 'bi-check-circle' : 'bi-exclamation-triangle';
            const alert = $(`
                <div class="alert alert-${type} alert-dismissible fade show d-flex align-items-center gap-2" role="alert">
                    <i class="bi ${icon}"></i>
                    <span>${escapeHtml(message)}</span>
                    <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Tutup"></button>
                </div>
            `);

            $('#alert-container').html(alert);
            setTimeout(() => bootstrap.Alert.getOrCreateInstance(alert[0]).close(), 4000);
        }

        function clearErrors() {
            $('#pendaftaranForm .invalid-feedback').text('');
            $('#pendaftaranForm .is-invalid').removeClass('is-invalid');
        }

        function emptyRow(icon, text) {
            return `
                <tr>
                    <td colspan="7">
                        <div class="empty-state">
                            <i class="bi ${icon}"></i>
                            <p class="mt-2 mb-0">${text}</p>
                        </div>
                    </td>
                </tr>`;
        }

        // Tampilkan data ke tabel sesuai kata kunci pencarian
        function renderTable() {
            const keyword = $('#search').val().trim().toLowerCase();
            const rows = records.filter((item) =>
                [item.nama, item.asal_sekolah, item.prodi_tujuan, item.email, item.no_hp]
                    .some((value) => (value || '').toLowerCase().includes(keyword))
            );

            $('#total-count').text(records.length);

            if (records.length === 0) {
                $('#pendaftaran-table tbody').html(emptyRow('bi-inbox', 'Belum ada pendaftar. Klik <strong>Tambah Pendaftar</strong> untuk memulai.'));
                return;
            }

            if (rows.length === 0) {
                $('#pendaftaran-table tbody').html(emptyRow('bi-search', 'Tidak ada pendaftar yang cocok dengan pencarian.'));
                return;
            }

            const html = rows.map((item, index) => `
                <tr>
                    <td class="text-muted">${index + 1}</td>
                    <td class="fw-medium">${escapeHtml(item.nama)}</td>
                    <td>${escapeHtml(item.asal_sekolah)}</td>
                    <td><span class="badge text-bg-light border">${escapeHtml(item.prodi_tujuan)}</span></td>
                    <td class="small">
                        <div>${escapeHtml(item.email)}</div>
                        <div class="text-muted">${escapeHtml(item.no_hp)}</div>
                    </td>
                    <td>${fileLink(item)}</td>
                    <td class="text-end text-nowrap">
                        <button data-id="${item.id}" class="edit btn btn-sm btn-outline-primary">
                            <i class="bi bi-pencil"></i> Edit
                        </button>
                        <button data-id="${item.id}" data-nama="${escapeHtml(item.nama)}" class="delete btn btn-sm btn-outline-danger">
                            <i class="bi bi-trash"></i> Hapus
                        </button>
                    </td>
                </tr>
            `).join('');

            $('#pendaftaran-table tbody').html(html);
        }

        // Muat data dari server
        function loadData() {
            $.ajax({
                url: urls.data,
                type: 'GET',
                dataType: 'json',
                success: function (data) {
                    records = data;
                    renderTable();
                },
                error: function (xhr) {
                    console.error('Error:', xhr.responseText);
                    $('#pendaftaran-table tbody').html(emptyRow('bi-wifi-off', 'Gagal memuat data. Silakan refresh halaman.'));
                }
            });
        }

        loadData();
        $('#search').on('input', renderTable);
```

*Gambar 8.35 Render tabel dan loadData()*

- const urls = {...} → URL dibuat dengan *helper* route() di Blade. *Placeholder* :id diganti dengan id data menggunakan replace(':id', id).
- $.ajaxSetup({ headers: {...} }) → setiap *request* AJAX otomatis membawa *header* X-CSRF-TOKEN dan Accept: application/json.

- escapeHtml() → mengubah karakter <, >, &, dan tanda kutip menjadi teks biasa sebelum dimasukkan ke tabel atau atribut HTML, sehingga data tidak dapat menyisipkan *script*.
- fileLink() → membuat tautan ke berkas_url dengan target="_blank" sehingga berkas terbuka di *tab* baru. Ikon PDF atau gambar dipilih berdasarkan ekstensi nama berkas.
- setFileMode() → mengatur input berkas: wajib (bertanda bintang) saat menambah data, dan opsional saat edit sambil menampilkan tautan berkas saat ini.
- renderTable() → menyaring data berdasarkan kata kunci pencarian lalu membangun baris tabel. Pencarian dilakukan di *browser* tanpa *request* baru ke *server*.
- loadData() → mengambil data dari *route* pendaftaran.data dengan *method* GET, lalu menyimpannya ke variabel records dan menampilkan tabel.

Modules/Pendaftaran/resources/views/index.blade.php

```
        // Simpan data (create / update)
        $('#saveBtn').click(function () {
            const btn = $(this);
            const id = $('#id').val();

            btn.prop('disabled', true);
            btn.find('.submit-text').text('Menyimpan...');
            btn.find('.spinner-border').removeClass('d-none');
            clearErrors();

            // FormData dipakai (bukan serialize) agar file ikut terkirim
            const formData = new FormData(document.getElementById('pendaftaranForm'));
            if (id) {
                formData.append('_method', 'PUT');
            }

            $.ajax({
                url: id ? urls.update.replace(':id', id) : urls.store,
                type: 'POST',
                data: formData,
                processData: false,
                contentType: false,
                dataType: 'json',
                success: function (response) {
                    pendaftaranModal.hide();
                    loadData();
                    showAlert(response.success);
                },
                error: function (xhr) {
                    if (xhr.status === 422) {
                        $.each(xhr.responseJSON.errors, function (key, value) {
                            $(`#${key}`).addClass('is-invalid');
                            $(`#${key}-error`).text(value[0]);
                        });
                        $('#pendaftaranForm .is-invalid:first').focus();
                    } else {
                        console.error('Error:', xhr.responseText);
                        pendaftaranModal.hide();
                        showAlert('Terjadi kesalahan. Silakan coba lagi.', 'danger');
                    }
                },
                complete: function () {
                    btn.prop('disabled', false);
                    btn.find('.submit-text').text('Simpan');
                    btn.find('.spinner-border').addClass('d-none');
                }
            });
        });
```

*Gambar 8.36 Menyimpan data (tambah dan edit) dengan AJAX*

- new FormData(document.getElementById('pendaftaranForm')) → mengambil seluruh isi *form* termasuk *file* yang dipilih dan *token* _token. FormData wajib digunakan karena serialize() hanya mengirim teks, sehingga *file* tidak ikut terkirim.
- formData.append('_method', 'PUT') → jika input id terisi, *request* dikirim ke *route* update dengan *method spoofing* PUT. *Request* tetap dikirim sebagai POST karena PHP hanya membaca *file upload* dari *request* POST.

- processData: false dan contentType: false → mencegah jQuery mengubah FormData menjadi teks, sehingga *browser* mengirimnya sebagai *multipart/form-data* beserta *boundary*-nya.
- if (xhr.status === 422) → *error* validasi dari *server* ditampilkan di bawah setiap *input* dengan menambahkan kelas is-invalid.
- complete → tombol Simpan diaktifkan kembali dan *spinner* disembunyikan, baik *request* berhasil maupun gagal.

Modules/Pendaftaran/resources/views/index.blade.php

```
335         // Edit data
336         $(document).on('click', '.edit', function () {
337             const id = $(this).data('id');
338
339             $.get(urls.edit.replace(':id', id), function (data) {
340                 $('#pendaftaranModalLabel').text('Edit Pendaftar');
341                 $('#id').val(data.id);
342                 $('#nama').val(data.nama);
343                 $('#asal_sekolah').val(data.asal_sekolah);
344                 $('#prodi_tujuan').val(data.prodi_tujuan);
345                 $('#email').val(data.email);
346                 $('#no_hp').val(data.no_hp);
347                 setFileMode(data);
348                 pendaftaranModal.show();
349             }).fail(function () {
350                 showAlert('Gagal memuat data untuk diedit.', 'danger');
351             });
352         });
353
354         // Hapus data
355         $(document).on('click', '.delete', function () {
356             const id = $(this).data('id');
357             const nama = $(this).data('nama');
358
359             confirmDelete(`Data pendaftar "${nama}" akan dihapus dan tidak bisa dikembalikan.`).then(function (confirmed) {
360                 if (! confirmed) {
361                     return;
362                 }
363
364                 $.ajax({
365                     type: 'DELETE',
366                     url: urls.destroy.replace(':id', id),
367                     dataType: 'json',
368                     success: function (response) {
369                         loadData();
370                         showAlert(response.success);
371                     },
372                     error: function () {
373                         showAlert('Gagal menghapus data. Silakan coba lagi.', 'danger');
374                     }
375                 });
376             });
377         });
378     });
379 </script>
380 @endpush
```

*Gambar 8.37 Mengambil data edit dan menghapus data dengan AJAX*

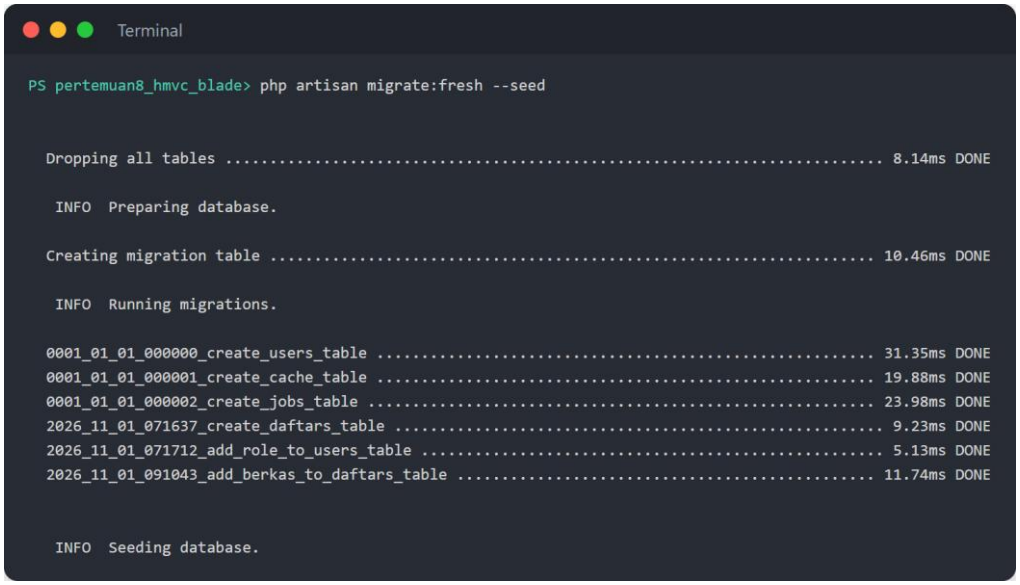
- $(document).on('click', '.edit', ...) → *event delegation*; tombol Edit dibuat ulang setiap kali tabel di-*render*, sehingga *event* dipasang pada document, bukan langsung pada tombol.
- $.get(urls.edit...) → mengambil satu data lalu mengisi *input* pada *modal*. Fungsi setFileMode(data) menampilkan tautan berkas saat ini dan membuat input berkas menjadi opsional.
- confirmDelete(...).then(...) → *request* DELETE baru dikirim setelah pengguna menekan **Ya, hapus** pada *modal* konfirmasi.

**Tahap F: Menjalankan dan Menguji Aplikasi**

1. Kosongkan *database*, jalankan seluruh *migration* (termasuk *migration* modul), lalu isi akun awal dari seeder:

```
php artisan migrate:fresh --seed
```

- migrate:fresh menghapus **seluruh** tabel lalu membuatnya ulang, sehingga seluruh data lama hilang. Gunakan php artisan migrate --seed jika *database* masih kosong dan tidak ingin menghapus apa pun.

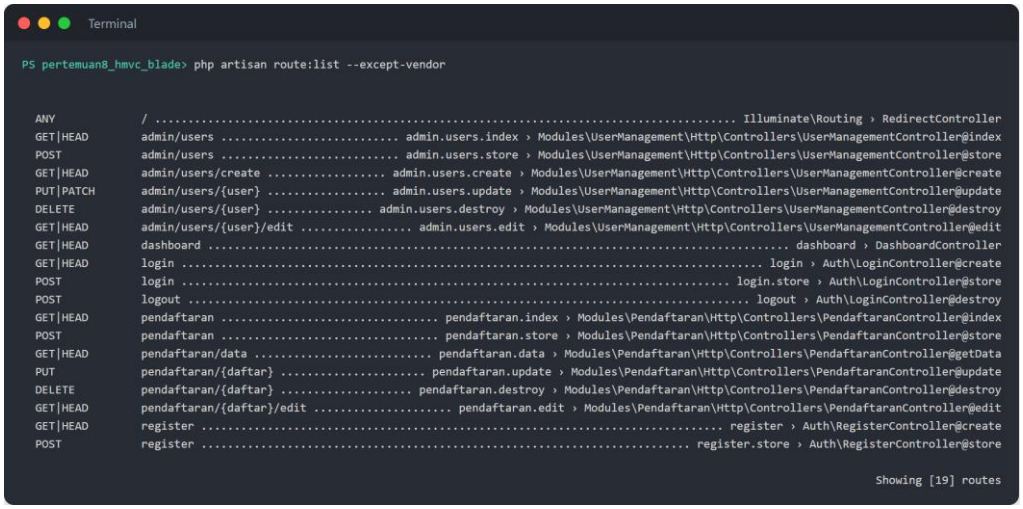



*Gambar 8.38 Hasil php artisan migrate:fresh --seed*

*Migration create_daftars_table dan add_berkas_to_daftars_table milik* modul Pendaftaran ikut dijalankan bersama *migration* aplikasi utama.

2. Periksa seluruh *route* yang terdaftar:

```
php artisan route:list --except-vendor
```

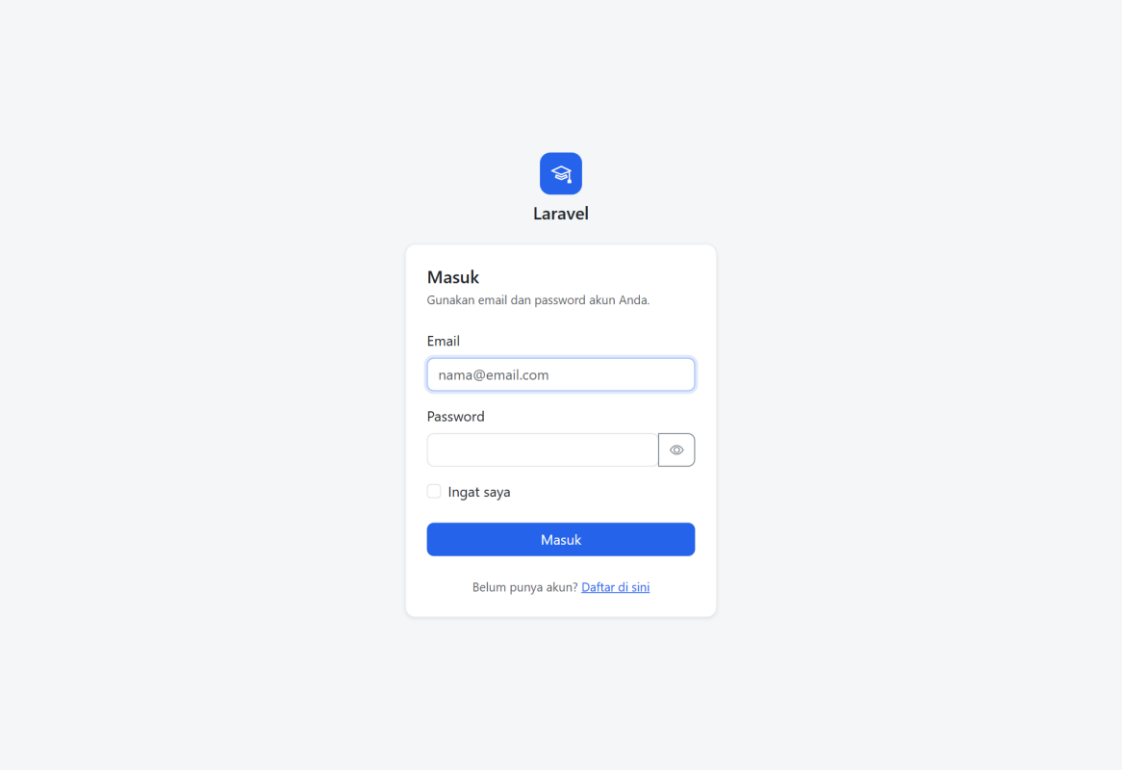



*Gambar 8.39 Hasil php artisan route:list --except-vendor*

Terdapat 19 route: *route authentication* dan *dashboard* dari routes/web.php, enam *route* admin/users dari modul UserManagement, dan enam *route* pendaftaran dari modul Pendaftaran.

3. Jalankan *server* dengan perintah php artisan serve, lalu buka http://127.0.0.1:8000. Uji fitur *authentication* dengan langkah berikut:

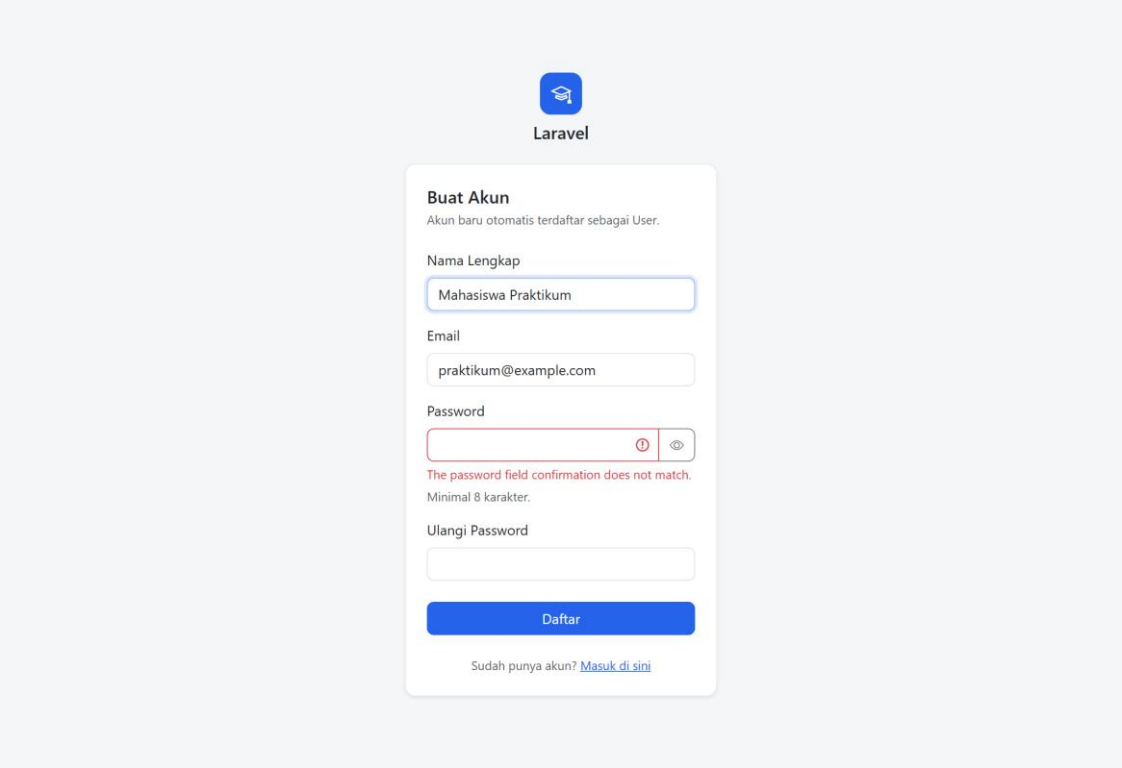
a. Buka http://127.0.0.1:8000/pendaftaran sebelum *login* → Anda otomatis diarahkan ke halaman *login* oleh *middleware* auth.




*Gambar 8.40 Halaman /pendaftaran diakses sebelum login*

*Middleware* auth mengarahkan tamu ke halaman *login*. Setelah *login* berhasil, redirect()->intended() mengembalikan pengguna ke halaman Pendaftaran yang tadi ingin dibuka.

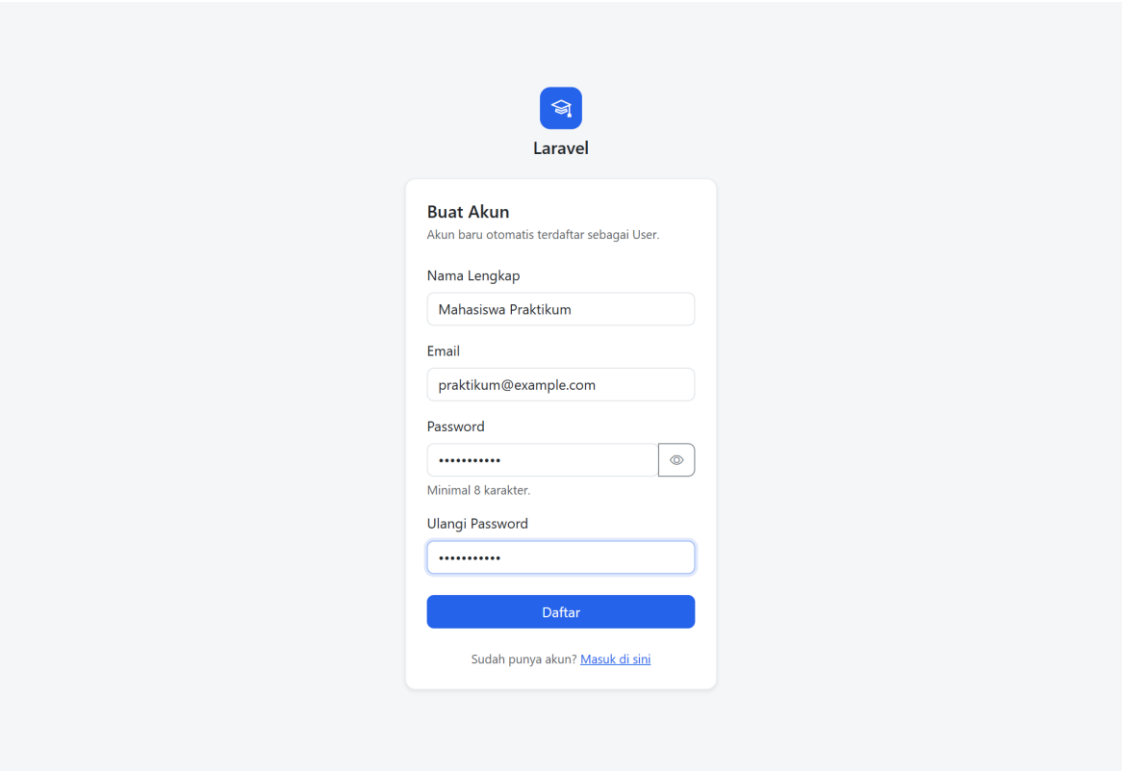
b. Klik **Daftar di sini**, lalu kirim *form register* dengan *password* kurang dari 8 karakter dan konfirmasi *password* yang berbeda → pesan *error* validasi muncul di bawah *input*.



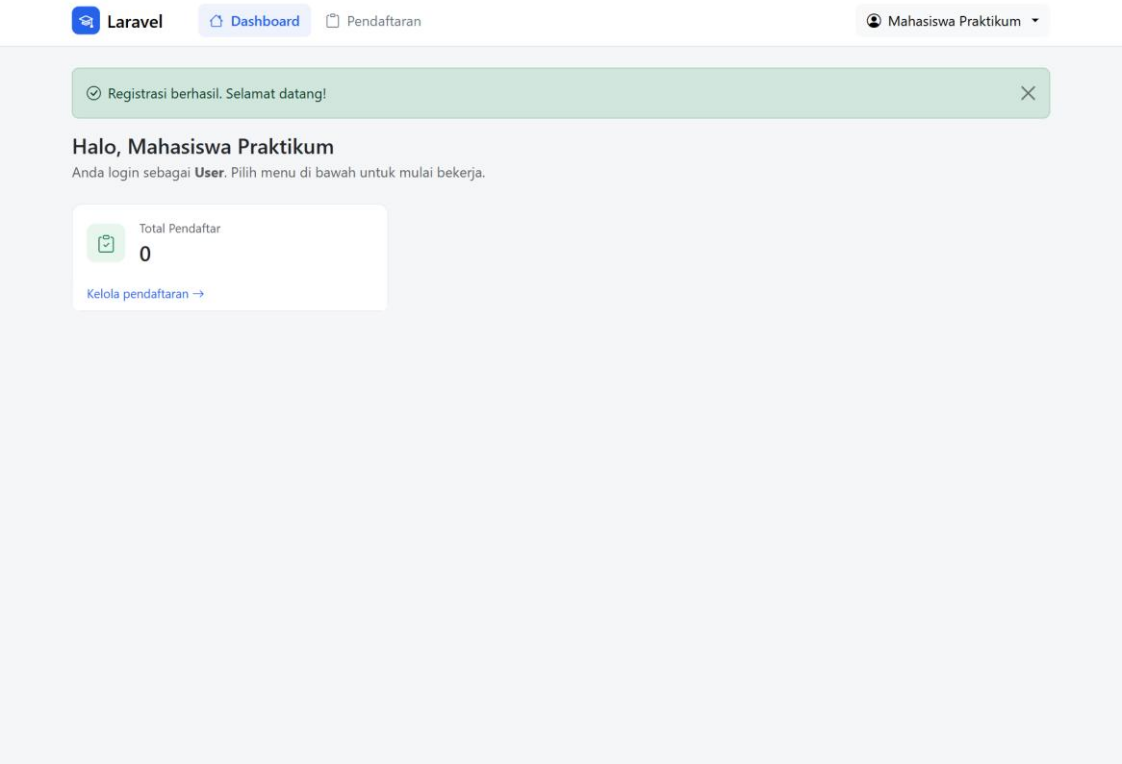

*Gambar 8.41 Pesan error validasi pada form register*

Aturan confirmed dan Password::defaults() menolak *password*; nama dan *email* tetap terisi karena old().

c. Isi kembali *password* dan konfirmasi *password* dengan benar, lalu klik **Daftar** → akun dibuat dan pengguna langsung masuk ke *dashboard* sebagai ***User***.



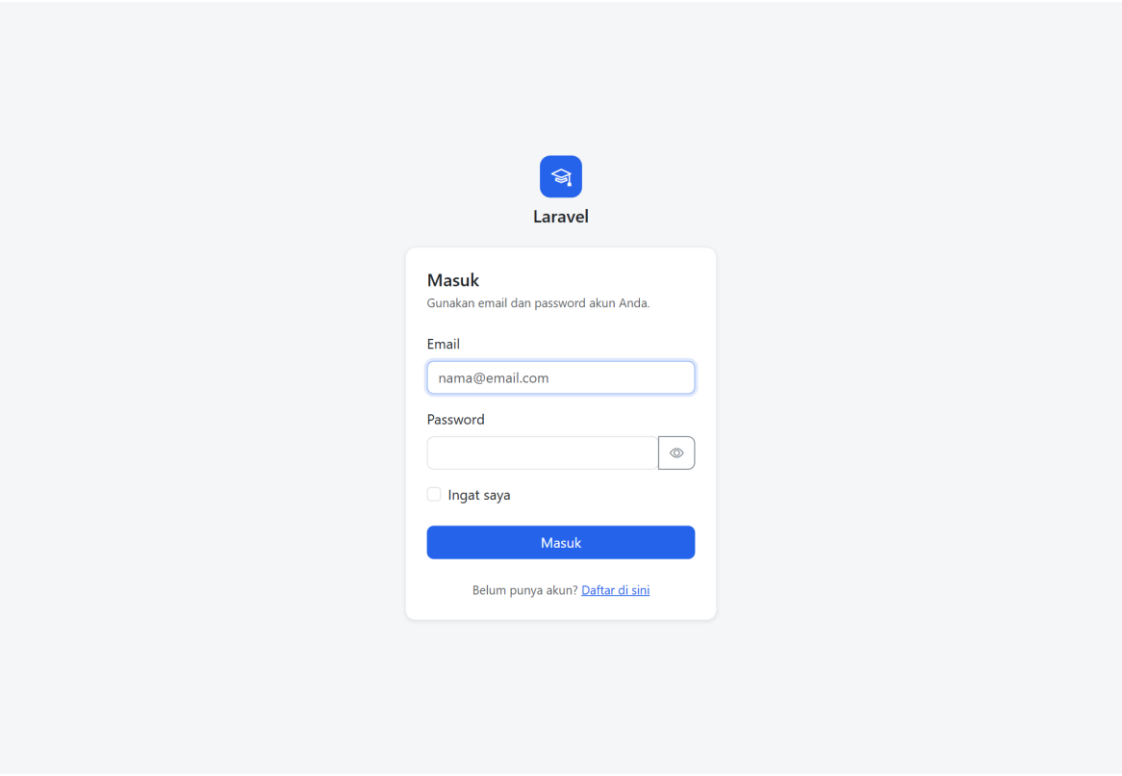

*Gambar 8.42 Form register yang sudah diisi dengan benar*




*Gambar 8.43 Dashboard setelah register berhasil*

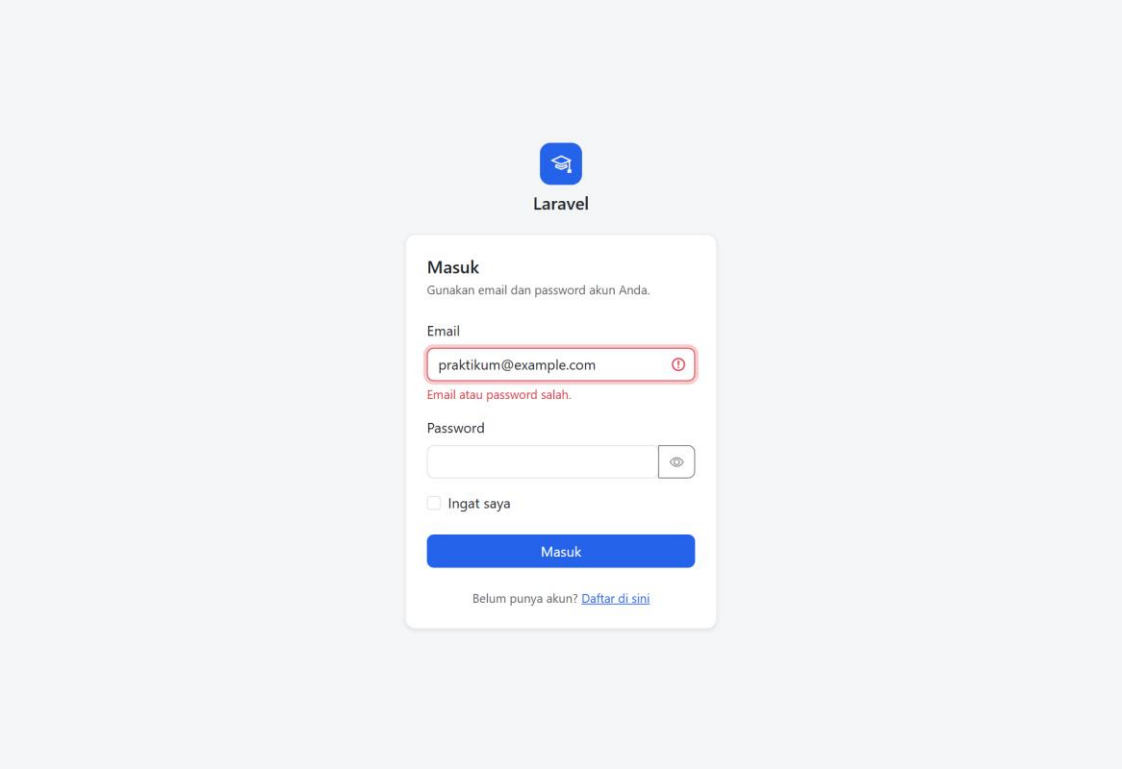
Akun hasil *register* selalu berperan *User*, sehingga *navbar* hanya menampilkan menu *Dashboard* dan Pendaftaran tanpa menu Kelola *User*.

d. Klik nama pengguna di kanan atas, lalu klik ***Logout*** → *session* diakhiri dan Anda kembali ke halaman *login*.




*Gambar 8.44 Halaman login setelah logout*

e. *Login* menggunakan *password* yang salah → muncul pesan ***Email* atau *password* salah.** Jika gagal lebih dari 5 kali, *login* dikunci sementara.




*Gambar 8.45 Pesan error saat login dengan password salah*

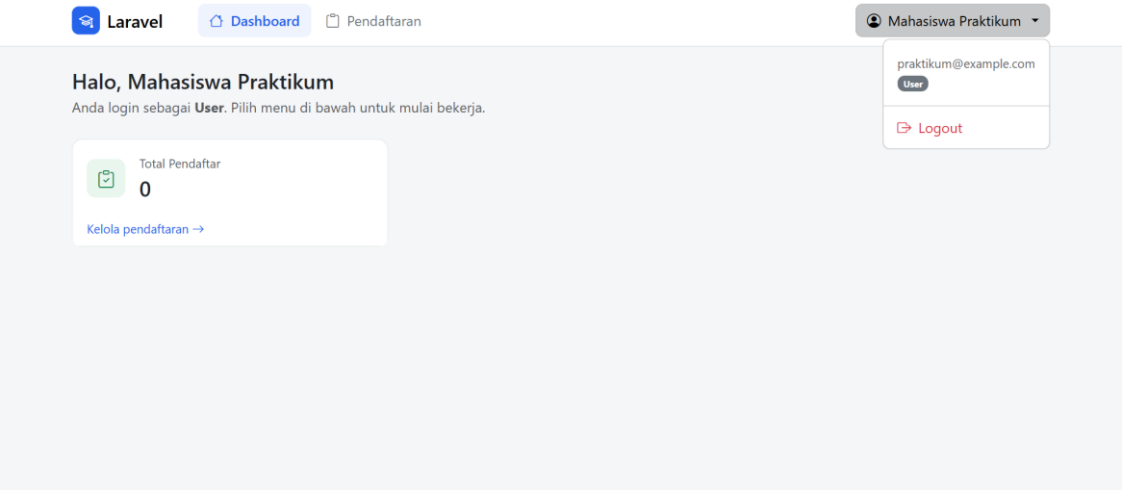
4. Uji fitur *authorization* (*middleware role*):

a. *Login* sebagai *user* (praktikum@example.com), lalu buka http://127.0.0.1:8000/admin/users secara langsung melalui address bar → muncul *error* **403**.




*Gambar 8.46 Error 403 saat user membuka halaman admin*

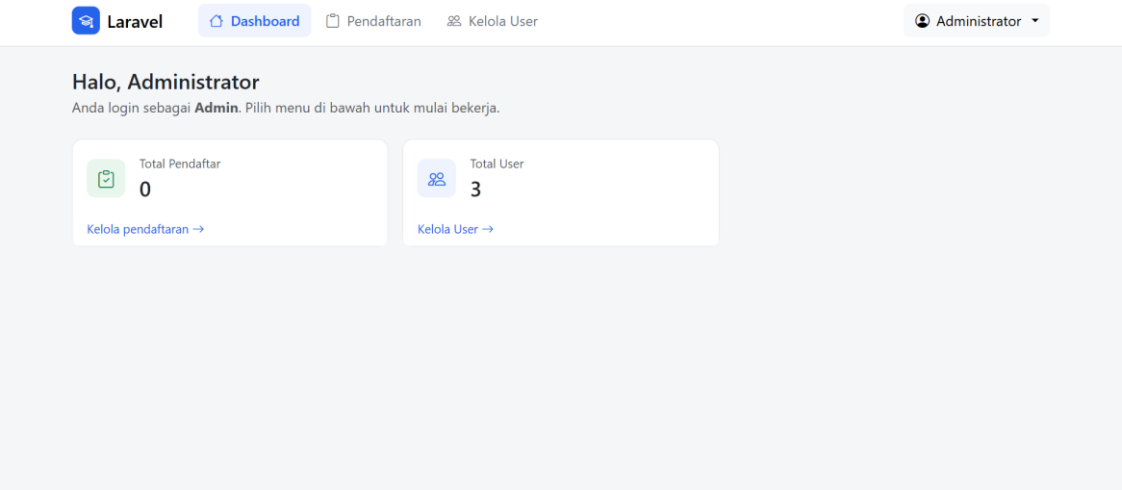
Walaupun menu Kelola *User* disembunyikan, *user* tetap tidak dapat masuk dengan mengetik URL karena *route* dilindungi *middleware* role:admin.




*Gambar 8.47 Dropdown akun user*

*Dropdown* menampilkan *email* dan *badge role*. Menu Kelola *User* tidak muncul untuk *role User*.

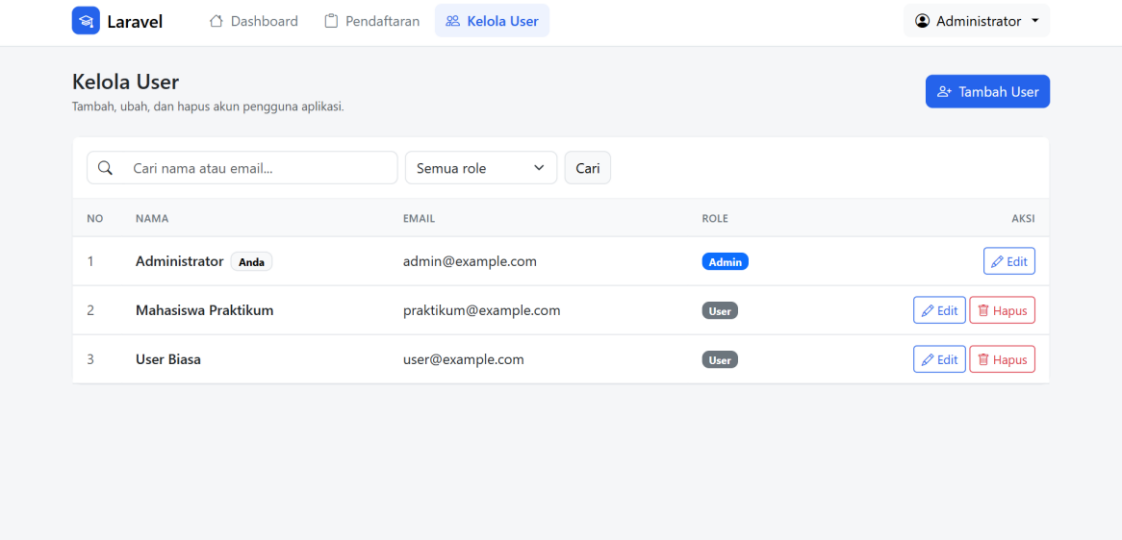
b. *Logout*, lalu *login* sebagai *admin* (admin@example.com / *password*) → *dashboard* menampilkan kartu jumlah *user* dan menu **Kelola *User***.




*Gambar 8.48 Dashboard admin dengan menu Kelola User*

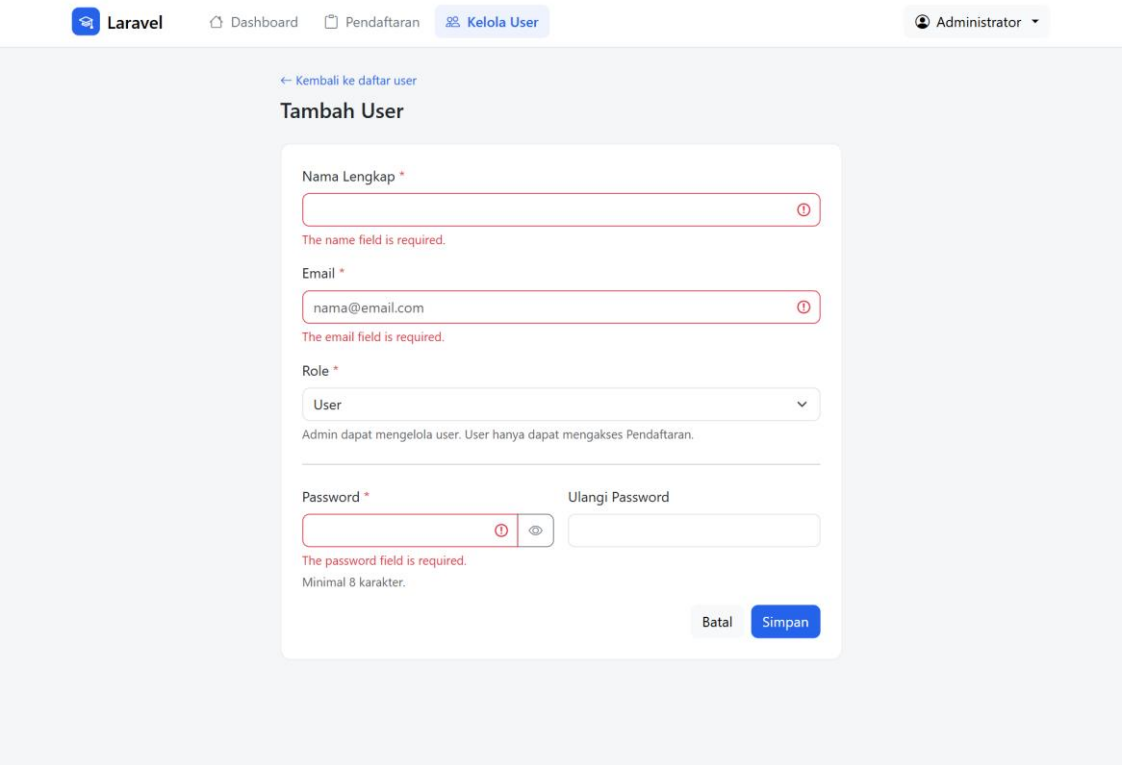
5. Uji modul UserManagement (CRUD *synchronous*) sebagai admin:

a. Klik menu **Kelola *User*** → tampil daftar *user* beserta *role*-nya. Akun yang sedang *login* diberi *badge* **Anda** dan tidak memiliki tombol Hapus.



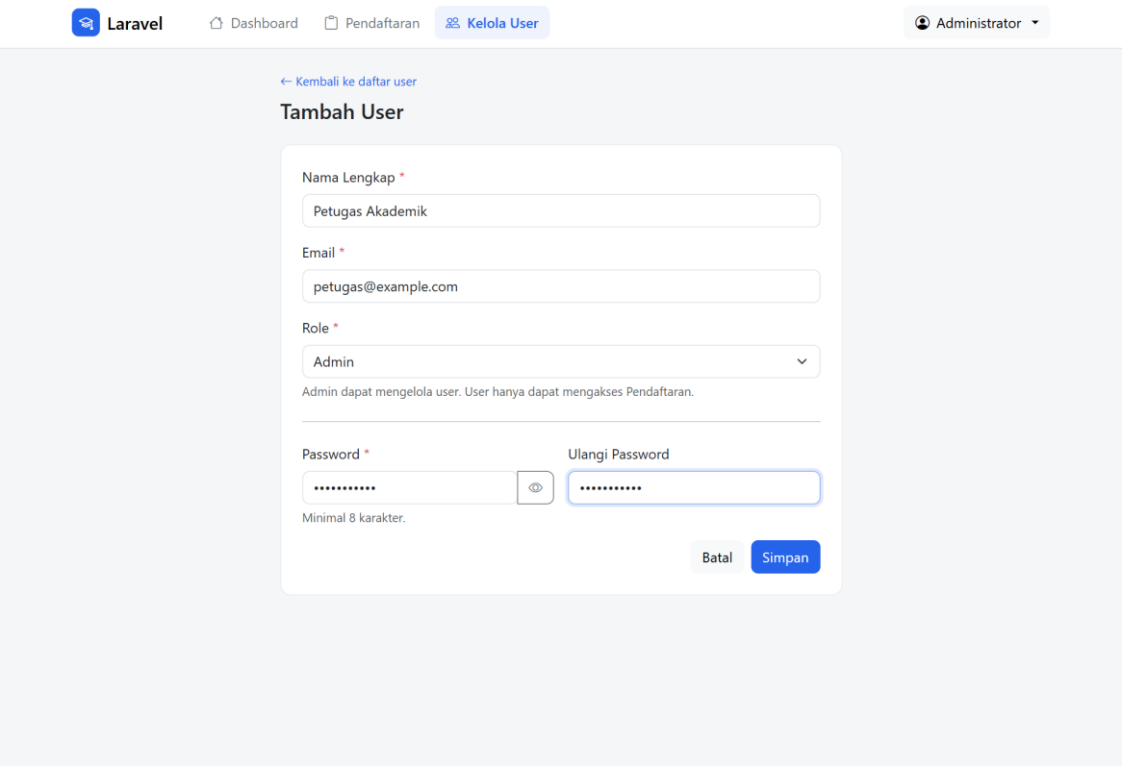

*Gambar 8.49 Daftar user pada modul UserManagement*

b. Klik **Tambah *User***, lalu klik **Simpan** tanpa mengisi *form* → pesan *error* muncul di bawah setiap *input*.

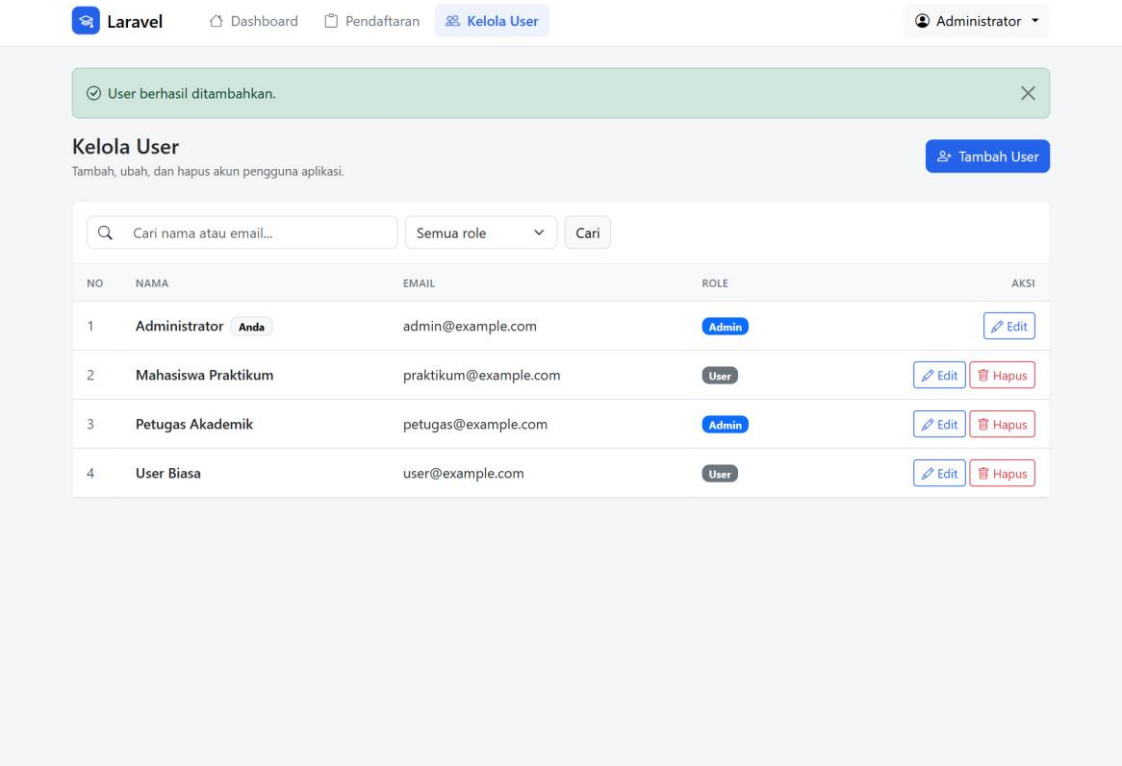



*Gambar 8.50 Pesan error validasi pada form tambah user*

c. Isi data *user* baru, pilih *role* ***Admin***, lalu klik **Simpan** → halaman dimuat ulang ke daftar *user* disertai pesan berhasil.

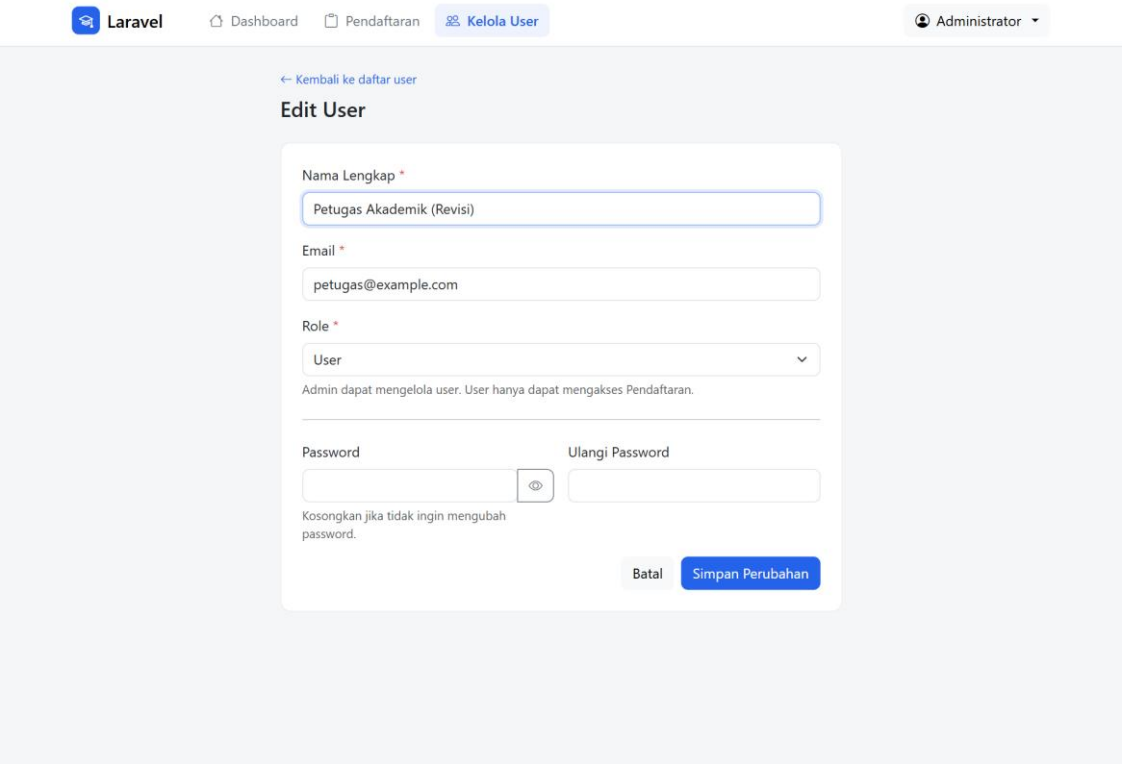



*Gambar 8.51 Form tambah user yang sudah diisi*

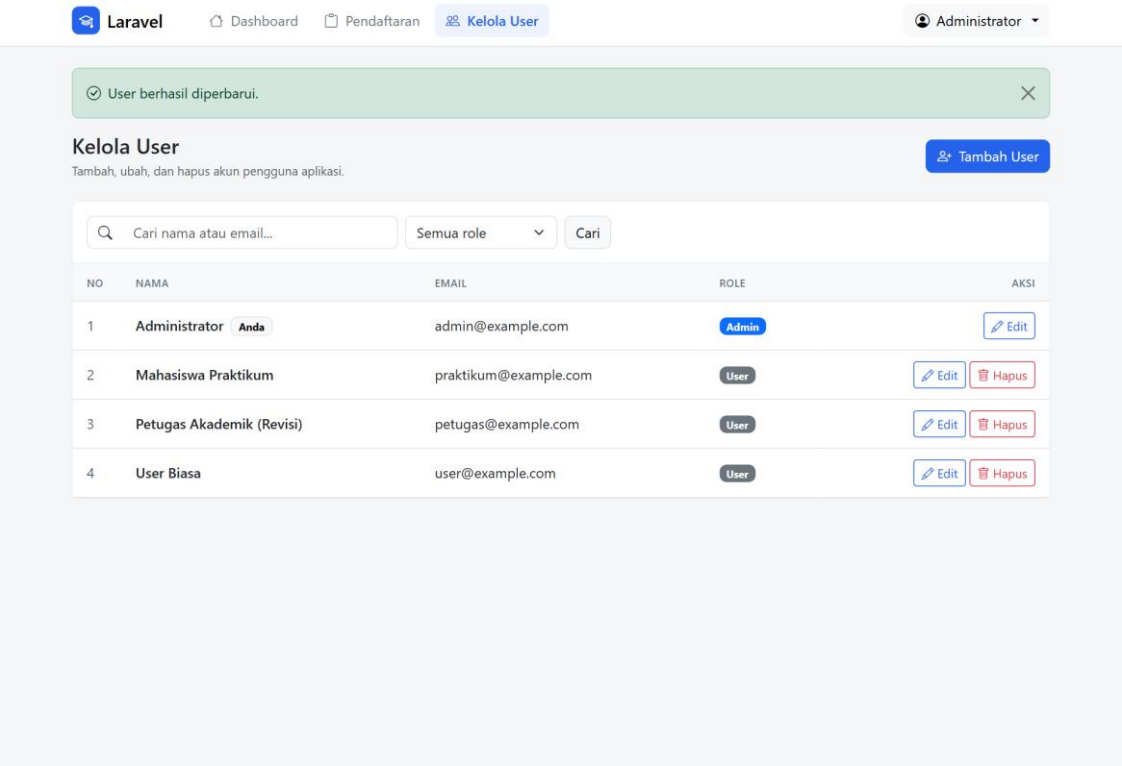



*Gambar 8.52 Pesan berhasil setelah user ditambahkan*

d. Klik **Edit** pada *user* tersebut, ubah nama dan *role*, kosongkan *password*, lalu klik **Simpan Perubahan** → data berubah dan *password* lama tetap berlaku.

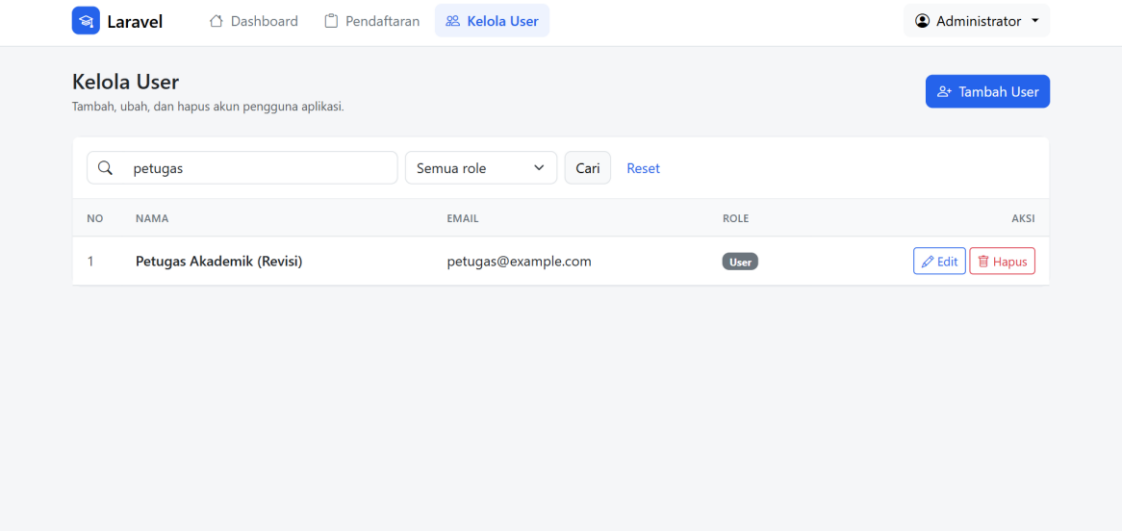



*Gambar 8.53 Form edit user*



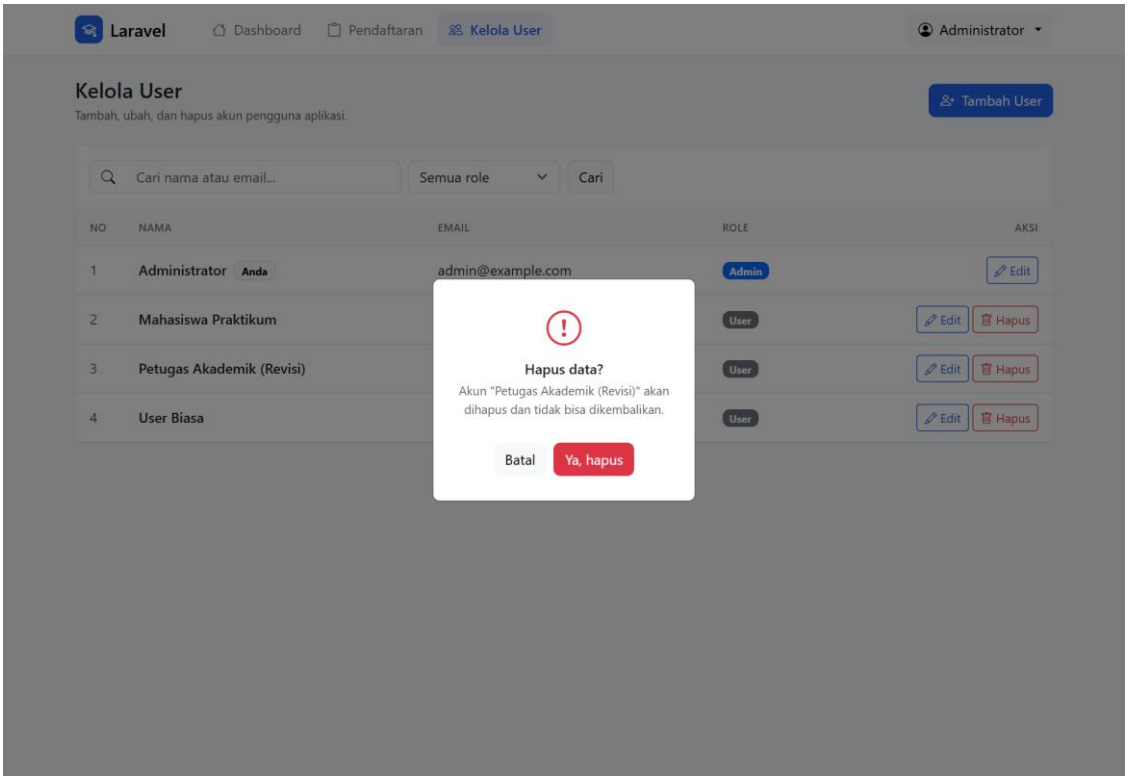

*Gambar 8.54 Pesan berhasil setelah user diperbarui*

e. Ketik kata kunci pada kotak pencarian lalu tekan Enter → daftar disaring berdasarkan nama atau *email*.

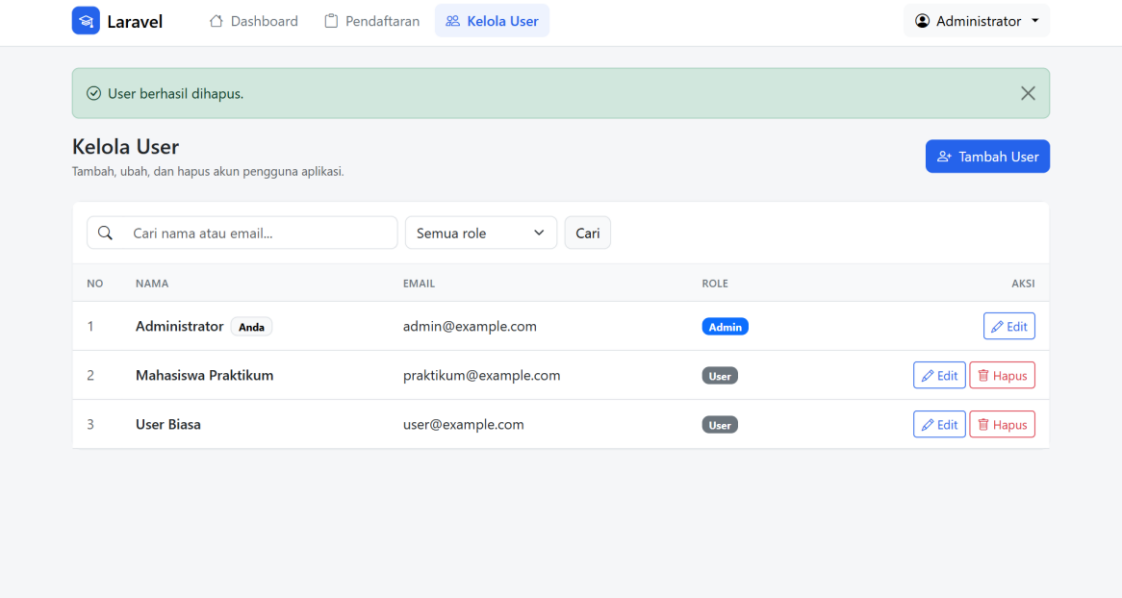



*Gambar 8.55 Pencarian user*

f. Klik **Hapus** → muncul *modal* konfirmasi. Klik **Ya, hapus** → *user* terhapus.

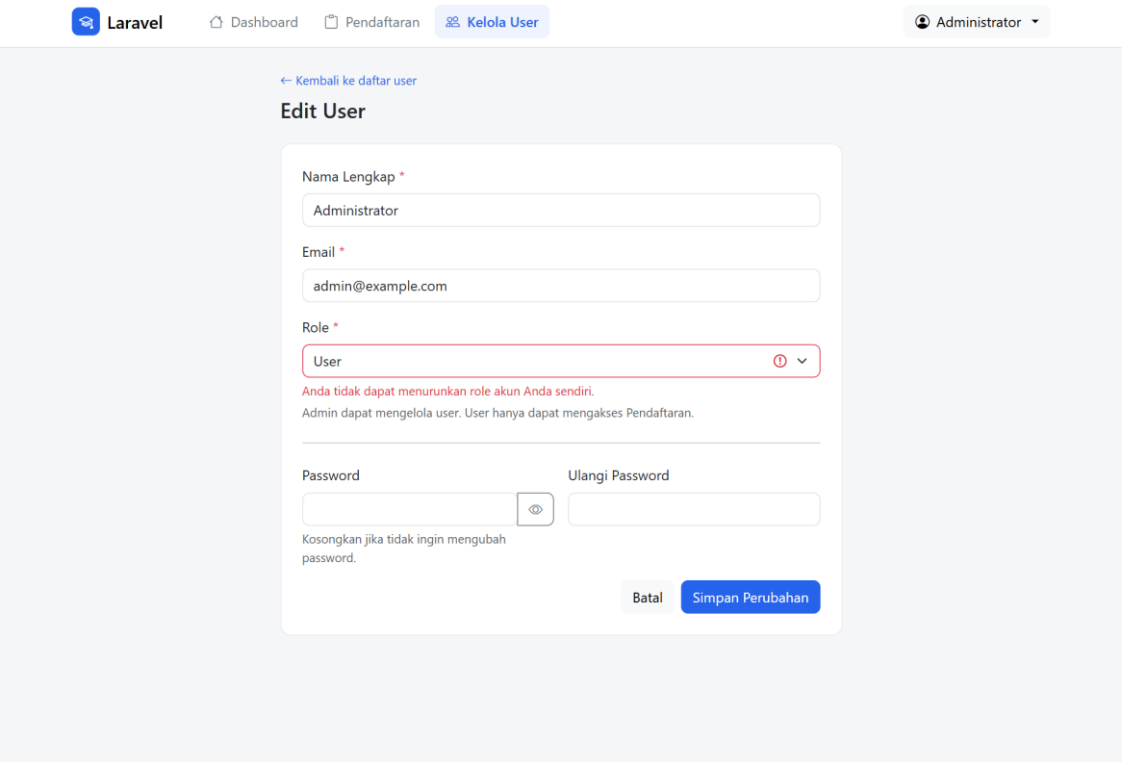



*Gambar 8.56 Modal konfirmasi hapus user*




*Gambar 8.57 Pesan berhasil setelah user dihapus*

g. Edit akun *admin* yang sedang *login* dan ubah *role*-nya menjadi *User* → perubahan ditolak.

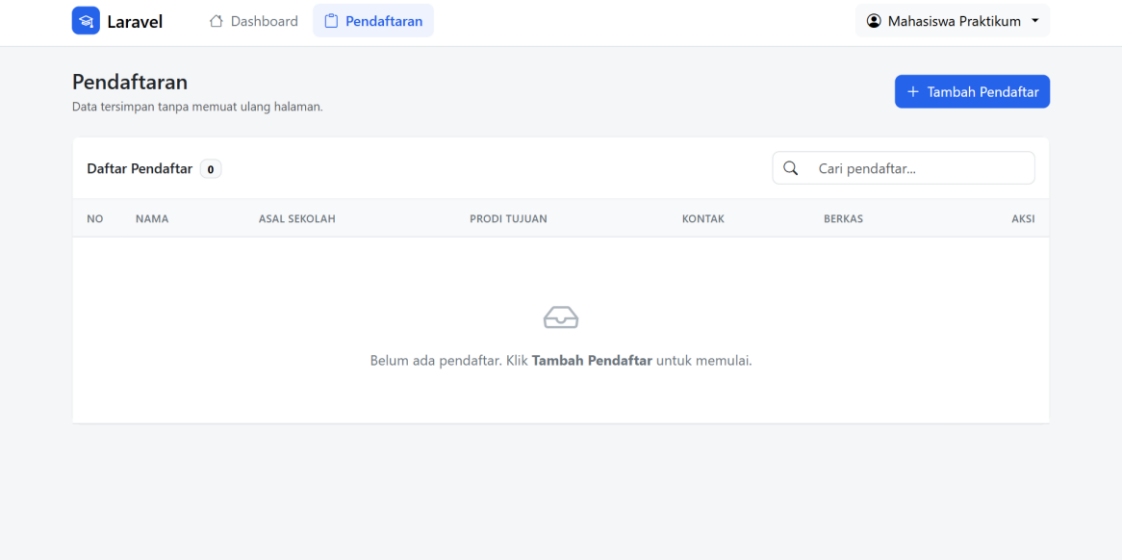



*Gambar 8.58 Admin tidak dapat menurunkan role akunnya sendiri*

Aturan Rule::when() pada UpdateUserRequest mencegah *admin* terkunci dari menu *admin*.

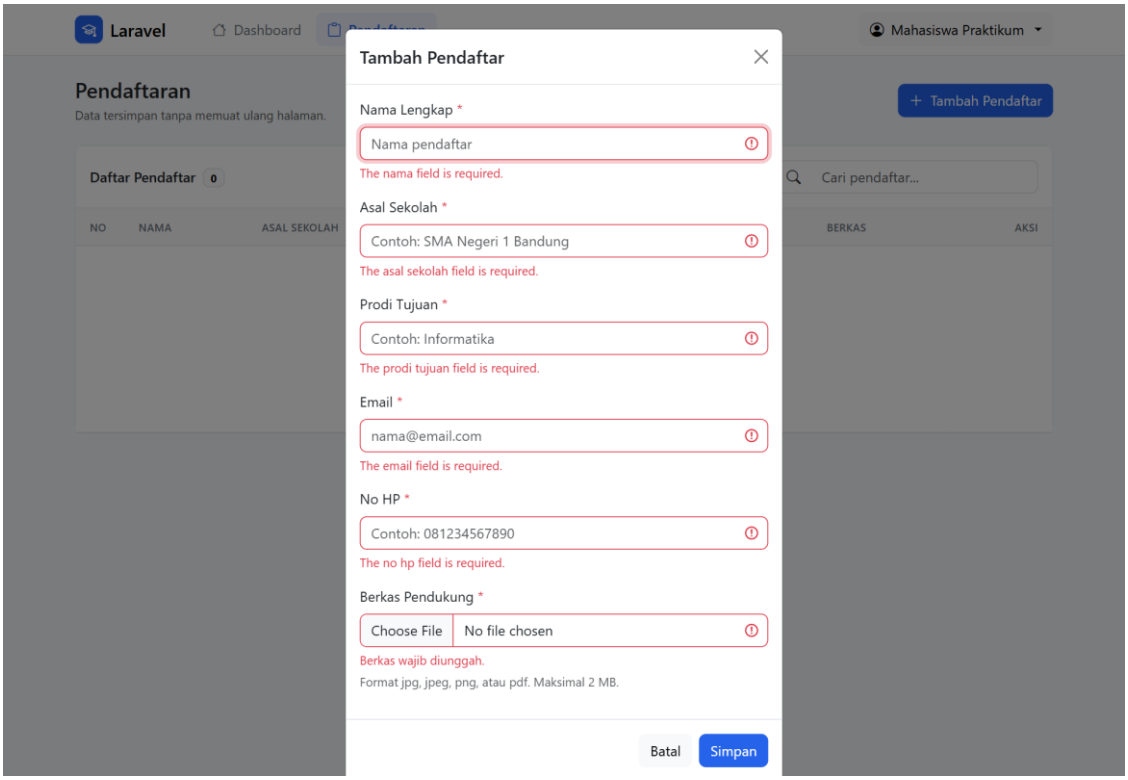
6. Uji modul Pendaftaran (CRUD AJAX dan upload berkas). Buka DevTools (F12) → *tab* ***Network*** → filter **Fetch/XHR** untuk melihat *request* AJAX yang dikirim tanpa memuat ulang halaman.

   a. *Login* sebagai *user*, lalu klik menu **Pendaftaran** → tabel menampilkan pesan data masih kosong.



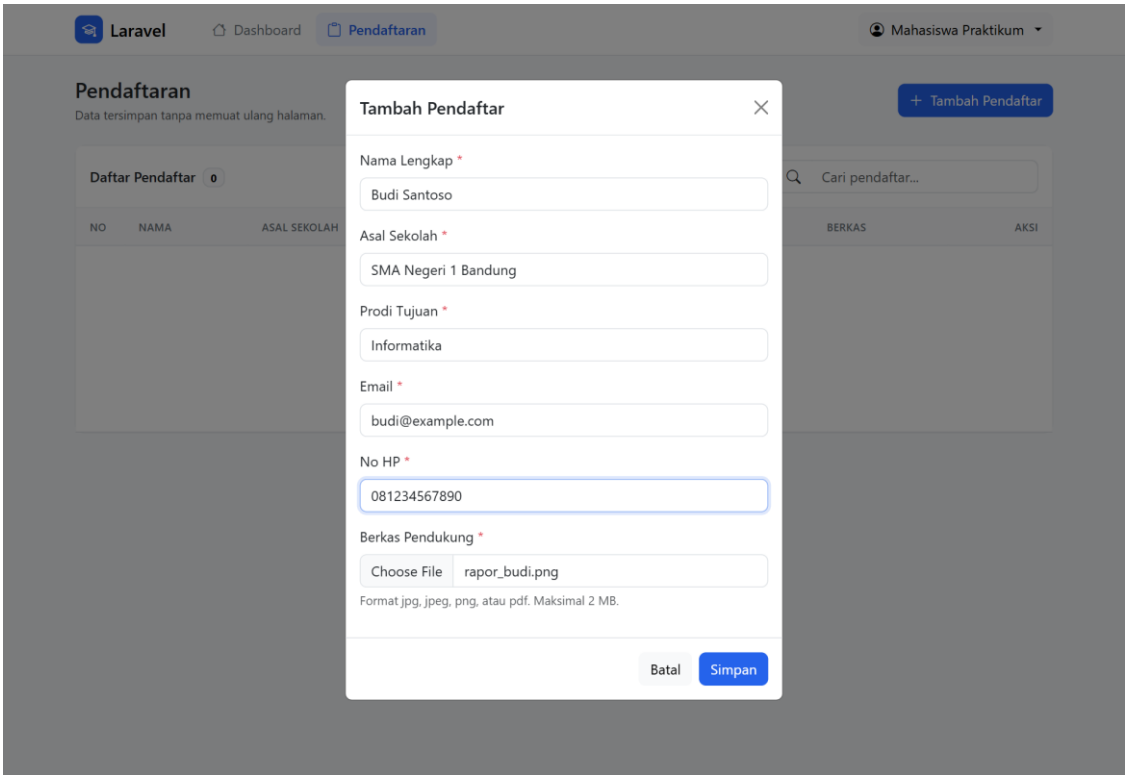

*Gambar 8.59 Halaman Pendaftaran masih kosong*

b. Klik **Tambah Pendaftar**, lalu klik **Simpan** tanpa mengisi *form* → *server* membalas kode 422 dan pesan *error* ditampilkan di bawah setiap *input* tanpa menutup *modal*. Input berkas juga ditandai karena berkas wajib diunggah saat menambah data.

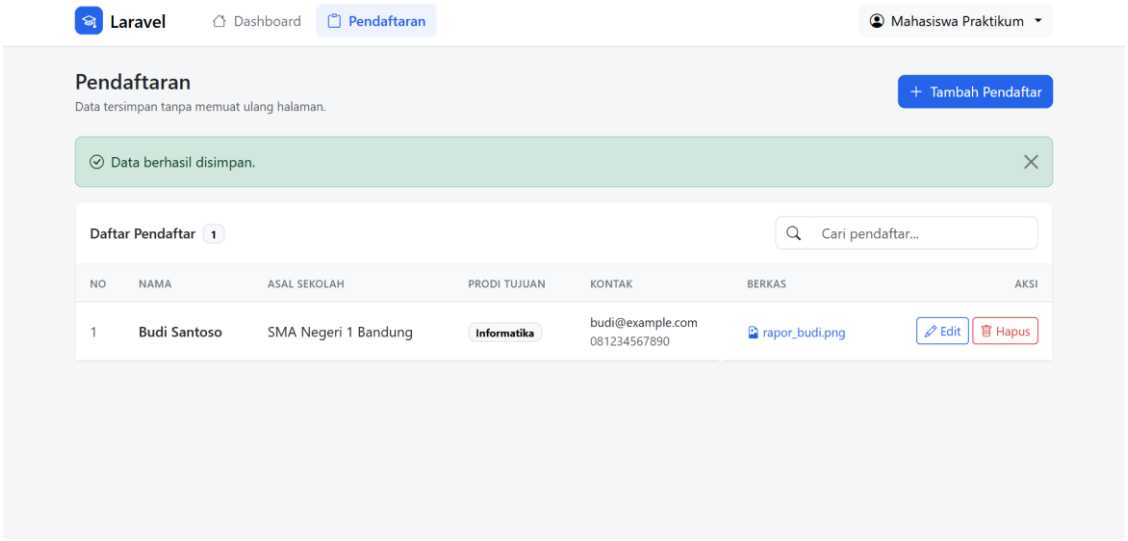



*Gambar 8.60 Pesan error validasi AJAX pada modal*

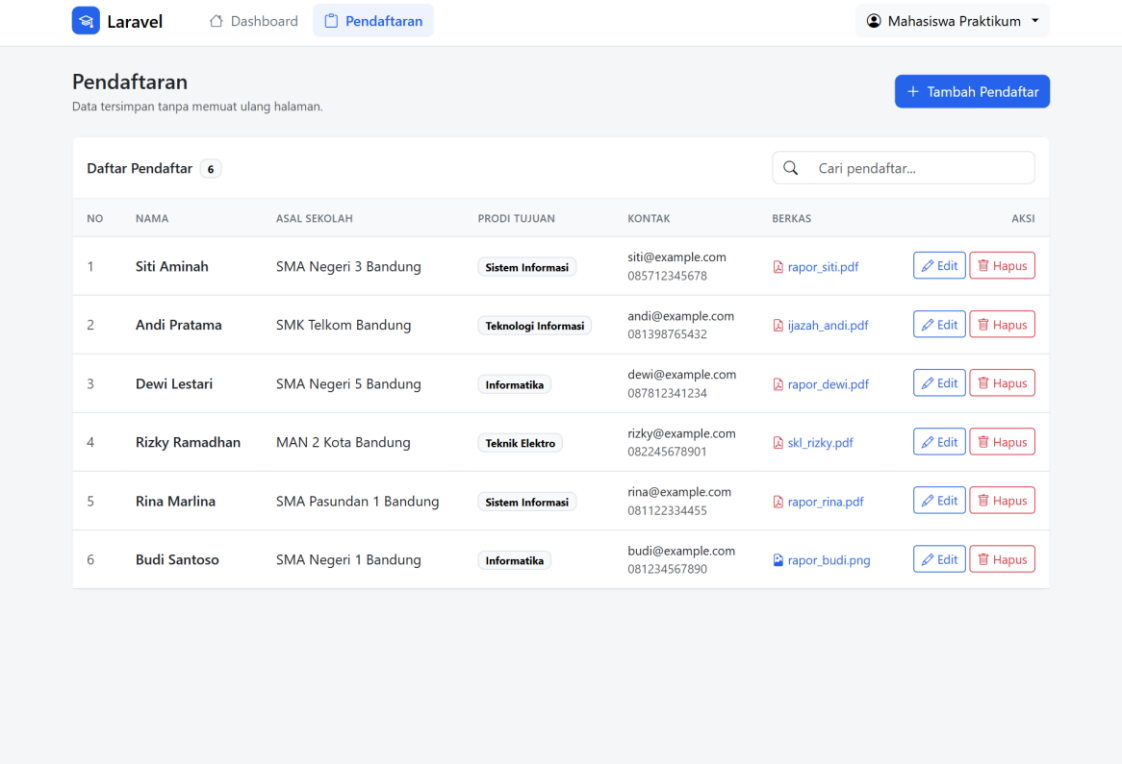
c. Isi seluruh data dan pilih berkas (misalnya rapor_budi.png), lalu klik **Simpan** → *modal* tertutup, tabel diperbarui, dan pesan berhasil muncul tanpa refresh halaman.




*Gambar 8.61 Modal tambah pendaftar yang sudah diisi beserta berkas*



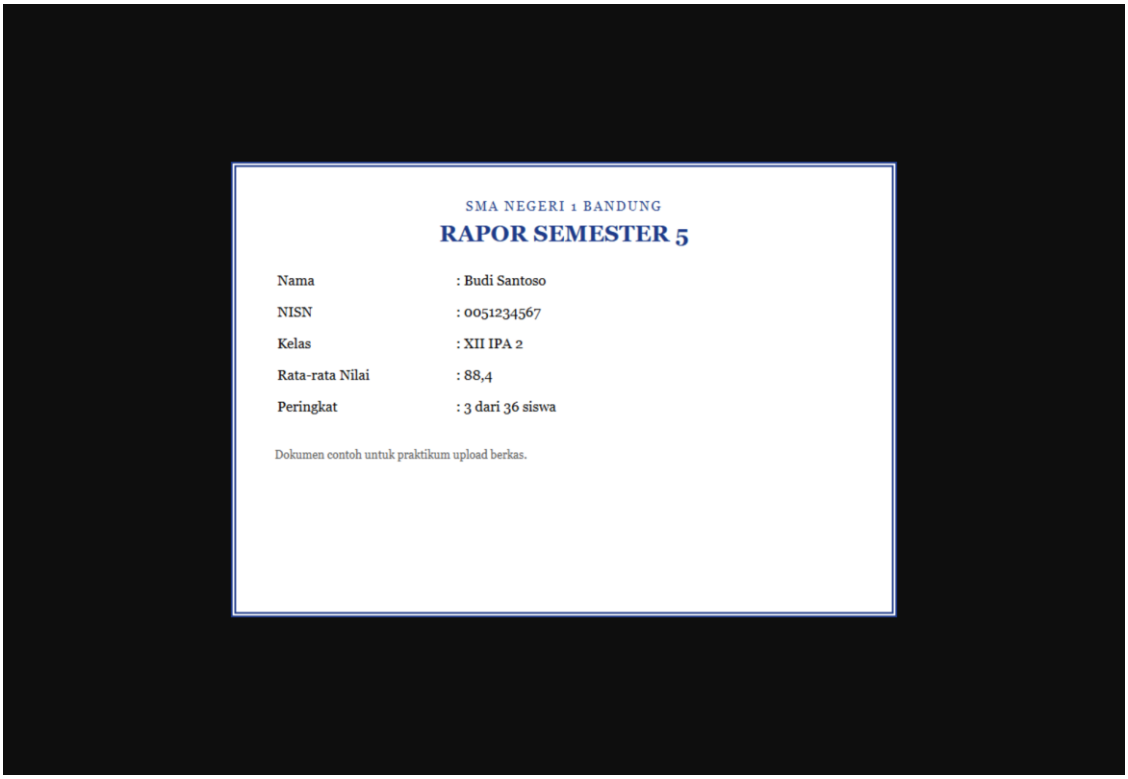

*Gambar 8.62 Pesan berhasil setelah data disimpan melalui AJAX*




*Gambar 8.63 Daftar pendaftar setelah beberapa data ditambahkan*

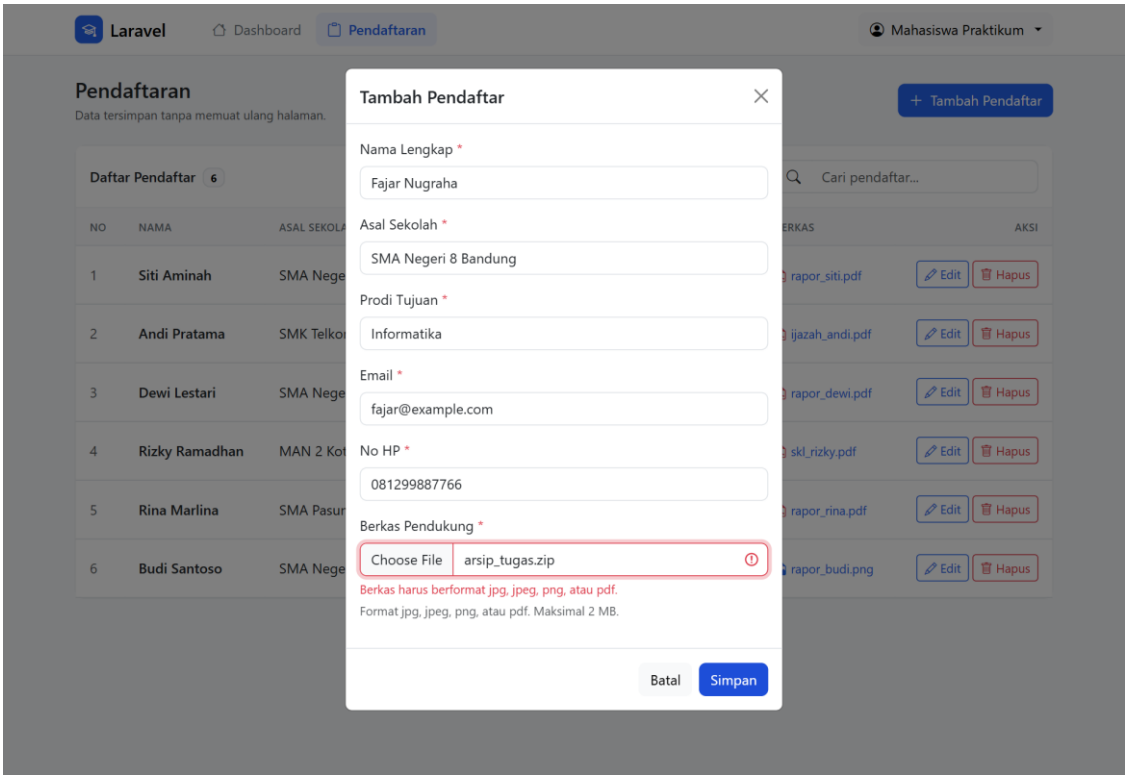
*Badge* di samping judul menampilkan jumlah seluruh data. Data diurutkan dari yang terbaru. Kolom Berkas menampilkan nama asli *file* yang diunggah.

d. Klik nama berkas pada kolom **Berkas** → berkas terbuka di *tab* baru melalui URL http://127.0.0.1:8000/storage/berkas_pendaftaran/xxxx.png. URL ini hanya dapat dibuka jika storage:link sudah dijalankan.




*Gambar 8.64 Berkas yang diunggah dibuka melalui URL storage*

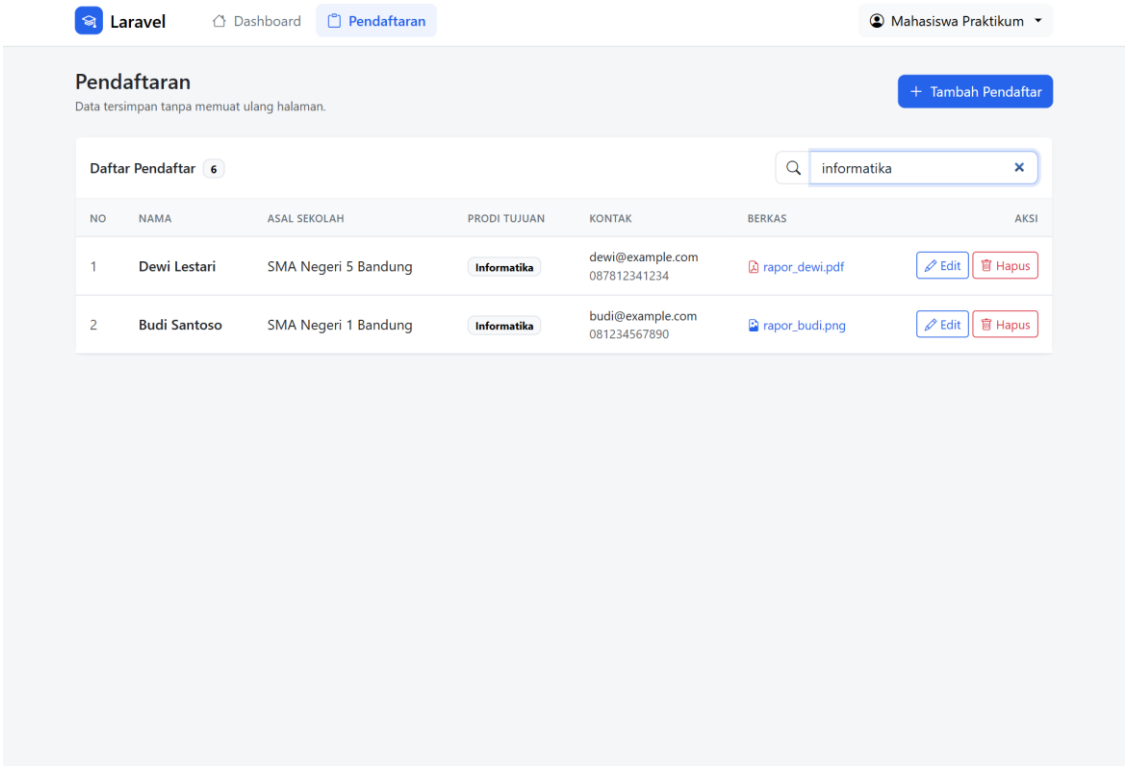
e. Klik **Tambah Pendaftar**, isi data, lalu pilih *file* berformat .zip dan klik **Simpan** → berkas ditolak dan pesan *error* muncul di bawah input berkas.




*Gambar 8.65 Pesan error saat mengunggah file .zip*

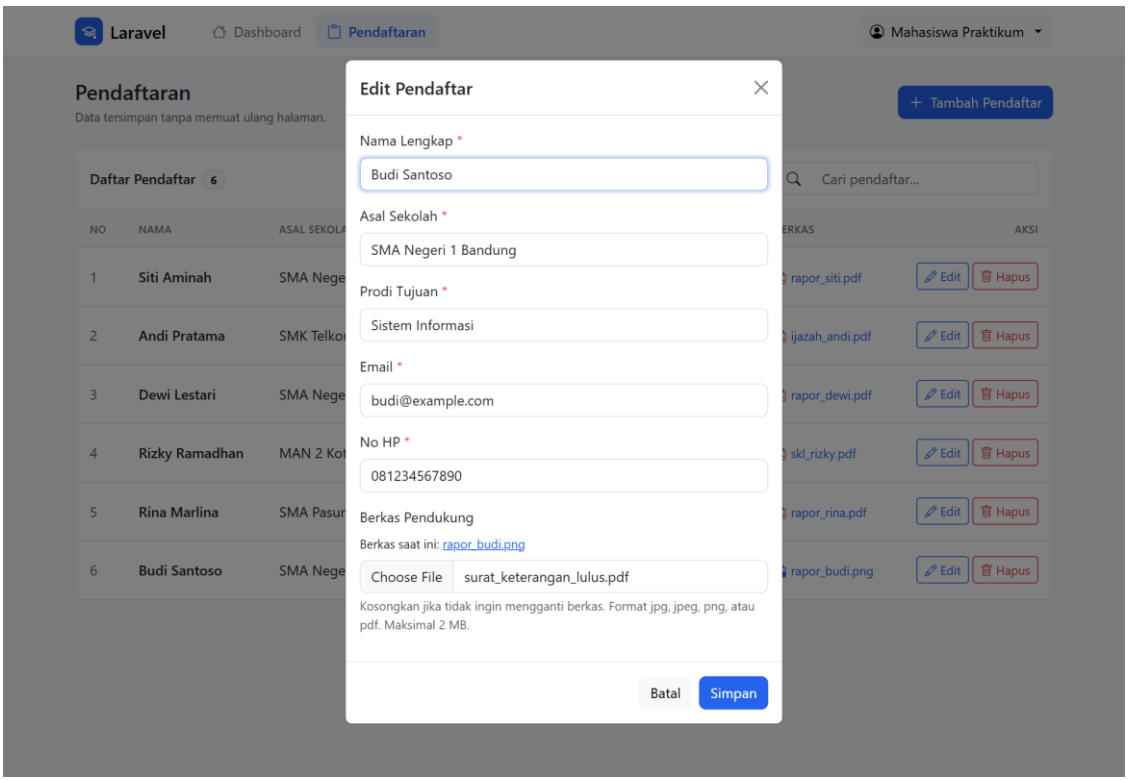
Validasi mimes menolak *file* .zip. Hal yang sama terjadi pada *file* berukuran lebih dari 2 MB karena aturan max:2048.

f. Ketik kata kunci pada kotak pencarian → tabel langsung tersaring saat mengetik, tanpa *request* baru ke *server*.

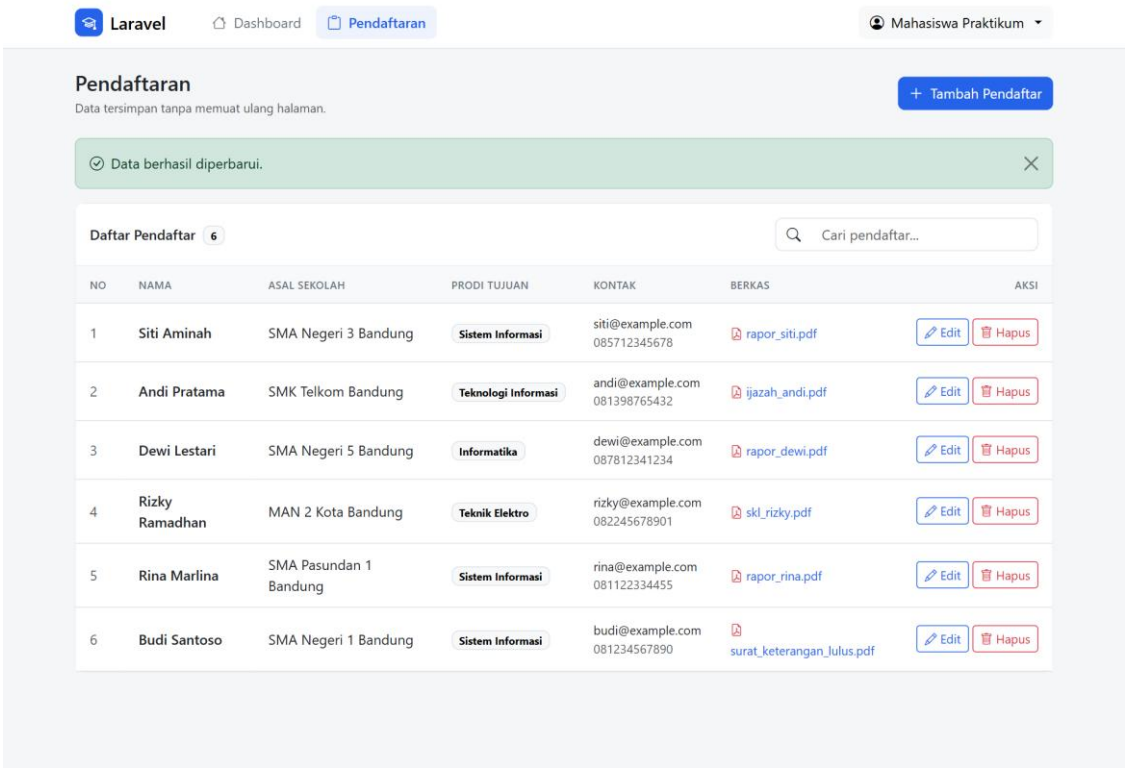



*Gambar 8.66 Pencarian instan pada tabel pendaftar*

g. Klik **Edit** pada data Budi Santoso → data diambil dengan AJAX dan *modal* menampilkan tautan **Berkas saat ini**. Ubah prodi tujuan, pilih berkas baru (surat_keterangan_lulus.pdf), lalu klik **Simpan** → baris pada tabel langsung berubah dan berkas lama dihapus dari *storage*.

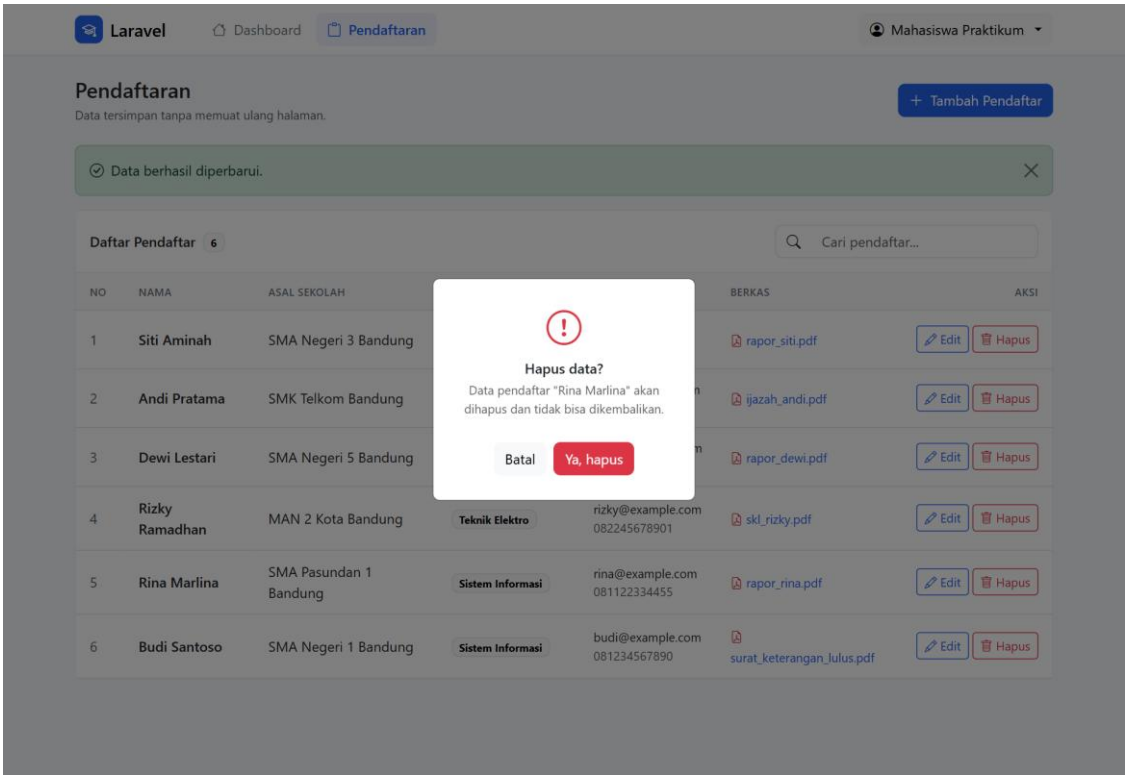



*Gambar 8.67 Modal edit pendaftar*

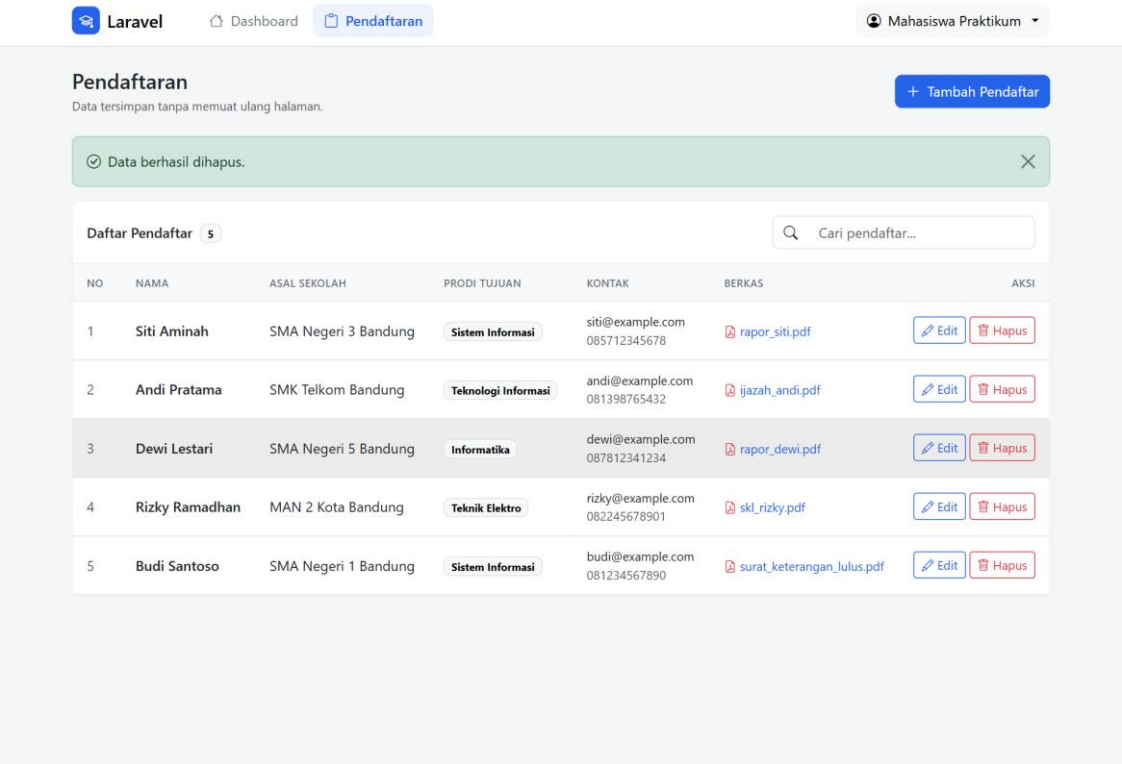



*Gambar 8.68 Pesan berhasil setelah data diperbarui*

h. Klik **Hapus** pada salah satu baris → muncul *modal* konfirmasi. Klik **Ya, hapus** → baris hilang dari tabel tanpa refresh halaman.



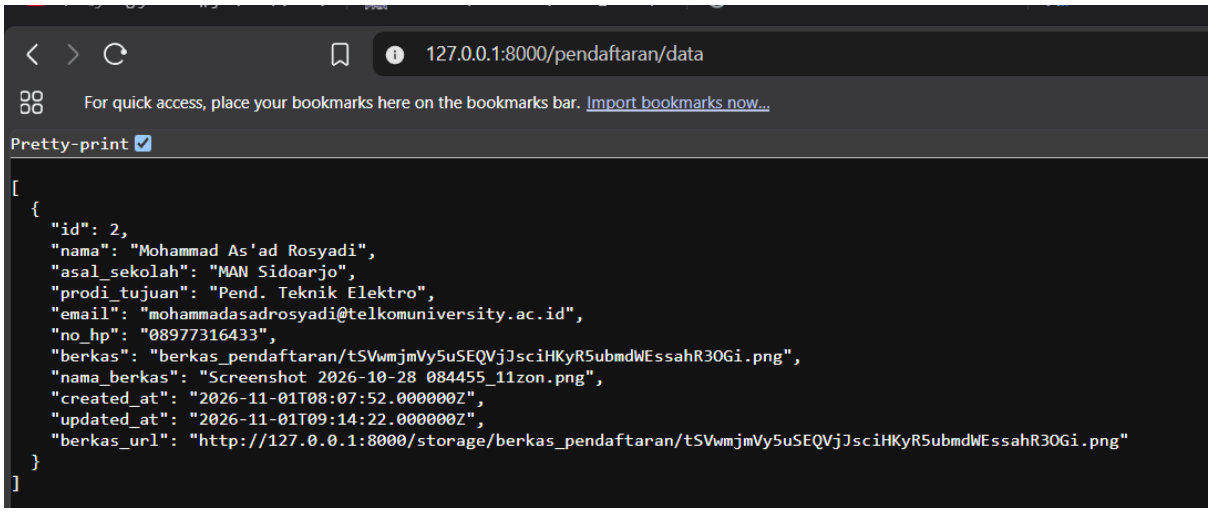

*Gambar 8.69 Modal konfirmasi hapus pendaftar*




*Gambar 8.70 Pesan berhasil setelah data dihapus*

Data Rina Marlina beserta berkasnya terhapus tanpa *refresh* halaman, dan jumlah data berkurang menjadi lima.

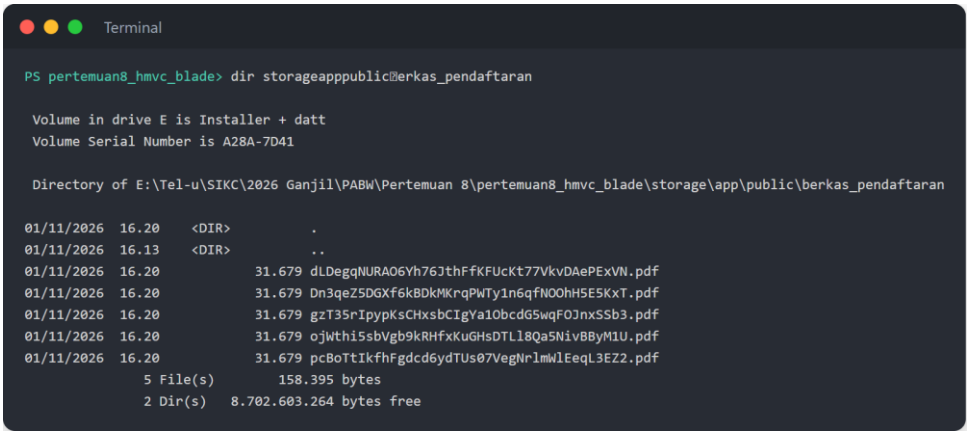
i. Buka http://127.0.0.1:8000/pendaftaran/data secara langsung → *browser* menampilkan data mentah berformat JSON. Data inilah yang diambil oleh fungsi loadData(). Setiap data kini memiliki berkas, nama_berkas, dan berkas_url.




*Gambar 8.71 Respons JSON dari route pendaftaran.data*

j. Buka *folder* storage/app/public/berkas_pendaftaran, misalnya dengan perintah dir → jumlah *file* sama dengan jumlah data pendaftar. Berkas lama milik Budi Santoso dan berkas milik Rina Marlina sudah terhapus.

```
dir storage\app\public\berkas_pendaftaran
```




*Gambar 8.72 Isi folder storage/app/public/berkas_pendaftaran*

*File* tersimpan dengan nama acak hasil store(), sedangkan *database* hanya menyimpan *path* (kolom berkas) dan nama asli *file* (kolom nama_berkas).

### 8.4.6 Pengujian Otomatis dengan Pest

Selain diuji melalui *browser*, seluruh fitur diuji secara otomatis menggunakan Pest. *File* pengujian berada di *folder* tests/Feature dan menggunakan *trait* RefreshDatabase (diaktifkan pada tests/Pest.php) sehingga setiap pengujian berjalan pada *database* kosong. Jalankan perintah berikut:

```
php artisan test
```




*Gambar 8.73 Hasil pengujian otomatis (Pest)*

Pengujian mencakup *redirect* tamu, *register* dan *login*, pembatasan percobaan *login*, *middleware role* (403 untuk *user* di halaman *admin*), CRUD *user* oleh *admin*, serta CRUD

pendaftaran melalui *request* JSON termasuk validasi 422 dan error 404, serta upload berkas (penolakan format dan ukuran, penggantian berkas, dan penghapusan berkas).

### 8.4.7 Perbandingan CRUD *Synchronous* dan CRUD AJAX

| Aspek | *Synchronous* (UserManagement) | AJAX (Pendaftaran) |
|---|---|---|
| Respons *controller* | view() atau redirect()->with(...) | response()->json([...]) |
| Perilaku halaman | Dimuat ulang setiap aksi | Tidak dimuat ulang; tabel diperbarui JavaScript |
| Pesan berhasil | *Session flash* (session('success')) | Alert dibuat JavaScript dari JSON |
| *Error* validasi | *Redirect* kembali + @error di Blade | Kode 422 + JSON *errors* ditampilkan jQuery |
| *Token* CSRF | @csrf di dalam *form* | *Header* X-CSRF-TOKEN melalui $.ajaxSetup() |
| *Form* tambah/edit | Halaman terpisah (create, edit) | Satu *modal* Bootstrap |
| Upload berkas | Form dengan enctype="multipart/form-data" (seperti Modul 7) | FormData + processData: false + contentType: false |
| Kelebihan | Sederhana, mudah di-debug, tetap berjalan tanpa JavaScript | Lebih cepat dan nyaman, tidak kehilangan posisi halaman |

## 8.5 Tugas Praktikum

1. Perpustakaan kampus membutuhkan aplikasi **SiPustaka** untuk mengelola data buku dan peminjaman. Petugas (*admin*) mengelola data buku dan akun pengguna, sedangkan anggota (*user*) dapat melihat daftar buku dan mencatat peminjaman. Buatlah aplikasi SiPustaka dari awal menggunakan *project* Laravel baru dengan arsitektur HMVC dan ketentuan sebagai berikut:
   - **Persiapan project:** buat *project* Laravel baru bernama sipustaka_hmvc, buat *database* db_sipustaka, pasang nwidart/laravel-modules, dan atur merge-plugin pada composer.json.
   - ***Authentication* di luar HMVC:** buat fitur *register*, *login*, dan *logout* secara manual. Tambahkan kolom *role* (*admin* dan *user*) pada tabel *users*, serta *seeder* untuk minimal satu akun *admin* dan satu akun *user*.
   - **Authorization:** buat *middleware role* dan daftarkan aliasnya di bootstrap/app.php. Akun hasil *register* wajib berperan *user*.
   - **Modul Buku (CRUD *synchronous*):** minimal lima item, misalnya judul, penulis, penerbit, tahun_terbit, dan stok. *Admin* dapat menambah, mengubah, dan menghapus buku, sedangkan *user* hanya dapat melihat daftar buku. Lengkapi dengan pencarian dan *pagination*.
   - **Modul Peminjaman (CRUD AJAX):** minimal lima item, misalnya nama_peminjam, judul_buku, tanggal_pinjam, tanggal_kembali, dan status (Dipinjam/Dikembalikan). Seluruh operasi tambah, tampil, ubah, dan hapus dilakukan tanpa refresh halaman dan dapat diakses oleh *admin* maupun *user*. Tambahkan satu input *upload file* (misalnya foto bukti peminjaman, jpg/png/pdf maksimal 2 MB) yang dapat dibuka

kembali melalui tautan, diganti saat edit, dan ikut terhapus dari *storage* ketika data dihapus.

- **Modul UserManagement:** *admin* dapat menambah, mengubah (termasuk *role*), dan menghapus *user*, tetapi tidak dapat menghapus atau menurunkan *role* akunnya sendiri.
- **Validasi dan pesan:** gunakan *Form Request* pada setiap modul, tampilkan pesan *error* di bawah setiap *input* (termasuk *error* 422 pada *form* AJAX), serta pesan berhasil setelah tambah, ubah, dan hapus data.
- **Pembuktian:** tampilkan *error* 403 ketika *user* membuka halaman khusus *admin*, serta tangkapan layar *tab Network* (Fetch/XHR) pada DevTools yang membuktikan modul Peminjaman menggunakan AJAX. Tunjukkan juga isi *folder* storage/app/public dan kolom *path* berkas pada phpMyAdmin.

2. Terapkan arsitektur HMVC, *authorization* berbasis *role*, dan AJAX pada *project* Tugas Akhir Pemrograman *Web* Saudara, dengan ketentuan: (a) minimal dua modul HMVC yang sesuai tema *project*, (b) satu modul menggunakan CRUD *synchronous* dan satu modul menggunakan CRUD AJAX, masing-masing minimal lima item, (c) minimal dua *role* pengguna dengan hak akses yang berbeda dan dilindungi *middleware*, serta (d) seluruh halaman pengelolaan data hanya dapat diakses setelah *login*. Modul AJAX wajib memiliki minimal satu input *upload file* yang dapat dibuka kembali.

## 8.6 Format Pengumpulan Tugas

Laporan disusun secara berurutan dengan sistematika berikut.

1. **Halaman Sampul (*Cover*)**

   Memuat judul praktikum, nama mahasiswa, NIM, kelas, mata kuliah, nama dosen pengampu, program studi, serta tahun akademik. Gunakan logo institusi dan tata letak yang rapi.

2. **Tujuan Praktikum**

   Menuliskan tujuan yang ingin dicapai pada praktikum ini dengan kalimat sendiri, bukan menyalin ulang tujuan pembelajaran pada modul.

3. **Alat dan Bahan**

   Menyebutkan spesifikasi perangkat keras yang digunakan serta perangkat lunak beserta versinya, misalnya XAMPP/Laragon, MySQL/MariaDB, phpMyAdmin, Composer, PHP, Laravel *Framework*, nwidart/laravel-modules, Bootstrap, jQuery, Visual Studio Code, dan Git.

4. **Langkah-Langkah Praktikum**

   Menguraikan tahapan pengerjaan secara runtut, mulai dari pemasangan HMVC dan pembuatan modul, pembuatan *authentication* di luar modul, *middleware role*, CRUD *synchronous*, CRUD AJAX, hingga pengujian melalui *browser*. Setiap langkah disertai potongan kode yang relevan dan penjelasan singkat.

5. **Hasil dan Dokumentasi Program**

   Menampilkan tangkapan layar hasil aplikasi yang berjalan, sekurang-kurangnya halaman *register* dan *login* (termasuk pesan *error*), *dashboard admin* dan *user*, *error* 403, CRUD *synchronous* (tambah, edit, hapus, pesan *error*), CRUD AJAX (*modal*, *error* 422, tabel yang diperbarui tanpa refresh), *tab Network* pada DevTools, serta hasil php artisan

route:list dan php artisan module:list. Setiap gambar diberi nomor, keterangan gambar, dan penjelasan singkat. Sertakan juga *form* dengan input berkas, pesan *error* ketika berkas tidak sesuai, berkas yang dibuka melalui tautan, serta isi *folder* storage/app/public.

6. **Analisis dan Pembahasan**

   Membahas cara kerja HMVC, perbedaan *authentication* dan *authorization*, cara kerja *middleware role*, perbedaan CRUD *synchronous* dan AJAX, serta kendala atau *error* yang ditemukan (misalnya class modul tidak ditemukan karena lupa composer dump-autoload, *error* 419 karena *token* CSRF tidak dikirim, *error* 422, atau *error* 403) beserta cara penyelesaiannya. Bahas juga alasan penggunaan FormData pada *upload* AJAX dan alasan berkas disimpan di *storage* sedangkan *database* hanya menyimpan *path*.

7. **Kesimpulan**

   Menuliskan ringkasan capaian praktikum dan pemahaman yang diperoleh, disusun dalam beberapa kalimat yang menjawab tujuan praktikum.

8. **Lampiran berisi *Link* Repositori GitHub**

   Mencantumkan tautan repositori GitHub yang dapat diakses oleh dosen pengampu dan memuat kode program lengkap, termasuk *folder Modules*, *middleware*, *controller authentication*, *route*, dan *view*.

9. **Presentasi Hasil Praktikum**

   Mahasiswa mempresentasikan hasil praktikum di kelas, meliputi demonstrasi aplikasi yang berjalan (*login* sebagai *admin* dan *user*, CRUD *synchronous*, dan CRUD AJAX) dan penjelasan kode program, serta menjawab pertanyaan dari dosen pengampu.

**TERIMAKASIH**